

Utils

Smart contract utils utilities and implementations

 Copy Markdown Open in Claude

Miscellaneous contracts and libraries containing utility functions you can use to improve security, work with new data types, or safely use low-level primitives.

- [Math](#), [SignedMath](#): Implementation of various arithmetic functions.
- [SafeCast](#): Checked downcasting functions to avoid silent truncation.
- [Nonces](#): Utility for tracking and verifying address nonces that only increment.
- [NoncesKeyed](#): Alternative to [Nonces](#), that support keyed nonces following [ERC-4337 specifications](#).
- [Pausable](#): A common emergency response mechanism that can pause functionality while a remediation is pending.
- [ReentrancyGuard](#): A modifier that can prevent reentrancy during certain functions.
- [ReentrancyGuardTransient](#): Variant of [ReentrancyGuard](#) that uses transient storage ([EIP-1153](#)).
- [ERC165](#), [ERC165Checker](#): Utilities for inspecting interfaces supported by contracts.
- [Accumulators](#): A library for merging an arbitrary dynamic number of bytes buffers.
- [BitMaps](#): A simple library to manage boolean value mapped to a numerical index in an efficient way.

- [Checkpoints](#): A data structure to store values mapped to a strictly increasing key. Can be used for storing and accessing values over time.

-
- [CircularBuffer](#): A data structure to store the last N values pushed to it.
-
- [DoubleEndedQueue](#): An implementation of a [double ended queue](#) whose values can be added or removed from both sides. Useful for FIFO and LIFO structures.
 - [EnumerableMap](#): A type like Solidity's [mapping](#), but with key-value *enumeration*: this will let you know how many entries a mapping has, and iterate over them (which is not possible with [mapping](#)).
 - [EnumerableSet](#): Like [EnumerableMap](#), but for [sets](#). Can be used to store privileged accounts, issued IDs, etc.
 - [Heap](#): A library that implements a [binary heap](#) in storage.
 - [MerkleTree](#): A library with [Merkle Tree](#) data structures and helper functions.
 - [Address](#): Collection of functions for overloading Solidity's [address](#) type.
 - [Arrays](#): Collection of functions that operate on [arrays](#).
 - [Base58](#): On-chain base58 encoding and decoding.
 - [Base64](#): On-chain base64 and base64URL encoding according to [RFC-4648](#).
 - [Blockhash](#): A library for accessing historical block hashes beyond the standard 256 block limit utilizing EIP-2935's historical blockhash functionality.
 - [Bytes](#): Common operations on bytes objects.
 - [CAIP2](#), [CAIP10](#): Libraries for formatting and parsing CAIP-2 and CAIP-10 identifiers.
 - [Calldata](#): Helpers for manipulating calldata.
 - [Comparators](#): A library that contains comparator functions to use with the [Heap](#) library.
 - [Context](#): A utility for abstracting the sender and calldata in the current execution context.
 - [Create2](#): Wrapper around the [CREATE2](#) [EVM opcode](#) for safe use without having to deal with low-level assembly.
 - [InteroperableAddress](#): Library for formatting and parsing ERC-7930 interoperable addresses.

- **LowLevelCall**: Collection of functions to perform calls with low-level assembly.
- **Memory**: A utility library to manipulate memory.
- **Multicall**: Abstract contract with a utility to allow batching together multiple calls in a single transaction. Useful for allowing EOAs to perform multiple operations at once.
- **Packing**: A library for packing and unpacking multiple values into bytes32.
- **Panic**: A library to revert with [Solidity panic codes](#).
- **RelayedCall**: A library for performing calls that use minimal and predictable relayers to hide the sender.
- **RLP**: Library for encoding and decoding data in Ethereum's Recursive Length Prefix format.
- **ShortStrings**: Library to encode (and decode) short strings into (or from) a single bytes32 slot for optimizing costs. Short strings are limited to 31 characters.
- **SlotDerivation**: Methods for deriving storage slot from ERC-7201 namespaces as well as from constructions such as mapping and arrays.
- **StorageSlot**: Methods for accessing specific storage slots formatted as common primitive types.
- **Strings**: Common operations for strings formatting.
- **Time**: A library that provides helpers for manipulating time-related objects, including a **Delay** type.
- **TransientSlot**: Primitives for reading from and writing to transient storage (only value types are currently supported).

i Because Solidity does not support generic types, [EnumerableMap](#) and [EnumerableSet](#) are specialized to a limited number of key-value types.

Math

[Math](#)

[SignedMath](#)

Security

[Nonces](#)

[NoncesKeyed](#)

[Pausable](#)

[ReentrancyGuard](#)

[ReentrancyGuardTransient](#)

Introspection

This set of interfaces and contracts deal with [type introspection](#) of contracts, that is, examining which functions can be called on them. This is usually referred to as a contract's *interface*.

Ethereum contracts have no native concept of an interface, so applications must usually simply trust that they are not making an incorrect call. For trusted setups this is a non-issue, but often unknown and untrusted third-party addresses need to be interacted with. There may not even be any direct calls to them! (e.g. ERC-20 tokens may be sent to a contract that lacks a way to transfer them out of it, locking them forever). In these cases, a contract *declaring* its interface can be very helpful in preventing errors.

[IERC165](#)

[ERC165](#)

[ERC165Checker](#)

Data Structures

[BitMaps](#)[Checkpoints](#)[CircularBuffer](#)[DoubleEndedQueue](#)[EnumerableMap](#)[EnumerableSet](#)[Heap](#)[MerkleTree](#)

Libraries

[Address](#)[Arrays](#)[Base58](#)[Base64](#)[Blockhash](#)[Bytes](#)[CAIP10](#)[CAIP2](#)[Calldata](#)

Context

[Create2](#)

[InteroperableAddress](#)

[LowLevelCall](#)

[Memory](#)

[Multicall](#)

[Packing](#)

[Panic](#)

[RelayedCall](#)

[RLP](#)

[ShortStrings](#)

[SlotDerivation](#)

[StorageSlot](#)

[Strings](#)

[Time](#)

[TransientSlot](#)



```
import "@openzeppelin/contracts/utils/Address.sol";
```



Collection of functions related to the address type

Functions

- [sendValue\(recipient, amount\)](#)
- [functionCall\(target, data\)](#)
- [functionCallWithValue\(target, data, value\)](#)
- [functionStaticCall\(target, data\)](#)
- [functionDelegateCall\(target, data\)](#)
- [verifyCallResultFromTarget\(target, success, returndata\)](#)
- [verifyCallResult\(success, returndata\)](#)

Errors

- [AddressEmptyCode\(target\)](#)

sendValue(address payable recipient, uint256 amount)

internal #

Replacement for Solidity's `transfer`: sends `amount` wei to `recipient`, forwarding all available gas and reverting on errors.

[EIP1884](#) increases the gas cost of certain opcodes, possibly making contracts go over the 2300 gas limit imposed by `transfer`, making them unable to receive funds via `transfer`. [Address.sendValue](#) removes this limitation.

⚠ because control is transferred to `recipient`, care must be taken to not create reentrancy vulnerabilities. Consider using [ReentrancyGuard](#) or the [checks-effects-interactions pattern](#).

`functionCall(address target, bytes data) → bytes`

internal #

Performs a Solidity function call using a low level `call`. A plain `call` is an unsafe replacement for a function call: use this function instead.

If `target` reverts with a revert reason or custom error, it is bubbled up by this function (like regular Solidity function calls). However, if the call reverted with no returned reason, this function reverts with a `Errors.FailedCall` error.

Returns the raw returned data. To convert to the expected return value, use [abi.decode](#).

Requirements:

- `target` must be a contract.
- calling `target` with `data` must not revert.

`functionCallWithValue(address target, bytes data, uint256 value) → bytes` internal #

Same as [functionCall](#), but also transferring `value` wei to `target`.

Requirements:

- the calling contract must have an ETH balance of at least `value`.
- the called Solidity function must be `payable`.

`functionStaticCall(address target, bytes data) → bytes`

internal #

Same as `functionCall`, but performing a static call.



`functionDelegateCall(address target, bytes data) → bytes`

internal #

Same as `functionCall`, but performing a delegate call.

`verifyCallResultFromTarget(address target, bool success, bytes returndata) → bytes`

internal #

Tool to verify that a low level call to smart-contract was successful, and reverts if the target was not a contract or bubbling up the revert reason (falling back to `Errors.FailedCall`) in case of an unsuccessful call.

 This function is DEPRECATED and may be removed in the next major release.

`verifyCallResult(bool success, bytes returndata) → bytes`

internal #

Tool to verify that a low level call was successful, and reverts if it wasn't, either by bubbling the revert reason or with a default `Errors.FailedCall` error.

`AddressEmptyCode(address target)`

error #

There's no code at `target` (it is not a contract).

Arrays





Collection of functions related to array types.

Functions

- [sort\(array, comp\)](#)
- [sort\(array\)](#)
- [sort\(array, comp\)](#)
- [sort\(array\)](#)
- [sort\(array, comp\)](#)
- [sort\(array\)](#)
- [findUpperBound\(array, element\)](#)
- [lowerBound\(array, element\)](#)
- [upperBound\(array, element\)](#)
- [lowerBoundMemory\(array, element\)](#)
- [upperBoundMemory\(array, element\)](#)
- [slice\(array, start\)](#)
- [slice\(array, start, end\)](#)
- [slice\(array, start\)](#)
- [slice\(array, start, end\)](#)
- [slice\(array, start\)](#)
- [slice\(array, start, end\)](#)
- [splice\(array, start\)](#)
- [splice\(array, start, end\)](#)
- [replace\(array, pos, replacement\)](#)
- [replace\(array, pos, replacement, offset, length\)](#)
- [splice\(array, start\)](#)

 **OpenZeppelin Docs**

- [`splice\(array, start, end\)`](#)
- [`replace\(array, pos, replacement\)`](#)
- [`replace\(array, pos, replacement, offset, length\)`](#)
- [`splice\(array, start\)`](#)
- [`splice\(array, start, end\)`](#)
- [`replace\(array, pos, replacement\)`](#)
- [`replace\(array, pos, replacement, offset, length\)`](#)
- [`unsafeAccess\(arr, pos\)`](#)
- [`unsafeAccess\(arr, pos\)`](#)
- [`unsafeAccess\(arr, pos\)`](#)
- [`unsafeAccess\(arr, pos\)`](#)
- [`unsafeAccess\(arr, pos\)`](#)
- [`unsafeMemoryAccess\(arr, pos\)`](#)
- [`unsafeMemoryAccess\(arr, pos\)`](#)
- [`unsafeMemoryAccess\(arr, pos\)`](#)
- [`unsafeMemoryAccess\(arr, pos\)`](#)
- [`unsafeMemoryAccess\(arr, pos\)`](#)
- [`unsafeSetLength\(array, len\)`](#)
- [`unsafeSetLength\(array, len\)`](#)
- [`unsafeSetLength\(array, len\)`](#)
- [`unsafeSetLength\(array, len\)`](#)
- [`unsafeSetLength\(array, len\)`](#)

`sort(uint256[] array, function (uint256,uint256) pure returns (bool comp) → uint256[]`

internal #

Sort an array of uint256 (in memory) following the provided comparator function.



This function does the sorting "in place", meaning that it overrides the input. The object is returned for convenience, but that returned value can be discarded safely if the caller has a memory pointer to the array.

i this function's cost is $O(n \cdot \log(n))$ in average and $O(n^2)$ in the worst case, with n the length of the array. Using it in view functions that are executed through `eth_call` is safe, but one should be very careful when executing this as part of a transaction. If the array being sorted is too large, the sort operation may consume more gas than is available in a block, leading to potential DoS.

⚠ Consider memory side-effects when using custom comparator functions that access memory in an unsafe way.

sort(uint256[] array) → uint256[]

internal #

Variant of `Arrays.sort` that sorts an array of uint256 in increasing order.

sort(address[] array, function (address,address) pure returns (bool) comp) → address[]

internal #

Sort an array of address (in memory) following the provided comparator function.

This function does the sorting "in place", meaning that it overrides the input. The object is returned for convenience, but that returned value can be discarded safely if the caller has a memory pointer to the array.

i this function's cost is $O(n \cdot \log(n))$ in average and $O(n^2)$ in the worst case, with n the length of the array. Using it in view functions that are executed through `eth_call` is safe, but one should be very careful when executing this as part of a transaction. If the array being sorted is too large, the sort operation may consume more gas than is available in a block, leading to potential DoS.

⚠ Consider memory side-effects when using custom comparator functions that access memory in an unsafe way.

Variant of `Arrays.sort` that sorts an array of `address` in increasing order.

`sort(bytes32[] array, function (bytes32,bytes32) pure returns (bool) comp) → bytes32[]`

internal #

Sort an array of `bytes32` (in memory) following the provided comparator function.

This function does the sorting "in place", meaning that it overrides the input. The object is returned for convenience, but that returned value can be discarded safely if the caller has a memory pointer to the array.

i this function's cost is $O(n \cdot \log(n))$ in average and $O(n^2)$ in the worst case, with n the length of the array. Using it in view functions that are executed through `eth_call` is safe, but one should be very careful when executing this as part of a transaction. If the array being sorted is too large, the sort operation may consume more gas than is available in a block, leading to potential DoS.

⚠ Consider memory side-effects when using custom comparator functions that access memory in an unsafe way.

`sort(bytes32[] array) → bytes32[]`

internal #

Variant of `Arrays.sort` that sorts an array of `bytes32` in increasing order.

`findUpperBound(uint256[] array, uint256 element) → uint256`

internal #

Searches a sorted `array` and returns the first index that contains a value greater or equal to `element`. If no such index exists (i.e. all values in the array are strictly less than `element`), the array length is returned. Time complexity $O(\log n)$.

 **Deprecated.** This implementation behaves as `Arrays.lowerBound` but lacks support for repeated elements in the array. The `Arrays.lowerBound` function should be used instead.

`lowerBound(uint256[] array, uint256 element) → uint256`

internal #

Searches an `array` sorted in ascending order and returns the first index that contains a value greater or equal than `element`. If no such index exists (i.e. all values in the array are strictly less than `element`), the array length is returned. Time complexity $O(\log n)$.

See C++'s [lower_bound](#).

`upperBound(uint256[] array, uint256 element) → uint256`

internal #

Searches an `array` sorted in ascending order and returns the first index that contains a value strictly greater than `element`. If no such index exists (i.e. all values in the array are strictly less than `element`), the array length is returned. Time complexity $O(\log n)$.

See C++'s [upper_bound](#).

`lowerBoundMemory(uint256[] array, uint256 element) → uint256`

internal #

Same as `Arrays.lowerBound`, but with an array in memory.

`upperBoundMemory(uint256[] array, uint256 element) → uint256`

internal #

Same as `Arrays.upperBound`, but with an array in memory.

`slice(address[] array, uint256 start) → address[]` internal #



Copies the content of `array`, from `start` (included) to the end of `array` into a new address array in memory.

 replicates the behavior of [Javascript's `Array.slice`](#)

`slice(address[] array, uint256 start, uint256 end) → address[]` internal #

Copies the content of `array`, from `start` (included) to `end` (excluded) into a new address array in memory. The `end` argument is truncated to the length of the `array`.

 replicates the behavior of [Javascript's `Array.slice`](#)

`slice(bytes32[] array, uint256 start) → bytes32[]` internal #

Copies the content of `array`, from `start` (included) to the end of `array` into a new bytes32 array in memory.

 replicates the behavior of [Javascript's `Array.slice`](#)

`slice(bytes32[] array, uint256 start, uint256 end) → bytes32[]` internal #

Copies the content of `array`, from `start` (included) to `end` (excluded) into a new bytes32 array in memory. The `end` argument is truncated to the length of the `array`.

 replicates the behavior of [Javascript's `Array.slice`](#)

Copies the content of `array`, from `start` (included) to the end of `array` into a new `uint256` array in memory.

 replicates the behavior of [Javascript's `Array.slice`](#)

`slice(uint256[] array, uint256 start, uint256 end) → uint256[]`

internal #

Copies the content of `array`, from `start` (included) to `end` (excluded) into a new `uint256` array in memory. The `end` argument is truncated to the length of the `array`.

 replicates the behavior of [Javascript's `Array.slice`](#)

`splice(address[] array, uint256 start) → address[]`

internal #

Moves the content of `array`, from `start` (included) to the end of `array` to the start of that array, and shrinks the array length accordingly, effectively overwriting the array with `array[start:]`.

 This function modifies the provided array in place. If you need to preserve the original array, use [`Arrays.slice`](#) instead.

`splice(address[] array, uint256 start, uint256 end) → address[]`

internal #

Moves the content of `array`, from `start` (included) to `end` (excluded) to the start of that array, and shrinks the array length accordingly, effectively overwriting the array with `array[start:end]`. The `end` argument is truncated to the length of the `array`.

replace(address[] array, uint256 pos, address[] replacement) → address[] internal #

Replaces elements in `array` starting at `pos` with all elements from `replacement`.

Parameters are clamped to valid ranges (e.g. `pos` is clamped to `[0, array.length]`). If `pos >= array.length`, no replacement occurs and the array is returned unchanged.

 This function modifies the provided array in place.

replace(address[] array, uint256 pos, address[] replacement, uint256 offset, uint256 length) → address[] internal #

Replaces elements in `array` starting at `pos` with elements from `replacement` starting at `offset`. Copies at most `length` elements from `replacement` to `array`.

Parameters are clamped to valid ranges (i.e. `pos` is clamped to `[0, array.length]`, `offset` is clamped to `[0, replacement.length]`, and `length` is clamped to `min(length, replacement.length - offset, array.length - pos)`). If `pos >= array.length` or `offset >= replacement.length`, no replacement occurs and the array is returned unchanged.

 This function modifies the provided array in place.

splice(bytes32[] array, uint256 start) → bytes32[] internal #

Moves the content of `array`, from `start` (included) to the end of `array` to the start of that array, and shrinks the array length accordingly, effectively overwriting the array with `array[start:]`.



This function modifies the provided array in place. If you need to preserve the original array, use [Arrays.slice](#) instead.

`splice(bytes32[] array, uint256 start, uint256 end) → bytes32[]`

internal #

Moves the content of `array`, from `start` (included) to `end` (excluded) to the start of that array, and shrinks the array length accordingly, effectively overwriting the array with `array[start:end]`. The `end` argument is truncated to the length of the `array`.

i This function modifies the provided array in place. If you need to preserve the original array, use [Arrays.slice](#) instead.

`replace(bytes32[] array, uint256 pos, bytes32[] replacement) → bytes32[]` internal #

Replaces elements in `array` starting at `pos` with all elements from `replacement`.

Parameters are clamped to valid ranges (e.g. `pos` is clamped to `[0, array.length]`). If `pos >= array.length`, no replacement occurs and the array is returned unchanged.

i This function modifies the provided array in place.

`replace(bytes32[] array, uint256 pos, bytes32[] replacement, uint256 offset, uint256 length) → bytes32[]`

internal #

Replaces elements in `array` starting at `pos` with elements from `replacement` starting at `offset`. Copies at most `length` elements from `replacement` to `array`.

Parameters are clamped to valid ranges (i.e. `pos` is clamped to `[0, array.length]`, `offset` is clamped to `[0, replacement.length]`, and `length` is clamped to `min(length, replacement.length - offset, array.length - pos)`). If `pos >= array.length` or `offset >= replacement.length`, no replacement occurs and the array is returned unchanged.

splice(uint256[] array, uint256 start) → uint256[]

internal #

Moves the content of `array`, from `start` (included) to the end of `array` to the start of that array, and shrinks the array length accordingly, effectively overwriting the array with `array[start:]`.

 This function modifies the provided array in place. If you need to preserve the original array, use [Arrays.slice](#) instead.

splice(uint256[] array, uint256 start, uint256 end) → uint256[]

internal #

Moves the content of `array`, from `start` (included) to `end` (excluded) to the start of that array, and shrinks the array length accordingly, effectively overwriting the array with `array[start:end]`. The `end` argument is truncated to the length of the `array`.

 This function modifies the provided array in place. If you need to preserve the original array, use [Arrays.slice](#) instead.

replace(uint256[] array, uint256 pos, uint256[] replacement) → uint256[] internal #

Replaces elements in `array` starting at `pos` with all elements from `replacement`.

Parameters are clamped to valid ranges (e.g. `pos` is clamped to `[0, array.length]`). If `pos >= array.length`, no replacement occurs and the array is returned unchanged.

 This function modifies the provided array in place.

replace(uint256[] array, uint256 pos, uint256[] replacement, uint256 offset) → uint256[] internal #

Replaces elements in `array` starting at `pos` with elements from `replacement` starting at `offset`. Copies at most `length` elements from `replacement` to `array`.

Parameters are clamped to valid ranges (i.e. `pos` is clamped to `[0, array.length]`, `offset` is clamped to `[0, replacement.length]`, and `length` is clamped to `min(length, replacement.length - offset, array.length - pos)`). If `pos >= array.length` or `offset >= replacement.length`, no replacement occurs and the array is returned unchanged.

 This function modifies the provided array in place.

unsafeAccess(address[] arr, uint256 pos) → struct StorageSlot.AddressSlot

internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

 Only use if you are certain `pos` is lower than the array length.

unsafeAccess(bytes32[] arr, uint256 pos) → struct StorageSlot.Bytes32Slot

internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

 Only use if you are certain `pos` is lower than the array length.

unsafeAccess(uint256[] arr, uint256 pos) → struct StorageSlot.Uint256Slot

internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

`unsafeAccess(bytes[] arr, uint256 pos) → struct StorageSlot.BytesSlot` internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

⚠️ Only use if you are certain `pos` is lower than the array length.

`unsafeAccess(string[] arr, uint256 pos) → struct StorageSlot.StringSlot` internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

⚠️ Only use if you are certain `pos` is lower than the array length.

`unsafeMemoryAccess(address[] arr, uint256 pos) → address res` internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

⚠️ Only use if you are certain `pos` is lower than the array length.

`unsafeMemoryAccess(bytes32[] arr, uint256 pos) → bytes32 res` internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

⚠️ Only use if you are certain `pos` is lower than the array length.

`unsafeMemoryAccess(uint256[] arr, uint256 pos) → uint256 res` internal #

 **OpenZeppelin** Docs

 Only use if you are certain `pos` is lower than the array length.

`unsafeMemoryAccess(bytes[] arr, uint256 pos) → bytes res`

internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

 Only use if you are certain `pos` is lower than the array length.

`unsafeMemoryAccess(string[] arr, uint256 pos) → string res`

internal #

Access an array in an "unsafe" way. Skips solidity "index-out-of-range" check.

 Only use if you are certain `pos` is lower than the array length.

`unsafeSetLength(address[] array, uint256 len)`

internal #

Helper to set the length of a dynamic array. Directly writing to `.length` is forbidden.

 this does not clear elements if length is reduced, or initialize elements if length is increased.

`unsafeSetLength(bytes32[] array, uint256 len)`

internal #

Helper to set the length of a dynamic array. Directly writing to `.length` is forbidden.

 this does not clear elements if length is reduced, or initialize elements if length is increased.

OpenZeppelin Docs

internal #

Helper to set the length of a dynamic array. Directly writing to `.length` is forbidden.

 this does not clear elements if length is reduced, or initialize elements if length is increased.

unsafeSetLength(bytes[] array, uint256 len)

internal #

Helper to set the length of a dynamic array. Directly writing to `.length` is forbidden.

 this does not clear elements if length is reduced, or initialize elements if length is increased.

unsafeSetLength(string[] array, uint256 len)

internal #

Helper to set the length of a dynamic array. Directly writing to `.length` is forbidden.

 this does not clear elements if length is reduced, or initialize elements if length is increased.

Base58



```
import "@openzeppelin/contracts/utils/Base58.sol";
```



Provides a set of functions to operate with Base58 strings.

Base58 is an encoding scheme that converts binary data into a human-readable text format. Similar to [Base64](#) but specifically designed for better human usability.

1. Human-friendly alphabet: Excludes visually similar characters to reduce human error:



OpenZeppelin Docs

- No 0 (zero) vs O (capital o) confusion
- No I (capital i) vs l (lowercase L) confusion
- No non-alphanumeric characters like + or =

2. URL-safe: Contains only alphanumeric characters, making it safe for URLs without encoding.

Initially based on [storyicon's implementation](#) (MIT). Based on the updated and improved [Vectorized version](#) (MIT).

Functions

- [encode\(input\)](#)
- [decode\(input\)](#)

Errors

- [InvalidBase58Char\(\)](#)

encode(bytes input) → string

internal #

Encode a `bytes` buffer as a Base58 `string`.

decode(string input) → bytes

internal #

Decode a Base58 `string` into a `bytes` buffer.

Unrecognized Base58 character on decoding.

Base64



```
import "@openzeppelin/contracts/utils/Base64.sol";
```



Provides a set of functions to operate with Base64 strings.

Functions

- [encode\(data\)](#)
- [encodeURL\(data\)](#)
- [decode\(data\)](#)

Errors

- [InvalidBase64Char\(\)](#)

encode(bytes data) → string

internal #

Converts a `bytes` to its Base64 `string` representation.

`encodeURL(bytes data) → string` internal #

 **OpenZeppelin Docs**

Converts a `bytes` to its Base64Url `string` representation. Output is not padded with `=` as specified in [rfc4648](#).

`decode(string data) → bytes`

internal #

Converts a Base64 `string` to the `bytes` it represents.

- Supports padded and unpadded inputs.
- Supports both encoding (`Base58.encode` and `Base64.encodeURL`) seamlessly.
- Reverts with `Base64.InvalidBase64Char` if the input contains an invalid character.

`InvalidBase64Char(bytes1)`

error #

Blockhash



```
import "@openzeppelin/contracts/utils/Blockhash.sol";
```



Library for accessing historical block hashes beyond the standard 256 block limit. Uses EIP-2935's history storage contract which maintains a ring buffer of the last 8191 block hashes in state.

For blocks within the last 256 blocks, it uses the native `BLOCKHASH` opcode. For blocks between 257 and 8191 blocks ago, it queries the EIP-2935 history storage. For blocks older than 8191 or future blocks, it returns zero, matching the `BLOCKHASH` behavior.



After EIP-8838 activation, it takes 8191 blocks to completely fill the history. Before that, only block hashes since the fork block will be available.

Functions

- [blockHash\(blockNumber\)](#)

blockHash(uint256 blockNumber) → bytes32

internal #

Retrieves the block hash for any historical block within the supported range.

i The function gracefully handles future blocks and blocks beyond the history window by returning zero, consistent with the EVM's native **BLOCKHASH** behavior.

Bytes



```
import "@openzeppelin/contracts/utils/Bytes.sol";
```



Bytes operations.

Functions

- [indexOf\(buffer, s\)](#)
- [indexOf\(buffer, s, pos\)](#)
- [lastIndexOf\(buffer, s\)](#)
- [lastIndexOf\(buffer, s, pos\)](#)

- [slice\(buffer, start\)](#)
- [slice\(buffer, start, end\)](#)
- [splice\(buffer, start\)](#)

- [splice\(buffer, start, end\)](#)
- [replace\(buffer, pos, replacement\)](#)
- [replace\(buffer, pos, replacement, offset, length\)](#)
- [concat\(buffer\)](#)
- [toNibbles\(input\)](#)
- [equal\(a, b\)](#)
- [reverseBytes32\(value\)](#)
- [reverseBytes16\(value\)](#)
- [reverseBytes8\(value\)](#)
- [reverseBytes4\(value\)](#)
- [reverseBytes2\(value\)](#)
- [clz\(buffer\)](#)

indexOf(bytes buffer, bytes1 s) → uint256

internal #

Forward search for `s` in `buffer`

- If `s` is present in the buffer, returns the index of the first instance
- If `s` is not present in the buffer, returns `type(uint256).max`

i replicates the behavior of [Javascript's Array.indexOf](#)

indexOf(bytes buffer, bytes1 s, uint256 pos) → uint256

internal #



- If `s` is present in the buffer (at or after `pos`), returns the index of the next instance
- If `s` is not present in the buffer (at or after `pos`), returns `type(uint256).max`

i replicates the behavior of [Javascript's `Array.indexOf`](#)

lastIndexOf(bytes buffer, bytes1 s) → uint256

internal #

Backward search for `s` in `buffer`

- If `s` is present in the buffer, returns the index of the last instance
- If `s` is not present in the buffer, returns `type(uint256).max`

i replicates the behavior of [Javascript's `Array.lastIndexOf`](#)

lastIndexOf(bytes buffer, bytes1 s, uint256 pos) → uint256

internal #

Backward search for `s` in `buffer` starting at position `pos`

- If `s` is present in the buffer (at or before `pos`), returns the index of the previous instance
- If `s` is not present in the buffer (at or before `pos`), returns `type(uint256).max`

i replicates the behavior of [Javascript's `Array.lastIndexOf`](#)

slice(bytes buffer, uint256 start) → bytes

internal #

Copies the content of `buffer`, from `start` (included) to the end of `buffer` into a new bytes object in memory.

slice(bytes buffer, uint256 start, uint256 end) → bytes

internal #

Copies the content of `buffer`, from `start` (included) to `end` (excluded) into a new bytes object in memory. The `end` argument is truncated to the length of the `buffer`.

 replicates the behavior of [Javascript's Array.slice](#)

splice(bytes buffer, uint256 start) → bytes

internal #

Moves the content of `buffer`, from `start` (included) to the end of `buffer` to the start of that buffer, and shrinks the buffer length accordingly, effectively overriding the content of `buffer` with `buffer[start:]`.

 This function modifies the provided buffer in place. If you need to preserve the original buffer, use [Arrays.slice](#) instead

splice(bytes buffer, uint256 start, uint256 end) → bytes

internal #

Moves the content of `buffer`, from `start` (included) to `end` (excluded) to the start of that buffer, and shrinks the buffer length accordingly, effectively overriding the content of `buffer` with `buffer[start:end]`. The `end` argument is truncated to the length of the `buffer`.

 This function modifies the provided buffer in place. If you need to preserve the original buffer, use [Arrays.slice](#) instead

replace(bytes buffer, uint256 pos, bytes replacement) → bytes

internal #

 **OpenZeppelin Docs**

Replaces bytes in `buffer` starting at `pos` with all bytes from `replacement`.

Parameters are clamped to valid ranges (i.e. `pos` is clamped to `[0, buffer.length]`). If `pos >= buffer.length`, no replacement occurs and the buffer is returned unchanged.

 This function modifies the provided buffer in place.

`replace(bytes buffer, uint256 pos, bytes replacement, uint256 offset, uint256 length) → bytes` internal #

Replaces bytes in `buffer` starting at `pos` with bytes from `replacement` starting at `offset`. Copies at most `length` bytes from `replacement` to `buffer`.

Parameters are clamped to valid ranges (i.e. `pos` is clamped to `[0, buffer.length]`, `offset` is clamped to `[0, replacement.length]`, and `length` is clamped to `min(length, replacement.length - offset, buffer.length - pos)`). If `pos >= buffer.length` or `offset >= replacement.length`, no replacement occurs and the buffer is returned unchanged.

 This function modifies the provided buffer in place.

`concat(bytes[] buffers) → bytes` internal #

Concatenate an array of bytes into a single bytes object.

For fixed bytes types, we recommend using the solidity built-in `bytes.concat` or (equivalent) `abi.encodePacked`.

 this could be done in assembly with a single loop that expands starting at the FMP, but that would be significantly less readable. It might be worth benchmarking the savings of the full-assembly approach.

Split each byte in `input` into two nibbles (4 bits each)

Example: hex"01234567" → hex"0001020304050607"

`equal(bytes a, bytes b) → bool`

internal #

Returns true if the two byte buffers are equal.

`reverseBytes32(bytes32 value) → bytes32`

internal #

Reverses the byte order of a bytes32 value, converting between little-endian and big-endian. Inspired by [Reverse Parallel](#)

`reverseBytes16(bytes16 value) → bytes16`

internal #

Same as [Bytes.reverseBytes32](#) but optimized for 128-bit values.

`reverseBytes8(bytes8 value) → bytes8`

internal #

Same as [Bytes.reverseBytes32](#) but optimized for 64-bit values.

`reverseBytes4(bytes4 value) → bytes4`

internal #

Same as [Bytes.reverseBytes32](#) but optimized for 32-bit values.

Same as [Bytes.reverseBytes32](#) but optimized for 16-bit values.

clz(bytes buffer) → uint256

internal #

Counts the number of leading zero bits a bytes array. Returns `8 * buffer.length` if the buffer is all zeros.

CAIP10



```
import "@openzeppelin/contracts/utils/CAIP10.sol";
```



Helper library to format and parse CAIP-10 identifiers

CAIP-10 defines account identifiers as: `account_id: chain_id + ":" + account_address`

`chain_id: [-a-z0-9]8:[-_a-zA-Z0-9]32` (See [CAIP2](#)) `account_address: [-.%a-zA-Z0-9]128`

⚠ According to [CAIP-10's canonicalization section](#), the implementation remains at the developer's discretion. Please note that case variations may introduce ambiguity. For example, when building hashes to identify accounts or data associated to them, multiple representations of the same account would derive to different hashes. For EVM chains, we recommend using checksummed addresses for the "account_address" part. They can be generated onchain using [Strings.toChecksumHexString](#).

Functions

- [local\(account\)](#)
- [format\(caip2, account\)](#)

local(address account) → string

internal #

Return the CAIP-10 identifier for an account on the current (local) chain.

format(string caip2, string account) → string

internal #

Return the CAIP-10 identifier for a given caip2 chain and account.

 This function does not verify that the inputs are properly formatted.

parse(string caip10) → string caip2, string account

internal #

Parse a CAIP-10 identifier into its components.

 This function does not verify that the CAIP-10 input is properly formatted. The `caip2` return can be parsed using the [CAIP2](#) library.

CAIP2



```
import "@openzeppelin/contracts/utils/CAIP2.sol";
```



Helper library to format and parse CAIP-2 identifiers

CAIP-2 defines chain identifiers as: `chain_id: namespace + ":" + reference` namespace: `[-a-z0-9]` reference: `[-_a-zA-Z0-9]`

⚠ In some cases, multiple CAIP-2 identifiers may all be valid representation of a single chain. For EVM chains, it is recommended to use `eip155:xxx` as the canonical representation (where `xxx` is the EIP-155 chain id). Consider the possible ambiguity when processing CAIP-2 identifiers or when using them in the context of hashes.

Functions

- [`local\(\)`](#)
- [`format\(namespace, ref\)`](#)
- [`parse\(caip2\)`](#)

`local()` → `string`

internal #

Return the CAIP-2 identifier for the current (local) chain.

`format(string namespace, string ref)` → `string`

internal #

Return the CAIP-2 identifier for a given namespace and reference.

ⓘ This function does not verify that the inputs are properly formatted.

`parse(string caip2)` → `string namespace, string ref`

internal #

Parse a CAIP-2 identifier into its components.

ⓘ This function does not verify that the CAIP-2 input is properly formatted.

Calldata



```
import "@openzeppelin/contracts/utils/Calldata.sol";
```



Helper library for manipulating objects in calldata.

Functions

- [emptyBytes\(\)](#)
- [emptyString\(\)](#)

emptyBytes() → bytes result

internal #

emptyString() → string result

internal #

Comparators



```
import "@openzeppelin/contracts/utils/Comparators.sol";
```



Provides a set of functions to compare values.

Available since v5.1.

Functions



```
lt(uint256 a, uint256 b) → bool
```

internal #

```
gt(uint256 a, uint256 b) → bool
```

internal #

Context



```
import "@openzeppelin/contracts/utils/Context.sol";
```



Provides information about the current execution context, including the sender of the transaction and its data. While these are generally available via `msg.sender` and `msg.data`, they should not be accessed in such a direct manner, since when dealing with meta-transactions the account sending and paying for execution may not be the actual sender (as far as an application is concerned).

This contract is only required for intermediate, library-like contracts.

Functions

- [_msgSender\(\)](#)
- [_msgData\(\)](#)
- [_contextSuffixLength\(\)](#)

 `msgSender() → address`
OpenZeppelin Docs

internal #

`_msgData() → bytes`

internal #

`_contextSuffixLength() → uint256`

internal #

Create2



```
import "@openzeppelin/contracts/utils/Create2.sol";
```



Helper to make usage of the `CREATE2` EVM opcode easier and safer. `CREATE2` can be used to compute in advance the address where a smart contract will be deployed, which allows for interesting new mechanisms known as 'counterfactual interactions'.

See the [EIP](#) for more information.

Functions

- [deploy\(amount, salt, bytecode\)](#)
- [computeAddress\(salt, bytecodeHash\)](#)
- [computeAddress\(salt, bytecodeHash, deployer\)](#)

Errors

- [Create2EmptyBytecode\(\)](#)

Deploys a contract using `CREATE2`. The address where the contract will be deployed can be known in advance via `Create2.computeAddress`.

The bytecode for a contract can be obtained from Solidity with `type(contractName).creationCode`.

Requirements:

- `bytecode` must not be empty.
- `salt` must have not been used for `bytecode` already.
- the factory must have a balance of at least `amount`.
- if `amount` is non-zero, `bytecode` must have a `payable` constructor.

`computeAddress(bytes32 salt, bytes32 bytecodeHash) → address` internal #

Returns the address where a contract will be stored if deployed via `Create2.deploy`. Any change in the `bytecodeHash` or `salt` will result in a new destination address.

`computeAddress(bytes32 salt, bytes32 bytecodeHash, address deployer) → address` internal #

Returns the address where a contract will be stored if deployed via `Create2.deploy` from a contract located at `deployer`. If `deployer` is this contract's address, returns the same value as `Create2.computeAddress`.

`Create2EmptyBytecode()` error #



Errors



```
import "@openzeppelin/contracts/utils/Errors.sol";
```



Collection of common custom errors used in multiple contracts

 Backwards compatibility is not guaranteed in future versions of the library. It is recommended to avoid relying on the error API for critical functionality.

Available since v5.1.

Errors

- [InsufficientBalance\(balance, needed\)](#)
- [FailedCall\(\)](#)
- [FailedDeployment\(\)](#)
- [MissingPrecompile\(\)](#)

InsufficientBalance(uint256 balance, uint256 needed)

error #

The ETH balance of the account is not enough to perform the operation.

FailedCall()

error #

 A call to an address target failed. The target may have reverted.
OpenZeppelin Docs

FailedDeployment()

error #

The deployment failed.

MissingPrecompile(address)

error #

A necessary precompile is missing.

LowLevelCall



```
import "@openzeppelin/contracts/utils/LowLevelCall.sol";
```



Library of low level call functions that implement different calling strategies to deal with the return data.

 Using this library requires an advanced understanding of Solidity and how the EVM works. It is recommended to use the [Address](#) library instead.

Functions

- [callNoReturn\(target, data\)](#)
- [callNoReturn\(target, value, data\)](#)
- [callReturn64Bytes\(target, data\)](#)
- [callReturn64Bytes\(target, value, data\)](#)

OpenZeppelin Docs

- [staticcallNoReturn\(target, data\)](#)
- [staticcallReturn64Bytes\(target, data\)](#)
- [delegatecallNoReturn\(target, data\)](#)
- [delegatecallReturn64Bytes\(target, data\)](#)
- [returnDataSize\(\)](#)
- [returnData\(\)](#)
- [bubbleRevert\(\)](#)
- [bubbleRevert\(returndata\)](#)

callNoReturn(address target, bytes data) → bool success

internal #

Performs a Solidity function call using a low level `call` and ignoring the return data.

callNoReturn(address target, uint256 value, bytes data) → bool success

internal #

Same as `callNoReturn-address-bytes`, but allows specifying the value to be sent in the call.

callReturn64Bytes(address target, bytes data) → bool success, bytes32 result1, bytes32 result2

internal #

Performs a Solidity function call using a low level `call` and returns the first 64 bytes of the result in the scratch space of memory. Useful for functions that return a tuple with two single-word values.

 Do not assume that the results are zero if `success` is false. Memory can be already allocated and this function doesn't zero it out.

callReturn64Bytes(address target, uint256 value, bytes data) → bool success, bytes32 result1, bytes32 result2 internal #

Same as `callReturn64Bytes-address-bytes`, but allows specifying the value to be sent in the call.

staticcallNoReturn(address target, bytes data) → bool success internal #

Performs a Solidity function call using a low level `staticcall` and ignoring the return data.

staticcallReturn64Bytes(address target, bytes data) → bool success, bytes32 result1, bytes32 result2 internal #

Performs a Solidity function call using a low level `staticcall` and returns the first 64 bytes of the result in the scratch space of memory. Useful for functions that return a tuple with two single-word values.

 Do not assume that the results are zero if `success` is false. Memory can be already allocated and this function doesn't zero it out.

delegatecallNoReturn(address target, bytes data) → bool success internal #

Performs a Solidity function call using a low level `delegatecall` and ignoring the return data.

delegatecallReturn64Bytes(address target, bytes data) → bool success, bytes32 result1, bytes32 result2 internal #

Performs a Solidity function call using a low level `delegatecall` and returns the first 64 bytes of the result in the scratch space of memory. Useful for functions that return a



Do not assume that the results are zero if `success` is false. Memory can be already allocated and this function doesn't zero it out.

returnDataSize() → uint256 size

internal #

Returns the size of the return data buffer.

returnData() → bytes result

internal #

Returns a buffer containing the return data from the last call.

bubbleRevert()

internal #

Revert with the return data from the last call.

bubbleRevert(bytes returndata)

internal #

Memory



```
import "@openzeppelin/contracts/utils/Memory.sol";
```



Utilities to manipulate memory.

Memory is a contiguous and dynamic byte array in which Solidity stores non-primitive types. This library provides functions to manipulate pointers to this dynamic array and work with slices of it.

Slices provide a view into a portion of memory without copying data, enabling efficient substring operations.

 When manipulating memory pointers or slices, make sure to follow the Solidity documentation guidelines for [Memory Safety](#).

Functions

- [getFreeMemoryPointer\(\)](#)
- [unsafeSetFreeMemoryPointer\(ptr\)](#)
- [forward\(ptr, offset\)](#)
- [equal\(ptr1, ptr2\)](#)
- [asSlice\(self\)](#)
- [length\(self\)](#)
- [slice\(self, offset\)](#)
- [slice\(self, offset, len\)](#)
- [load\(self, offset\)](#)
- [toBytes\(self\)](#)
- [equal\(a, b\)](#)
- [isReserved\(self\)](#)

`getFreeMemoryPointer()` → **`Memory.Pointer`** **`ptr`**

internal #

Returns a `Pointer` to the current free `Pointer`.

OpenZeppelin Docs

internal #

Sets the free `Pointer` to a specific value.

The solidity memory layout requires that the FMP is never set to a value lower than 0x80. Setting the FMP to a value lower than 0x80 may cause unexpected behavior. Deallocating all memory can be achieved by setting the FMP to 0x80.

 Everything after the pointer may be overwritten.

`forward(Memory.Pointer ptr, uint256 offset) → Memory.Pointer`

internal #

Move a pointer forward by a given offset.

`equal(Memory.Pointer ptr1, Memory.Pointer ptr2) → bool`

internal #

Equality comparator for memory pointers.

`asSlice(bytes self) → Memory.Slice result`

internal #

Get a slice representation of a bytes object in memory

`length(Memory.Slice self) → uint256 result`

internal #

Returns the length of a given slice (equiv to self.length for calldata slices)

`slice(Memory.Slice self, uint256 offset) → Memory.Slice`

internal #

Offset a memory slice (equivalent to self[offset:] for calldata slices)

 **OpenZeppelin Docs** `self, uint256 offset, uint256 len) → Memory.Slice` internal #

Offset and cut a Slice (equivalent to `self[offset:offset+len]` for calldata slices)

`load(Memory.Slice self, uint256 offset) → bytes32 value` internal #

Read a bytes32 buffer from a given Slice at a specific offset

 If `offset > length(slice) - 0x20`, part of the return value will be out of bound of the slice. These bytes are zeroed.

`toBytes(Memory.Slice self) → bytes result` internal #

Extract the data corresponding to a Slice (allocate new memory)

`equal(Memory.Slice a, Memory.Slice b) → bool result` internal #

Returns true if the two slices contain the same data.

`isReserved(Memory.Slice self) → bool result` internal #

Returns true if the memory occupied by the slice is reserved (i.e. before the free memory pointer)

Multicall





Provides a function to batch together multiple calls in a single external call.

Consider any assumption about calldata validation performed by the sender may be violated if it's not especially careful about sending transactions invoking `Multicall.multicall`. For example, a relay address that filters function selectors won't filter calls nested within a `Multicall.multicall` operation.

i Since 5.0.1 and 4.9.4, this contract identifies non-canonical contexts (i.e. `msg.sender` is not `Context._msgSender`). If a non-canonical context is identified, the following self `delegatecall` appends the last bytes of `msg.data` to the subcall. This makes it safe to use with `ERC2771Context`. Contexts that don't affect the resolution of `Context._msgSender` are not propagated to subcalls.

Functions

- `multicall(data)`

`multicall(bytes[] data) → bytes[] results`

public #

Receives and executes a batch of function calls on this contract.

Nonces



```
import "@openzeppelin/contracts/utils/Nonces.sol";
```



Provides tracking nonces for addresses. Nonces will only increment.

- [nonces\(owner\)](#)
- [_useNonce\(owner\)](#)
- [_useCheckedNonce\(owner, nonce\)](#)

Errors

- [InvalidAccountNonce\(account, currentNonce\)](#)

nonces(address owner) → uint256

public #

Returns the next unused nonce for an address.

_useNonce(address owner) → uint256

internal #

Consumes a nonce.

Returns the current value and increments nonce.

_useCheckedNonce(address owner, uint256 nonce)

internal #

Same as [Nonces._useNonce](#) but checking that `nonce` is the next valid for `owner`.

InvalidAccountNonce(address account, uint256 currentNonce)

error #

The nonce used for an account is not the expected current nonce.

NoncesKeyed



```
import "@openzeppelin/contracts/utils/NoncesKeyed.sol";
```



Alternative to [Nonces](#), that supports key-ed nonces.

Follows the [ERC-4337's semi-abstracted nonce system](#).

i This contract inherits from [Nonces](#) and reuses its storage for the first nonce key (i.e. `0`). This makes upgrading from [Nonces](#) to [NoncesKeyed](#) safe when using their upgradeable versions (e.g. [NoncesKeyedUpgradeable](#)). Doing so will NOT reset the current state of nonces, avoiding replay attacks where a nonce is reused after the upgrade.

Functions

- [nonces\(owner, key\)](#)
- [_useNonce\(owner, key\)](#)
- [_useCheckedNonce\(owner, keyNonce\)](#)
- [_useCheckedNonce\(owner, key, nonce\)](#)

► [Nonces](#)

Errors

► [Nonces](#)

 **OpenZeppelin Docs**

public #

Returns the next unused nonce for an address and key. Result contains the key prefix.

_useNonce(address owner, uint192 key) → uint256

internal #

Consumes the next unused nonce for an address and key.

Returns the current value without the key prefix. Consumed nonce is increased, so calling this function twice with the same arguments will return different (sequential) results.

_useCheckedNonce(address owner, uint256 keyNonce)

internal #

Same as [Nonces._useNonce](#) but checking that `nonce` is the next valid for `owner`.

This version takes the key and the nonce in a single uint256 parameter:

- use the first 24 bytes for the key
- use the last 8 bytes for the nonce

_useCheckedNonce(address owner, uint192 key, uint64 nonce)

internal #

Same as [Nonces._useNonce](#) but checking that `nonce` is the next valid for `owner`.

This version takes the key and the nonce as two different parameters.



```
import "@openzeppelin/contracts/utils/Packing.sol";
```



Helper library packing and unpacking multiple values into bytesXX.

Example usage:

```
library MyPacker {
    type MyType is bytes32;

    function _pack(address account, bytes4 selector, uint64 period) external pure returns
        bytes12 subpack = Packing.pack_4_8(selector, bytes8(period));
        bytes32 pack = Packing.pack_20_12(bytes20(account), subpack);
        return MyType.wrap(pack);
    }

    function _unpack(MyType self) external pure returns (address, bytes4, uint64) {
        bytes32 pack = MyType.unwrap(self);
        return (
            address(Packing.extract_32_20(pack, 0)),
            Packing.extract_32_4(pack, 20),
            uint64(Packing.extract_32_8(pack, 24))
        );
    }
}
```



Available since v5.1.

Functions

- [pack_1_1\(left, right\)](#)
- [pack_2_2\(left, right\)](#)
- [pack_2_4\(left, right\)](#)
- [pack_2_6\(left, right\)](#)
- [pack_2_8\(left, right\)](#)



- [pack_2_10\(left, right\)](#)
- [pack_2_20\(left, right\)](#)
- [pack_2_22\(left, right\)](#)
- [pack_4_2\(left, right\)](#)
- [pack_4_4\(left, right\)](#)
- [pack_4_6\(left, right\)](#)
- [pack_4_8\(left, right\)](#)
- [pack_4_12\(left, right\)](#)
- [pack_4_16\(left, right\)](#)
- [pack_4_20\(left, right\)](#)
- [pack_4_24\(left, right\)](#)
- [pack_4_28\(left, right\)](#)
- [pack_6_2\(left, right\)](#)
- [pack_6_4\(left, right\)](#)
- [pack_6_6\(left, right\)](#)
- [pack_6_10\(left, right\)](#)
- [pack_6_16\(left, right\)](#)
- [pack_6_22\(left, right\)](#)
- [pack_8_2\(left, right\)](#)
- [pack_8_4\(left, right\)](#)
- [pack_8_8\(left, right\)](#)
- [pack_8_12\(left, right\)](#)
- [pack_8_16\(left, right\)](#)
- [pack_8_20\(left, right\)](#)
- [pack_8_24\(left, right\)](#)
- [pack_10_2\(left, right\)](#)
- [pack_10_6\(left, right\)](#)

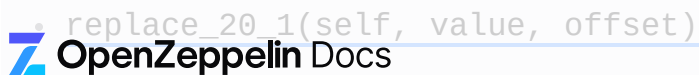


- [pack_10_10\(left, right\)](#)
- [pack_10_12\(left, right\)](#)
- [pack_10_22\(left, right\)](#)
- [pack_12_4\(left, right\)](#)
- [pack_12_8\(left, right\)](#)
- [pack_12_10\(left, right\)](#)
- [pack_12_12\(left, right\)](#)
- [pack_12_16\(left, right\)](#)
- [pack_12_20\(left, right\)](#)
- [pack_16_4\(left, right\)](#)
- [pack_16_6\(left, right\)](#)
- [pack_16_8\(left, right\)](#)
- [pack_16_12\(left, right\)](#)
- [pack_16_16\(left, right\)](#)
- [pack_20_2\(left, right\)](#)
- [pack_20_4\(left, right\)](#)
- [pack_20_8\(left, right\)](#)
- [pack_20_12\(left, right\)](#)
- [pack_22_2\(left, right\)](#)
- [pack_22_6\(left, right\)](#)
- [pack_22_10\(left, right\)](#)
- [pack_24_4\(left, right\)](#)
- [pack_24_8\(left, right\)](#)
- [pack_28_4\(left, right\)](#)
- [extract_2_1\(self, offset\)](#)
- [replace_2_1\(self, value, offset\)](#)
- [extract_4_1\(self, offset\)](#)



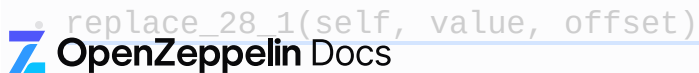
- [replace_4_1\(self, value, offset\)](#)
- [extract_4_2\(self, offset\)](#)
- [replace_4_2\(self, value, offset\)](#)
- [extract_6_1\(self, offset\)](#)
- [replace_6_1\(self, value, offset\)](#)
- [extract_6_2\(self, offset\)](#)
- [replace_6_2\(self, value, offset\)](#)
- [extract_6_4\(self, offset\)](#)
- [replace_6_4\(self, value, offset\)](#)
- [extract_8_1\(self, offset\)](#)
- [replace_8_1\(self, value, offset\)](#)
- [extract_8_2\(self, offset\)](#)
- [replace_8_2\(self, value, offset\)](#)
- [extract_8_4\(self, offset\)](#)
- [replace_8_4\(self, value, offset\)](#)
- [extract_8_6\(self, offset\)](#)
- [replace_8_6\(self, value, offset\)](#)
- [extract_10_1\(self, offset\)](#)
- [replace_10_1\(self, value, offset\)](#)
- [extract_10_2\(self, offset\)](#)
- [replace_10_2\(self, value, offset\)](#)
- [extract_10_4\(self, offset\)](#)
- [replace_10_4\(self, value, offset\)](#)
- [extract_10_6\(self, offset\)](#)
- [replace_10_6\(self, value, offset\)](#)
- [extract_10_8\(self, offset\)](#)
- [replace_10_8\(self, value, offset\)](#)

- 
- [`extract\_12\_1\(self, offset\)`](#)
 - [`replace\_12\_1\(self, value, offset\)`](#)
 - [`extract\_12\_2\(self, offset\)`](#)
 - [`replace\_12\_2\(self, value, offset\)`](#)
 - [`extract\_12\_4\(self, offset\)`](#)
 - [`replace\_12\_4\(self, value, offset\)`](#)
 - [`extract\_12\_6\(self, offset\)`](#)
 - [`replace\_12\_6\(self, value, offset\)`](#)
 - [`extract\_12\_8\(self, offset\)`](#)
 - [`replace\_12\_8\(self, value, offset\)`](#)
 - [`extract\_12\_10\(self, offset\)`](#)
 - [`replace\_12\_10\(self, value, offset\)`](#)
 - [`extract\_16\_1\(self, offset\)`](#)
 - [`replace\_16\_1\(self, value, offset\)`](#)
 - [`extract\_16\_2\(self, offset\)`](#)
 - [`replace\_16\_2\(self, value, offset\)`](#)
 - [`extract\_16\_4\(self, offset\)`](#)
 - [`replace\_16\_4\(self, value, offset\)`](#)
 - [`extract\_16\_6\(self, offset\)`](#)
 - [`replace\_16\_6\(self, value, offset\)`](#)
 - [`extract\_16\_8\(self, offset\)`](#)
 - [`replace\_16\_8\(self, value, offset\)`](#)
 - [`extract\_16\_10\(self, offset\)`](#)
 - [`replace\_16\_10\(self, value, offset\)`](#)
 - [`extract\_16\_12\(self, offset\)`](#)
 - [`replace\_16\_12\(self, value, offset\)`](#)
 - [`extract\_20\_1\(self, offset\)`](#)



- [replace_20_1\(self, value, offset\)](#)
- [extract_20_2\(self, offset\)](#)
- [replace_20_2\(self, value, offset\)](#)
- [extract_20_4\(self, offset\)](#)
- [replace_20_4\(self, value, offset\)](#)
- [extract_20_6\(self, offset\)](#)
- [replace_20_6\(self, value, offset\)](#)
- [extract_20_8\(self, offset\)](#)
- [replace_20_8\(self, value, offset\)](#)
- [extract_20_10\(self, offset\)](#)
- [replace_20_10\(self, value, offset\)](#)
- [extract_20_12\(self, offset\)](#)
- [replace_20_12\(self, value, offset\)](#)
- [extract_20_16\(self, offset\)](#)
- [replace_20_16\(self, value, offset\)](#)
- [extract_22_1\(self, offset\)](#)
- [replace_22_1\(self, value, offset\)](#)
- [extract_22_2\(self, offset\)](#)
- [replace_22_2\(self, value, offset\)](#)
- [extract_22_4\(self, offset\)](#)
- [replace_22_4\(self, value, offset\)](#)
- [extract_22_6\(self, offset\)](#)
- [replace_22_6\(self, value, offset\)](#)
- [extract_22_8\(self, offset\)](#)
- [replace_22_8\(self, value, offset\)](#)
- [extract_22_10\(self, offset\)](#)
- [replace_22_10\(self, value, offset\)](#)

- 
- [`extract\_22\_12\(self, offset\)`](#)
 - [`replace\_22\_12\(self, value, offset\)`](#)
 - [`extract\_22\_16\(self, offset\)`](#)
 - [`replace\_22\_16\(self, value, offset\)`](#)
 - [`extract\_22\_20\(self, offset\)`](#)
 - [`replace\_22\_20\(self, value, offset\)`](#)
 - [`extract\_24\_1\(self, offset\)`](#)
 - [`replace\_24\_1\(self, value, offset\)`](#)
 - [`extract\_24\_2\(self, offset\)`](#)
 - [`replace\_24\_2\(self, value, offset\)`](#)
 - [`extract\_24\_4\(self, offset\)`](#)
 - [`replace\_24\_4\(self, value, offset\)`](#)
 - [`extract\_24\_6\(self, offset\)`](#)
 - [`replace\_24\_6\(self, value, offset\)`](#)
 - [`extract\_24\_8\(self, offset\)`](#)
 - [`replace\_24\_8\(self, value, offset\)`](#)
 - [`extract\_24\_10\(self, offset\)`](#)
 - [`replace\_24\_10\(self, value, offset\)`](#)
 - [`extract\_24\_12\(self, offset\)`](#)
 - [`replace\_24\_12\(self, value, offset\)`](#)
 - [`extract\_24\_16\(self, offset\)`](#)
 - [`replace\_24\_16\(self, value, offset\)`](#)
 - [`extract\_24\_20\(self, offset\)`](#)
 - [`replace\_24\_20\(self, value, offset\)`](#)
 - [`extract\_24\_22\(self, offset\)`](#)
 - [`replace\_24\_22\(self, value, offset\)`](#)
 - [`extract\_28\_1\(self, offset\)`](#)



- [replace_28_1\(self, value, offset\)](#)
- [extract_28_2\(self, offset\)](#)
- [replace_28_2\(self, value, offset\)](#)
- [extract_28_4\(self, offset\)](#)
- [replace_28_4\(self, value, offset\)](#)
- [extract_28_6\(self, offset\)](#)
- [replace_28_6\(self, value, offset\)](#)
- [extract_28_8\(self, offset\)](#)
- [replace_28_8\(self, value, offset\)](#)
- [extract_28_10\(self, offset\)](#)
- [replace_28_10\(self, value, offset\)](#)
- [extract_28_12\(self, offset\)](#)
- [replace_28_12\(self, value, offset\)](#)
- [extract_28_16\(self, offset\)](#)
- [replace_28_16\(self, value, offset\)](#)
- [extract_28_20\(self, offset\)](#)
- [replace_28_20\(self, value, offset\)](#)
- [extract_28_22\(self, offset\)](#)
- [replace_28_22\(self, value, offset\)](#)
- [extract_28_24\(self, offset\)](#)
- [replace_28_24\(self, value, offset\)](#)
- [extract_32_1\(self, offset\)](#)
- [replace_32_1\(self, value, offset\)](#)
- [extract_32_2\(self, offset\)](#)
- [replace_32_2\(self, value, offset\)](#)
- [extract_32_4\(self, offset\)](#)
- [replace_32_4\(self, value, offset\)](#)

- 
- [`extract\_32\_6\(self, offset\)`](#)
 - [`replace\_32\_6\(self, value, offset\)`](#)
 - [`extract\_32\_8\(self, offset\)`](#)
 - [`replace\_32\_8\(self, value, offset\)`](#)
 - [`extract\_32\_10\(self, offset\)`](#)
 - [`replace\_32\_10\(self, value, offset\)`](#)
 - [`extract\_32\_12\(self, offset\)`](#)
 - [`replace\_32\_12\(self, value, offset\)`](#)
 - [`extract\_32\_16\(self, offset\)`](#)
 - [`replace\_32\_16\(self, value, offset\)`](#)
 - [`extract\_32\_20\(self, offset\)`](#)
 - [`replace\_32\_20\(self, value, offset\)`](#)
 - [`extract\_32\_22\(self, offset\)`](#)
 - [`replace\_32\_22\(self, value, offset\)`](#)
 - [`extract\_32\_24\(self, offset\)`](#)
 - [`replace\_32\_24\(self, value, offset\)`](#)
 - [`extract\_32\_28\(self, offset\)`](#)
 - [`replace\_32\_28\(self, value, offset\)`](#)

Errors

- [`OutOfRangeAccess\(\)`](#)

`pack_1_1(bytes1 left, bytes1 right) → bytes2 result`

internal #

`pack_2_4(bytes2 left, bytes4 right) → bytes6 result`

internal #

`pack_2_6(bytes2 left, bytes6 right) → bytes8 result`

internal #

`pack_2_8(bytes2 left, bytes8 right) → bytes10 result`

internal #

`pack_2_10(bytes2 left, bytes10 right) → bytes12 result`

internal #

`pack_2_20(bytes2 left, bytes20 right) → bytes22 result`

internal #

`pack_2_22(bytes2 left, bytes22 right) → bytes24 result`

internal #

`pack_4_2(bytes4 left, bytes2 right) → bytes6 result`

internal #

`pack_4_4(bytes4 left, bytes4 right) → bytes8 result`

internal #

`pack_4_6(bytes4 left, bytes6 right) → bytes10 result`

internal #

`pack_4_8(bytes4 left, bytes8 right) → bytes12 result`

internal #

`pack_4_12(bytes4 left, bytes12 right) → bytes16 result`

internal #

`pack_4_16(bytes4 left, bytes16 right) → bytes20 result`

internal #

`pack_4_20(bytes4 left, bytes20 right) → bytes24 result` internal #

`pack_4_24(bytes4 left, bytes24 right) → bytes28 result` internal #

`pack_4_28(bytes4 left, bytes28 right) → bytes32 result` internal #

`pack_6_2(bytes6 left, bytes2 right) → bytes8 result` internal #

`pack_6_4(bytes6 left, bytes4 right) → bytes10 result` internal #

`pack_6_6(bytes6 left, bytes6 right) → bytes12 result` internal #

`pack_6_10(bytes6 left, bytes10 right) → bytes16 result` internal #

`pack_6_16(bytes6 left, bytes16 right) → bytes22 result` internal #

`pack_6_22(bytes6 left, bytes22 right) → bytes28 result` internal #

`pack_8_2(bytes8 left, bytes2 right) → bytes10 result` internal #

`pack_8_4(bytes8 left, bytes4 right) → bytes12 result` internal #

`pack_8_8(bytes8 left, bytes8 right) → bytes16 result` internal #

`pack_8_12(bytes8 left, bytes12 right) → bytes20 result` internal #

 **OpenZeppelin Docs**, `bytes16 right) → bytes24 result`

internal #

`pack_8_20(bytes8 left, bytes20 right) → bytes28 result`

internal #

`pack_8_24(bytes8 left, bytes24 right) → bytes32 result`

internal #

`pack_10_2(bytes10 left, bytes2 right) → bytes12 result`

internal #

`pack_10_6(bytes10 left, bytes6 right) → bytes16 result`

internal #

`pack_10_10(bytes10 left, bytes10 right) → bytes20 result`

internal #

`pack_10_12(bytes10 left, bytes12 right) → bytes22 result`

internal #

`pack_10_22(bytes10 left, bytes22 right) → bytes32 result`

internal #

`pack_12_4(bytes12 left, bytes4 right) → bytes16 result`

internal #

`pack_12_8(bytes12 left, bytes8 right) → bytes20 result`

internal #

`pack_12_10(bytes12 left, bytes10 right) → bytes22 result`

internal #

`pack_12_12(bytes12 left, bytes12 right) → bytes24 result`

internal #

`pack_12_16(bytes12 left, bytes16 right) → bytes28 result`

internal #

 **OpenZeppelin** Docs

<code>pack_12_20(bytes12 left, bytes20 right) → bytes32 result</code>	internal #
---	------------

<code>pack_16_4(bytes16 left, bytes4 right) → bytes20 result</code>	internal #
---	------------

<code>pack_16_6(bytes16 left, bytes6 right) → bytes22 result</code>	internal #
---	------------

<code>pack_16_8(bytes16 left, bytes8 right) → bytes24 result</code>	internal #
---	------------

<code>pack_16_12(bytes16 left, bytes12 right) → bytes28 result</code>	internal #
---	------------

<code>pack_16_16(bytes16 left, bytes16 right) → bytes32 result</code>	internal #
---	------------

<code>pack_20_2(bytes20 left, bytes2 right) → bytes22 result</code>	internal #
---	------------

<code>pack_20_4(bytes20 left, bytes4 right) → bytes24 result</code>	internal #
---	------------

<code>pack_20_8(bytes20 left, bytes8 right) → bytes28 result</code>	internal #
---	------------

<code>pack_20_12(bytes20 left, bytes12 right) → bytes32 result</code>	internal #
---	------------

<code>pack_22_2(bytes22 left, bytes2 right) → bytes24 result</code>	internal #
---	------------

<code>pack_22_6(bytes22 left, bytes6 right) → bytes28 result</code>	internal #
---	------------

<code>pack_22_10(bytes22 left, bytes10 right) → bytes32 result</code>	internal #
---	------------

 **OpenZeppelin** Docs `st, bytes4 right) → bytes28 result` internal #

`pack_24_8(bytes24 left, bytes8 right) → bytes32 result` internal #

`pack_28_4(bytes28 left, bytes4 right) → bytes32 result` internal #

`extract_2_1(bytes2 self, uint8 offset) → bytes1 result` internal #

`replace_2_1(bytes2 self, bytes1 value, uint8 offset) → bytes2 result` internal #

`extract_4_1(bytes4 self, uint8 offset) → bytes1 result` internal #

`replace_4_1(bytes4 self, bytes1 value, uint8 offset) → bytes4 result` internal #

`extract_4_2(bytes4 self, uint8 offset) → bytes2 result` internal #

`replace_4_2(bytes4 self, bytes2 value, uint8 offset) → bytes4 result` internal #

`extract_6_1(bytes6 self, uint8 offset) → bytes1 result` internal #

`replace_6_1(bytes6 self, bytes1 value, uint8 offset) → bytes6 result` internal #

`extract_6_2(bytes6 self, uint8 offset) → bytes2 result` internal #

`replace_6_2(bytes6 self, bytes2 value, uint8 offset) → bytes6 result` internal #

 **OpenZeppelin Docs**

`replace_6_4(bytes6 self, bytes4 value, uint8 offset) → bytes6 result` internal #

`extract_8_1(bytes8 self, uint8 offset) → bytes1 result` internal #

`replace_8_1(bytes8 self, bytes1 value, uint8 offset) → bytes8 result` internal #

`extract_8_2(bytes8 self, uint8 offset) → bytes2 result` internal #

`replace_8_2(bytes8 self, bytes2 value, uint8 offset) → bytes8 result` internal #

`extract_8_4(bytes8 self, uint8 offset) → bytes4 result` internal #

`replace_8_4(bytes8 self, bytes4 value, uint8 offset) → bytes8 result` internal #

`extract_8_6(bytes8 self, uint8 offset) → bytes6 result` internal #

`replace_8_6(bytes8 self, bytes6 value, uint8 offset) → bytes8 result` internal #

`extract_10_1(bytes10 self, uint8 offset) → bytes1 result` internal #

`replace_10_1(bytes10 self, bytes1 value, uint8 offset) → bytes10 result` internal #

`extract_10_2(bytes10 self, uint8 offset) → bytes2 result` internal #

 **OpenZeppelin Docs** `self, bytes2 value, uint8 offset) → bytes10 result` internal #

`extract_10_4(bytes10 self, uint8 offset) → bytes4 result` internal #

`replace_10_4(bytes10 self, bytes4 value, uint8 offset) → bytes10 result` internal #

`extract_10_6(bytes10 self, uint8 offset) → bytes6 result` internal #

`replace_10_6(bytes10 self, bytes6 value, uint8 offset) → bytes10 result` internal #

`extract_10_8(bytes10 self, uint8 offset) → bytes8 result` internal #

`replace_10_8(bytes10 self, bytes8 value, uint8 offset) → bytes10 result` internal #

`extract_12_1(bytes12 self, uint8 offset) → bytes1 result` internal #

`replace_12_1(bytes12 self, bytes1 value, uint8 offset) → bytes12 result` internal #

`extract_12_2(bytes12 self, uint8 offset) → bytes2 result` internal #

`replace_12_2(bytes12 self, bytes2 value, uint8 offset) → bytes12 result` internal #

`extract_12_4(bytes12 self, uint8 offset) → bytes4 result` internal #

`replace_12_4(bytes12 self, bytes4 value, uint8 offset) → bytes12 result` internal #

 **OpenZeppelin Docs**

`extract_12_6(bytes12 self, uint8 offset) → bytes6 result` internal #

`replace_12_6(bytes12 self, bytes6 value, uint8 offset) → bytes12 result` internal #

`extract_12_8(bytes12 self, uint8 offset) → bytes8 result` internal #

`replace_12_8(bytes12 self, bytes8 value, uint8 offset) → bytes12 result` internal #

`extract_12_10(bytes12 self, uint8 offset) → bytes10 result` internal #

`replace_12_10(bytes12 self, bytes10 value, uint8 offset) → bytes12 result` internal #

`extract_16_1(bytes16 self, uint8 offset) → bytes1 result` internal #

`replace_16_1(bytes16 self, bytes1 value, uint8 offset) → bytes16 result` internal #

`extract_16_2(bytes16 self, uint8 offset) → bytes2 result` internal #

`replace_16_2(bytes16 self, bytes2 value, uint8 offset) → bytes16 result` internal #

`extract_16_4(bytes16 self, uint8 offset) → bytes4 result` internal #

`replace_16_4(bytes16 self, bytes4 value, uint8 offset) → bytes16 result` internal #

`extract_16_6(bytes16 self, uint8 offset) → bytes6 result` internal #

 **OpenZeppelin** Docs

`replace_16_6(bytes16 self, bytes6 value, uint8 offset) → bytes16 result` internal #

`extract_16_8(bytes16 self, uint8 offset) → bytes8 result` internal #

`replace_16_8(bytes16 self, bytes8 value, uint8 offset) → bytes16 result` internal #

`extract_16_10(bytes16 self, uint8 offset) → bytes10 result` internal #

`replace_16_10(bytes16 self, bytes10 value, uint8 offset) → bytes16 result` internal #

`extract_16_12(bytes16 self, uint8 offset) → bytes12 result` internal #

`replace_16_12(bytes16 self, bytes12 value, uint8 offset) → bytes16 result` internal #

`extract_20_1(bytes20 self, uint8 offset) → bytes1 result` internal #

`replace_20_1(bytes20 self, bytes1 value, uint8 offset) → bytes20 result` internal #

`extract_20_2(bytes20 self, uint8 offset) → bytes2 result` internal #

`replace_20_2(bytes20 self, bytes2 value, uint8 offset) → bytes20 result` internal #

`extract_20_4(bytes20 self, uint8 offset) → bytes4 result` internal #

`replace_20_4(bytes20 self, bytes4 value, uint8 offset) → bytes20 result` internal #

 **OpenZeppelin Docs** `extract_20_6(bytes20 self, uint8 offset) → bytes6 result` internal #

`replace_20_6(bytes20 self, bytes6 value, uint8 offset) → bytes20 result` internal #

`extract_20_8(bytes20 self, uint8 offset) → bytes8 result` internal #

`replace_20_8(bytes20 self, bytes8 value, uint8 offset) → bytes20 result` internal #

`extract_20_10(bytes20 self, uint8 offset) → bytes10 result` internal #

`replace_20_10(bytes20 self, bytes10 value, uint8 offset) → bytes20 result` internal #

`extract_20_12(bytes20 self, uint8 offset) → bytes12 result` internal #

`replace_20_12(bytes20 self, bytes12 value, uint8 offset) → bytes20 result` internal #

`extract_20_16(bytes20 self, uint8 offset) → bytes16 result` internal #

`replace_20_16(bytes20 self, bytes16 value, uint8 offset) → bytes20 result` internal #

`extract_22_1(bytes22 self, uint8 offset) → bytes1 result` internal #

`replace_22_1(bytes22 self, bytes1 value, uint8 offset) → bytes22 result` internal #

 **OpenZeppelin Docs** `self, uint8 offset) → bytes2 result` internal #

`replace_22_2(bytes22 self, bytes2 value, uint8 offset) → bytes22 result` internal #

`extract_22_4(bytes22 self, uint8 offset) → bytes4 result` internal #

`replace_22_4(bytes22 self, bytes4 value, uint8 offset) → bytes22 result` internal #

`extract_22_6(bytes22 self, uint8 offset) → bytes6 result` internal #

`replace_22_6(bytes22 self, bytes6 value, uint8 offset) → bytes22 result` internal #

`extract_22_8(bytes22 self, uint8 offset) → bytes8 result` internal #

`replace_22_8(bytes22 self, bytes8 value, uint8 offset) → bytes22 result` internal #

`extract_22_10(bytes22 self, uint8 offset) → bytes10 result` internal #

`replace_22_10(bytes22 self, bytes10 value, uint8 offset) → bytes22 result` internal #

`extract_22_12(bytes22 self, uint8 offset) → bytes12 result` internal #

`replace_22_12(bytes22 self, bytes12 value, uint8 offset) → bytes22 result` internal #

`extract_22_16(bytes22 self, uint8 offset) → bytes16 result` internal #

 **OpenZeppelin Docs** `self, bytes16 value, uint8 offset) → bytes22 result` internal #

`extract_22_20(bytes22 self, uint8 offset) → bytes20 result` internal #

`replace_22_20(bytes22 self, bytes20 value, uint8 offset) → bytes22 result` internal #

`extract_24_1(bytes24 self, uint8 offset) → bytes1 result` internal #

`replace_24_1(bytes24 self, bytes1 value, uint8 offset) → bytes24 result` internal #

`extract_24_2(bytes24 self, uint8 offset) → bytes2 result` internal #

`replace_24_2(bytes24 self, bytes2 value, uint8 offset) → bytes24 result` internal #

`extract_24_4(bytes24 self, uint8 offset) → bytes4 result` internal #

`replace_24_4(bytes24 self, bytes4 value, uint8 offset) → bytes24 result` internal #

`extract_24_6(bytes24 self, uint8 offset) → bytes6 result` internal #

`replace_24_6(bytes24 self, bytes6 value, uint8 offset) → bytes24 result` internal #

`extract_24_8(bytes24 self, uint8 offset) → bytes8 result` internal #

`replace_24_8(bytes24 self, bytes8 value, uint8 offset) → bytes24 result` internal #

 **OpenZeppelin Docs** `self, uint8 offset) → bytes10 result` internal #

`replace_24_10(bytes24 self, bytes10 value, uint8 offset) → bytes24 result` internal #

`extract_24_12(bytes24 self, uint8 offset) → bytes12 result` internal #

`replace_24_12(bytes24 self, bytes12 value, uint8 offset) → bytes24 result` internal #

`extract_24_16(bytes24 self, uint8 offset) → bytes16 result` internal #

`replace_24_16(bytes24 self, bytes16 value, uint8 offset) → bytes24 result` internal #

`extract_24_20(bytes24 self, uint8 offset) → bytes20 result` internal #

`replace_24_20(bytes24 self, bytes20 value, uint8 offset) → bytes24 result` internal #

`extract_24_22(bytes24 self, uint8 offset) → bytes22 result` internal #

`replace_24_22(bytes24 self, bytes22 value, uint8 offset) → bytes24 result` internal #

`extract_28_1(bytes28 self, uint8 offset) → bytes1 result` internal #

`replace_28_1(bytes28 self, bytes1 value, uint8 offset) → bytes28 result` internal #

 **OpenZeppelin Docs** `extract_28_2(bytes28 self, uint8 offset) → bytes2 result` internal #

`replace_28_2(bytes28 self, bytes2 value, uint8 offset) → bytes28 result` internal #

`extract_28_4(bytes28 self, uint8 offset) → bytes4 result` internal #

`replace_28_4(bytes28 self, bytes4 value, uint8 offset) → bytes28 result` internal #

`extract_28_6(bytes28 self, uint8 offset) → bytes6 result` internal #

`replace_28_6(bytes28 self, bytes6 value, uint8 offset) → bytes28 result` internal #

`extract_28_8(bytes28 self, uint8 offset) → bytes8 result` internal #

`replace_28_8(bytes28 self, bytes8 value, uint8 offset) → bytes28 result` internal #

`extract_28_10(bytes28 self, uint8 offset) → bytes10 result` internal #

`replace_28_10(bytes28 self, bytes10 value, uint8 offset) → bytes28 result` internal #

`extract_28_12(bytes28 self, uint8 offset) → bytes12 result` internal #

`replace_28_12(bytes28 self, bytes12 value, uint8 offset) → bytes28 result` internal #

 **OpenZeppelin Docs** `self, uint8 offset) → bytes16 result` internal #

`replace_28_16(bytes28 self, bytes16 value, uint8 offset) → bytes28 result` internal #

`extract_28_20(bytes28 self, uint8 offset) → bytes20 result` internal #

`replace_28_20(bytes28 self, bytes20 value, uint8 offset) → bytes28 result` internal #

`extract_28_22(bytes28 self, uint8 offset) → bytes22 result` internal #

`replace_28_22(bytes28 self, bytes22 value, uint8 offset) → bytes28 result` internal #

`extract_28_24(bytes28 self, uint8 offset) → bytes24 result` internal #

`replace_28_24(bytes28 self, bytes24 value, uint8 offset) → bytes28 result` internal #

`extract_32_1(bytes32 self, uint8 offset) → bytes1 result` internal #

`replace_32_1(bytes32 self, bytes1 value, uint8 offset) → bytes32 result` internal #

`extract_32_2(bytes32 self, uint8 offset) → bytes2 result` internal #

`replace_32_2(bytes32 self, bytes2 value, uint8 offset) → bytes32 result` internal #

 **OpenZeppelin** Docs

`extract_32_4(bytes32 self, uint8 offset) → bytes4 result` internal #

`replace_32_4(bytes32 self, bytes4 value, uint8 offset) → bytes32 result` internal #

`extract_32_6(bytes32 self, uint8 offset) → bytes6 result` internal #

`replace_32_6(bytes32 self, bytes6 value, uint8 offset) → bytes32 result` internal #

`extract_32_8(bytes32 self, uint8 offset) → bytes8 result` internal #

`replace_32_8(bytes32 self, bytes8 value, uint8 offset) → bytes32 result` internal #

`extract_32_10(bytes32 self, uint8 offset) → bytes10 result` internal #

`replace_32_10(bytes32 self, bytes10 value, uint8 offset) → bytes32 result` internal #

`extract_32_12(bytes32 self, uint8 offset) → bytes12 result` internal #

`replace_32_12(bytes32 self, bytes12 value, uint8 offset) → bytes32 result` internal #

`extract_32_16(bytes32 self, uint8 offset) → bytes16 result` internal #

`replace_32_16(bytes32 self, bytes16 value, uint8 offset) → bytes32 result` internal #

`extract_32_20(bytes32 self, uint8 offset) → bytes20 result` internal #

 **OpenZeppelin Docs** `self, bytes20 value, uint8 offset) → bytes32 result` internal #

`extract_32_22(bytes32 self, uint8 offset) → bytes22 result` internal #

`replace_32_22(bytes32 self, bytes22 value, uint8 offset) → bytes32 result` internal #

`extract_32_24(bytes32 self, uint8 offset) → bytes24 result` internal #

`replace_32_24(bytes32 self, bytes24 value, uint8 offset) → bytes32 result` internal #

`extract_32_28(bytes32 self, uint8 offset) → bytes28 result` internal #

`replace_32_28(bytes32 self, bytes28 value, uint8 offset) → bytes32 result` internal #

`OutOfRangeAccess()` error #

Panic



```
import "@openzeppelin/contracts/utils/Panic.sol";
```



Helper library for emitting standardized panic codes.

OpenZeppelin Docs



```
// Use any of the declared internal constants
function foo() { Panic.GENERIC.panic(); }

// Alternatively
function foo() { Panic.panic(Panic.GENERIC); }

}
```

Follows the list from [libsolutil](#).

Available since v5.1.

Functions

- [panic\(code\)](#)

panic(uint256 code)

internal #

Reverts with a panic code. Recommended to use with the internal constants with predefined codes.

Pausable



```
import "@openzeppelin/contracts/utils/Pausable.sol";
```



Contract module which allows children to implement an emergency stop mechanism that can be triggered by an authorized account.

 This module is used through inheritance. It will make available the modifiers `whenNotPaused` and `whenPaused`, which can be applied to the functions of your contract. Note that they will not be pausable by simply including this module, only once the modifiers are put in place.

Modifiers

- [`whenNotPaused\(\)`](#)
- [`whenPaused\(\)`](#)

Functions

- [`paused\(\)`](#)
- [`\_requireNotPaused\(\)`](#)
- [`\_requirePaused\(\)`](#)
- [`\_pause\(\)`](#)
- [`\_unpause\(\)`](#)

Events

- [`Paused\(account\)`](#)
- [`Unpaused\(account\)`](#)

Errors

- [`EnforcedPause\(\)`](#)
- [`ExpectedPause\(\)`](#)

Modifier to make a function callable only when the contract is not paused.

Requirements:

- The contract must not be paused.

whenPaused()

internal #

Modifier to make a function callable only when the contract is paused.

Requirements:

- The contract must be paused.

paused() → bool

public #

Returns true if the contract is paused, and false otherwise.

_requireNotPaused()

internal #

Throws if the contract is paused.

_requirePaused()

internal #

Throws if the contract is not paused.

_pause()

internal #



Requirements:

- The contract must not be paused.

_unpause()

internal #

Returns to normal state.

Requirements:

- The contract must be paused.

Paused(address account)

event #

Emitted when the pause is triggered by `account`.

Unpaused(address account)

event #

Emitted when the pause is lifted by `account`.

EnforcedPause()

error #

The operation failed because the contract is paused.

ExpectedPause()

error #

The operation failed because the contract is not paused.



```
import "@openzeppelin/contracts/utils/RLP.sol";
```



Library for encoding and decoding data in RLP format. Recursive Length Prefix (RLP) is the main encoding method used to serialize objects in Ethereum. It's used for encoding everything from transactions to blocks to Patricia-Merkle tries.

Inspired by

- <https://github.com/succinctlabs/optimism-bedrock-contracts/blob/main/rlp/RLPWriter.sol>
- <https://github.com/succinctlabs/optimism-bedrock-contracts/blob/main/rlp/RLPReader.sol>

Canonical vs Non-Canonical Encodings

According to the Ethereum Yellow Paper, a "canonical" RLP encoding is the unique, minimal representation of a value. For scalar values (integers), this means:

- No leading zero bytes (e.g., `0x0123` should be encoded as 2 bytes, not `0x000123` as 3 bytes)
- Single bytes less than `0x80` must be encoded directly without a prefix wrapper
- Zero is represented as an empty byte array (prefix `0x80`)

A "non-canonical" encoding represents the same value but doesn't follow these minimality rules. For example, encoding the integer 1234 (`0x04d2`) with a leading zero as `0x830004d2` instead of the canonical `0x8204d2`.



This implementation takes a permissive approach to decoding, accepting some non-canonical encodings (e.g., scalar values with leading zero bytes) that would be rejected by strict implementations like go-ethereum. This design choice prioritizes compatibility with diverse RLP

encoders in the ecosystem over strict adherence to the Yellow Paper specification's requirements.



OpenZeppelin Docs

Users should be aware that:

- Multiple different RLP encodings may decode to the same value (non-injective)
- Encoding followed by decoding is guaranteed to work correctly
- External RLP data from untrusted sources may have non-canonical encodings
- Improperly wrapped single bytes (< 0x80) are still rejected as invalid

Functions

- [encoder\(\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [push\(self, input\)](#)
- [encode\(input\)](#)
- [encode\(input\)](#)
- [encode\(input\)](#)
- [encode\(input\)](#)
- [encode\(input\)](#)
- [encode\(input\)](#)
- [encode\(self\)](#)
- [readBool\(item\)](#)
- [readAddress\(item\)](#)



- [readUint256\(item\)](#)
- [readBytes32\(item\)](#)
- [readBytes\(item\)](#)
- [readString\(item\)](#)
- [readList\(item\)](#)
- [decodeBool\(item\)](#)
- [decodeAddress\(item\)](#)
- [decodeUint256\(item\)](#)
- [decodeBytes32\(item\)](#)
- [decodeBytes\(item\)](#)
- [decodeString\(item\)](#)
- [decodeList\(value\)](#)

Errors

- [RLPInvalidEncoding\(\)](#)

encoder() → struct RLP.Encoder enc

internal #

Create an empty RLP Encoder.

push(struct RLP.Encoder self, bool input) → struct RLP.Encoder

internal #

Add a boolean to a given RLP Encoder.

push(struct RLP.Encoder self, address input) → struct RLP.Encoder

internal #



```
push(struct RLP.Encoder self, uint256 input) → struct RLP.Encoder internal #
```

Add a uint256 to a given RLP Encoder.

```
push(struct RLP.Encoder self, bytes32 input) → struct RLP.Encoder internal #
```

Add a bytes32 to a given RLP Encoder.

```
push(struct RLP.Encoder self, bytes input) → struct RLP.Encoder internal #
```

Add a bytes buffer to a given RLP Encoder.

```
push(struct RLP.Encoder self, string input) → struct RLP.Encoder internal #
```

Add a string to a given RLP Encoder.

```
push(struct RLP.Encoder self, bytes[] input) → struct RLP.Encoder internal #
```

Add an array of bytes to a given RLP Encoder.

```
push(struct RLP.Encoder self, struct RLP.Encoder input) → struct RLP.Encoder internal #
```

Add an (input) Encoder to a (target) Encoder. The input is RLP encoded as a list of bytes, and added to the target Encoder.

OpenZeppelin Docs

internal #

Encode a boolean as RLP.

Boolean `true` is encoded as 0x01, `false` as 0x80 (equivalent to encoding integers 1 and 0). This follows the de facto ecosystem standard where booleans are treated as 0/1 integers.

encode(address input) → bytes result

internal #

Encode an address as an RLP item of fixed size (20 bytes).

The address is encoded with its leading zeros (if it has any). If someone wants to encode the address as a scalar, they can cast it to an uint256 and then call the corresponding `Base58.encode` function.

encode(uint256 input) → bytes result

internal #

Encode an uint256 as an RLP scalar.

Unlike `encode-bytes32`, this function uses scalar encoding that removes the prefix zeros.

encode(bytes32 input) → bytes result

internal #

Encode a bytes32 as an RLP item of fixed size (32 bytes).

Unlike `encode-uint256`, this function uses array encoding that preserves the prefix zeros.

encode(bytes input) → bytes

internal #



encode(string input) → bytes

internal #

Encode a string as RLP. Type alias for `encode-bytes-`.

encode(bytes[] input) → bytes

internal #

Encode an array of bytes as RLP. This function expects an array of already encoded bytes, not raw bytes. Users should call `Base58.encode` on each element of the array before calling it.

encode(struct RLP.Encoder self) → bytes

internal #

Encode an encoder (list of bytes) as RLP

readBool(Memory.Slice item) → bool

internal #

Decode an RLP encoded bool. See `encode-bool`

i This function treats any non-zero value as `true`, which is more permissive than some implementations (e.g., go-ethereum only accepts `0x00` for false and `0x01` for true). For example, `0x02`, `0x03`, etc. will all decode as `true`.

readAddress(Memory.Slice item) → address

internal #

Decode an RLP encoded address. See `encode-address`



This function accepts both single-byte encodings (for values 0-127, including precompile addresses like 0x01) and the standard 21-byte encoding with the 0x94 prefix. For example, 0x01 decodes to 0x0001.

Additionally, like `RLP.readUint256`, this function accepts non-canonical encodings with leading zeros. For instance, both 0x01 and

0x940001 decode to the same address.

`readUint256(Memory.Slice item) → uint256`

internal #

Decode an RLP encoded uint256. See `encode-uint256`

i This function accepts non-canonical encodings with leading zero bytes for multi-byte values, which differs from the Ethereum Yellow Paper specification and some reference implementations like go-ethereum. For example, both 0x88ab54a98ceb1f0ad2 and 0x8900ab54a98ceb1f0ad2 will decode to the same uint256 value (12345678901234567890).

However, single bytes less than 0x80 must NOT be wrapped with a prefix. For example, 0x8100 is invalid (should be 0x00), but 0x820000 is valid (two zero bytes).

This permissive behavior is intentional for compatibility with various RLP encoders in the ecosystem, but users should be aware that multiple RLP encodings may map to the same decoded value (non-injective decoding).

`readBytes32(Memory.Slice item) → bytes32`

internal #

Decode an RLP encoded bytes32. See `encode-bytes32`

i Since this function delegates to `RLP.readUint256`, it inherits the non-canonical encoding acceptance behavior for multi-byte values. Multiple RLP encodings with different leading zero bytes may decode to the same bytes32 value, but single bytes < 0x80 must not be wrapped with a prefix (e.g., 0x820000 is valid, but 0x8100 is not).

`readBytes(Memory.Slice item) → bytes`

internal #

 Decodes an RLP encoded bytes. See [encode-bytes](#)

OpenZeppelin Docs

`readString(Memory.Slice item) → string`

internal #

Decodes an RLP encoded string. See [encode-string](#)

`readList(Memory.Slice item) → Memory.Slice[] list`

internal #

Decodes an RLP encoded list in a memory slice into an array of RLP Items.

i The returned array contains slice references into the original payload, not copied bytes. Any further modification of the input buffer may cause the output result to become invalid.

`decodeBool(bytes item) → bool`

internal #

Decode an RLP encoded bool from bytes. See [RLP.readBool](#)

`decodeAddress(bytes item) → address`

internal #

Decode an RLP encoded address from bytes. See [RLP.readAddress](#)

`decodeUint256(bytes item) → uint256`

internal #

Decode an RLP encoded uint256 from bytes. See [RLP.readUint256](#)

`decodeBytes32(bytes item) → bytes32`

internal #



Decode an RLP encoded bytes32 from bytes. See [RLP.readBytes32](#)

decodeBytes(bytes item) → bytes

internal #

Decode an RLP encoded bytes from bytes. See [RLP.readBytes](#)

decodeString(bytes item) → string

internal #

Decode an RLP encoded string from bytes. See [RLP.readString](#)

decodeList(bytes value) → Memory.Slice[]

internal #

Decode an RLP encoded list from bytes. See [RLP.readList](#)



The returned array contains slice references into the original payload, not copied bytes. Any further modification of the input buffer may cause the output result to become invalid.

RLPInvalidEncoding()

error #

The item is not properly formatted and cannot be decoded.

ReentrancyGuard



```
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
```



Contract module that helps prevent reentrant calls to a function.



Inheriting from `ReentrancyGuard` will make the `ReentrancyGuard.nonReentrant` modifier available, which can be applied to functions to make sure there are no nested (reentrant) calls to them.

Note that because there is a single `nonReentrant` guard, functions marked as `nonReentrant` may not call one another. This can be worked around by making those functions `private`, and then adding `external nonReentrant` entry points to them.

i If EIP-1153 (transient storage) is available on the chain you're deploying at, consider using `ReentrancyGuardTransient` instead.

i If you would like to learn more about reentrancy and alternative ways to protect against it, check out our blog post [Reentrancy After Istanbul](#).

⚠ Deprecated. This storage-based reentrancy guard will be removed and replaced by the `ReentrancyGuardTransient` variant in v6.0.

@custom:stateless

Modifiers

- [nonReentrant\(\)](#)
- [nonReentrantView\(\)](#)

Functions

- [constructor\(\)](#)
- [_reentrancyGuardEntered\(\)](#)
- [_reentrancyGuardStorageSlot\(\)](#)

- [ReentrancyGuardReentrantCall\(\)](#)

nonReentrant()

internal #

Prevents a contract from calling itself, directly or indirectly. Calling a `nonReentrant` function from another `nonReentrant` function is not supported. It is possible to prevent this from happening by making the `nonReentrant` function external, and making it call a `private` function that does the actual work.

nonReentrantView()

internal #

A `view` only version of [ReentrancyGuard.nonReentrant](#). Use to block view functions from being called, preventing reading from inconsistent contract state.

 This is a "view" modifier and does not change the reentrancy status. Use it only on view functions. For payable or non-payable functions, use the standard [ReentrancyGuard.nonReentrant](#) modifier instead.

constructor()

internal #

_reentrancyGuardEntered() → bool

internal #

Returns true if the reentrancy guard is currently set to "entered", which indicates there is a `nonReentrant` function in the call stack.

_reentrancyGuardStorageSlot() → bytes32

internal #

Unauthorized reentrant call.

ReentrancyGuardTransient



```
import "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";
```



Variant of [ReentrancyGuard](#) that uses transient storage.

 This variant only works on networks where EIP-1153 is available.

Available since v5.1.

@custom:stateless

Modifiers

- [nonReentrant\(\)](#)
- [nonReentrantView\(\)](#)

Functions

- [_reentrancyGuardEntered\(\)](#)
- [_reentrancyGuardStorageSlot\(\)](#)

- [ReentrancyGuardReentrantCall\(\)](#)

nonReentrant()

internal #

Prevents a contract from calling itself, directly or indirectly. Calling a `nonReentrant` function from another `nonReentrant` function is not supported. It is possible to prevent this from happening by making the `nonReentrant` function external, and making it call a `private` function that does the actual work.

nonReentrantView()

internal #

A `view` only version of [ReentrancyGuard.nonReentrant](#). Use to block view functions from being called, preventing reading from inconsistent contract state.

 This is a "view" modifier and does not change the reentrancy status. Use it only on view functions. For payable or non-payable functions, use the standard [ReentrancyGuard.nonReentrant](#) modifier instead.

_reentrancyGuardEntered() → bool

internal #

Returns true if the reentrancy guard is currently set to "entered", which indicates there is a `nonReentrant` function in the call stack.

_reentrancyGuardStorageSlot() → bytes32

internal #

ReentrancyGuardReentrantCall()

error #

RelayedCall



```
import "@openzeppelin/contracts/utils/RelayedCall.sol";
```



Library for performing external calls through dynamically deployed relay contracts that hide the original caller's address from the target contract. This pattern is used in ERC-4337's EntryPoint for account factory calls and ERC-6942 for safe factory interactions.

When privileged contracts need to make arbitrary external calls based on user input, calling the target directly can be risky because the target sees the privileged contract as `msg.sender` and could exploit this trust relationship. This library solves this by deploying minimal relay contracts that act as intermediaries, ensuring the target only sees the unprivileged relay address as `msg.sender`.

For example, instead of `target.call(data)` where the target sees this contract as `msg.sender`, use [RelayedCall.relayCall](#) where the target sees a relay address as `msg.sender`.

i This library uses the PUSH0 opcode that was introduced in the Shanghai hardfork. While this instruction is now widely supported, developers using the library on exotic chains should verify that their target chain has supports for EIP-3855.

Functions

- [relayCall\(target, data\)](#)
- [relayCall\(target, value, data\)](#)
- [relayCall\(target, data, salt\)](#)
- [relayCall\(target, value, data, salt\)](#)



relayCall(address target, bytes data) → bool, bytes

internal #

Relays a call to the target contract through a dynamically deployed relay contract.

relayCall(address target, uint256 value, bytes data) → bool, bytes

internal #

Same as [RelayedCall.relayCall](#) but with a value.

relayCall(address target, bytes data, bytes32 salt) → bool, bytes

internal #

Same as [RelayedCall.relayCall](#) but with a salt.

relayCall(address target, uint256 value, bytes data, bytes32 salt) → bool, bytes

internal #

Same as [RelayedCall.relayCall](#) but with a salt and a value.

getRelayer() → address

internal #

Same as [RelayedCall.getRelayer](#) but with a `bytes32(0)` default salt.

getRelayer(bytes32 salt) → address relayer

internal #

 Returns the relayer address for a given salt.

ShortString



```
import "@openzeppelin/contracts/utils/ShortStrings.sol";
```



ShortStrings



```
import "@openzeppelin/contracts/utils/ShortStrings.sol";
```



This library provides functions to convert short memory strings into a `ShortString` type that can be used as an immutable variable.

Strings of arbitrary length can be optimized using this library if they are short enough (up to 31 bytes) by packing them with their length (1 byte) in a single EVM word (32 bytes). Additionally, a fallback mechanism can be used for every other case.

Usage example:

```
contract Named {  
    using ShortStrings for *;  
  
    ShortString private immutable _name;  
    string private _nameFallback;  
  
    constructor(string memory contractName) {  
        _name = contractName.toShortStringWithFallback(_nameFallback);  
    }  
}
```



```
function name() external view returns (string memory) {
  stringWithFallback(_nameFallback);
}
```

Functions

- [toShortString\(str\)](#)
- [toString\(sstr\)](#)
- [byteLength\(sstr\)](#)
- [toShortStringWithFallback\(value, store\)](#)
- [toStringWithFallback\(value, store\)](#)
- [byteLengthWithFallback\(value, store\)](#)

Errors

- [StringTooLong\(str\)](#)
- [InvalidShortString\(\)](#)

toShortString(string str) → ShortString

internal #

Encode a string of at most 31 chars into a `ShortString`.

This will trigger a `StringTooLong` error if the input string is too long.

toString(ShortString sstr) → string

internal #

Decode a `ShortString` back to a "normal" string.


`byteLength(ShortString sstr) → uint256`

internal #

Return the length of a `ShortString`.

`toShortStringWithFallback(string value, string store) → ShortString`

internal #

Encode a string into a `ShortString`, or write it to storage if it is too long.

`toStringWithFallback(ShortString value, string store) → string`

internal #

Decode a string that was encoded to `ShortString` or written to storage using [`ShortStrings.toShortStringWithFallback`](#).

`byteLengthWithFallback(ShortString value, string store) → uint256`

internal #

Return the length of a string that was encoded to `ShortString` or written to storage using [`ShortStrings.toShortStringWithFallback`](#).

 This will return the "byte length" of the string. This may not reflect the actual length in terms of actual characters as the UTF-8 encoding of a single character can span over multiple bytes.

`StringTooLong(string str)`

error #

`InvalidShortString()`

error #



```
import "@openzeppelin/contracts/utils/SlotDerivation.sol";
```



Library for computing storage (and transient storage) locations from namespaces and deriving slots corresponding to standard patterns. The derivation method for array and mapping matches the storage layout used by the solidity language / compiler.

See [Solidity docs for mappings and dynamic arrays](#)..

Example usage:

```
contract Example {  
    // Add the library methods  
    using StorageSlot for bytes32;  
    using SlotDerivation for *;  
  
    // Declare a namespace  
    string private constant _NAMESPACE = "<namespace>"; // eg. OpenZeppelin.Slot  
  
    function setValueInNamespace(uint256 key, address newValue) internal {  
        _NAMESPACE.erc7201Slot().deriveMapping(key).getAddressSlot().value = newValue;  
    }  
  
    function getValueInNamespace(uint256 key) internal view returns (address) {  
        return _NAMESPACE.erc7201Slot().deriveMapping(key).getAddressSlot().value;  
    }  
}
```



 Consider using this library along with [StorageSlot](#).

 This library provides a way to manipulate storage locations in a non-standard way. Tooling for checking upgrade safety will ignore the slots accessed through this library.

Available since v5.1.

Functions

- [erc7201Slot\(namespace\)](#)
- [offset\(slot, pos\)](#)
- [deriveArray\(slot\)](#)

- [deriveMapping\(slot, key\)](#)
- [deriveMapping\(slot, key\)](#)
- [deriveMapping\(slot, key\)](#)
- [deriveMapping\(slot, key\)](#)
- [deriveMapping\(slot, key\)](#)
- [deriveMapping\(slot, key\)](#)
- [deriveMapping\(slot, key\)](#)

erc7201Slot(string namespace) → bytes32 slot

internal #

Derive an ERC-7201 slot from a string (namespace).

offset(bytes32 slot, uint256 pos) → bytes32 result

internal #

Add an offset to a slot to get the n-th element of a structure or an array.

deriveArray(bytes32 slot) → bytes32 result

internal #

Derive the location of the first element in an array from the slot where the length is stored.

deriveMapping(bytes32 slot, address key) → bytes32 result

internal #



`deriveMapping(bytes32 slot, bool key) → bytes32 result`

internal #

Derive the location of a mapping element from the key.

`deriveMapping(bytes32 slot, bytes32 key) → bytes32 result`

internal #

Derive the location of a mapping element from the key.

`deriveMapping(bytes32 slot, uint256 key) → bytes32 result`

internal #

Derive the location of a mapping element from the key.

`deriveMapping(bytes32 slot, int256 key) → bytes32 result`

internal #

Derive the location of a mapping element from the key.

`deriveMapping(bytes32 slot, string key) → bytes32 result`

internal #

Derive the location of a mapping element from the key.

`deriveMapping(bytes32 slot, bytes key) → bytes32 result`

internal #

Derive the location of a mapping element from the key.



```
import "@openzeppelin/contracts/utils/StorageSlot.sol";
```



Library for reading and writing primitive types to specific storage slots.

Storage slots are often used to avoid storage conflict when dealing with upgradeable contracts. This library helps with reading and writing to such slots without the need for inline assembly.

The functions in this library return Slot structs that contain a `value` member that can be used to read or write.

Example usage to set ERC-1967 implementation slot:

```
contract ERC1967 {  
    // Define the slot. Alternatively, use the SlotDerivation library to derive the slot.  
    bytes32 internal constant _IMPLEMENTATION_SLOT = 0x360894a13ba1a3210667c828492db98dca  
  
    function _getImplementation() internal view returns (address) {  
        return StorageSlot.getAddressSlot(_IMPLEMENTATION_SLOT).value;  
    }  
  
    function _setImplementation(address newImplementation) internal {  
        require(newImplementation.code.length > 0);  
        StorageSlot.getAddressSlot(_IMPLEMENTATION_SLOT).value = newImplementation;  
    }  
}
```



 Consider using this library along with [SlotDerivation](#).

Functions

- [getAddressSlot\(slot\)](#)
- [getBooleanSlot\(slot\)](#)
- [getBytes32Slot\(slot\)](#)



- [getUint256Slot\(slot\)](#)
- [getInt256Slot\(slot\)](#)
- [getStringSlot\(slot\)](#)
- [getStringSlot\(store\)](#)
- [getBytesSlot\(slot\)](#)
- [getBytesSlot\(store\)](#)

getAddressSlot(bytes32 slot) → struct StorageSlot.AddressSlot r internal #

Returns an `AddressSlot` with member `value` located at `slot`.

getBooleanSlot(bytes32 slot) → struct StorageSlot.BooleanSlot r internal #

Returns a `BooleanSlot` with member `value` located at `slot`.

getBytes32Slot(bytes32 slot) → struct StorageSlot.Bytes32Slot r internal #

Returns a `Bytes32Slot` with member `value` located at `slot`.

getUint256Slot(bytes32 slot) → struct StorageSlot.Uint256Slot r internal #

Returns a `Uint256Slot` with member `value` located at `slot`.

getInt256Slot(bytes32 slot) → struct StorageSlot.Int256Slot r internal #

Returns a `Int256Slot` with member `value` located at `slot`.

 **OpenZeppelin Docs** `slot) → struct StorageSlot.StringSlot r` internal #

Returns a `StringSlot` with member `value` located at `slot`.

`getStringSlot(string store) → struct StorageSlot.StringSlot r` internal #

Returns an `StringSlot` representation of the string storage pointer `store`.

`getBytesSlot(bytes32 slot) → struct StorageSlot.BytesSlot r` internal #

Returns a `BytesSlot` with member `value` located at `slot`.

`getBytesSlot(bytes store) → struct StorageSlot.BytesSlot r` internal #

Returns an `BytesSlot` representation of the bytes storage pointer `store`.

Strings



```
import "@openzeppelin/contracts/utils/Strings.sol";
```



String operations.

Functions

- [`toString\(value\)`](#)



- [toStringSigned\(value\)](#)
- [toHexString\(value\)](#)
- [toHexString\(value, length\)](#)
- [toHexString\(addr\)](#)
- [toChecksumHexString\(addr\)](#)
- [toHexString\(input\)](#)
- [equal\(a, b\)](#)
- [parseUint\(input\)](#)
- [parseUint\(input, begin, end\)](#)
- [tryParseUint\(input\)](#)
- [tryParseUint\(input, begin, end\)](#)
- [parseInt\(input\)](#)
- [parseInt\(input, begin, end\)](#)
- [tryParseInt\(input\)](#)
- [tryParseInt\(input, begin, end\)](#)
- [parseHexUint\(input\)](#)
- [parseHexUint\(input, begin, end\)](#)
- [tryParseHexUint\(input\)](#)
- [tryParseHexUint\(input, begin, end\)](#)
- [parseAddress\(input\)](#)
- [parseAddress\(input, begin, end\)](#)
- [tryParseAddress\(input\)](#)
- [tryParseAddress\(input, begin, end\)](#)
- [escapeJSON\(input\)](#)

Errors

- [StringsInsufficientHexLength\(value, length\)](#)
- [StringsInvalidChar\(\)](#)
- [StringsInvalidAddressFormat\(\)](#)

toString(uint256 value) → string

internal #

Converts a `uint256` to its ASCII `string` decimal representation.

toStringSigned(int256 value) → string

internal #

Converts a `int256` to its ASCII `string` decimal representation.

toHexString(uint256 value) → string

internal #

Converts a `uint256` to its ASCII `string` hexadecimal representation.

toHexString(uint256 value, uint256 length) → string

internal #

Converts a `uint256` to its ASCII `string` hexadecimal representation with fixed length.

toHexString(address addr) → string

internal #

Converts an `address` with fixed length of 20 bytes to its not checksummed ASCII `string` hexadecimal representation.

toChecksumHexString(address addr) → string

internal #

 Converts an `address` with fixed length of 20 bytes to its checksummed ASCII `string` hexadecimal representation, according to EIP-55.

`toHexString(bytes input) → string`

internal #

Converts a `bytes` buffer to its ASCII `string` hexadecimal representation.

`equal(string a, string b) → bool`

internal #

Returns true if the two strings are equal.

`parseUint(string input) → uint256`

internal #

Parse a decimal string and returns the value as a `uint256`.

Requirements:

- The string must be formatted as `[0-9]*`
- The result must fit into an `uint256` type

`parseUint(string input, uint256 begin, uint256 end) → uint256`

internal #

Variant of `parseUint-string` that parses a substring of `input` located between position `begin` (included) and `end` (excluded).

Requirements:

- The substring must be formatted as `[0-9]*`
- The result must fit into an `uint256` type

Variant of `parseInt-string` that returns false if the parsing fails because of an invalid character.

 This function will revert if the result does not fit in a `uint256`.

tryParseUint(string input, uint256 begin, uint256 end) → bool success, uint256 value internal #

Variant of `parseInt-string-uint256-uint256` that returns false if the parsing fails because of an invalid character.

 This function will revert if the result does not fit in a `uint256`.

parseInt(string input) → int256 internal #

Parse a decimal string and returns the value as a `int256`.

Requirements:

- The string must be formatted as `[ -+ ]?[ 0-9 ]*`
- The result must fit in an `int256` type.

parseInt(string input, uint256 begin, uint256 end) → int256 internal #

Variant of `parseInt-string` that parses a substring of `input` located between position `begin` (included) and `end` (excluded).

Requirements:



The substring must be formatted as `[ -+ ]?[0-9]*`

- The result must fit in an `int256` type.

tryParseInt(string input) → bool success, int256 value

internal #

Variant of `parseInt-string` that returns false if the parsing fails because of an invalid character or if the result does not fit in a `int256`.

i This function will revert if the absolute value of the result does not fit in a `uint256`.

tryParseInt(string input, uint256 begin, uint256 end) → bool success, int256 value

internal #

Variant of `parseInt-string-uint256-uint256` that returns false if the parsing fails because of an invalid character or if the result does not fit in a `int256`.

i This function will revert if the absolute value of the result does not fit in a `uint256`.

parseHexUint(string input) → uint256

internal #

Parse a hexadecimal string (with or without "0x" prefix), and returns the value as a `uint256`.

Requirements:

- The string must be formatted as `(0x)?[0-9a-fA-F]*`
- The result must fit in an `uint256` type.

parseHexUint(string input, uint256 begin, uint256 end) → uint256

internal #

 Variant of `parseHexUint-string` that parses a substring of `input` located between position `begin` (included) and `end` (excluded).

Requirements:

- The substring must be formatted as `(0x)?[0-9a-fA-F]*`
- The result must fit in an `uint256` type.

`tryParseHexUint(string input) → bool success, uint256 value`

internal #

Variant of `parseHexUint-string` that returns false if the parsing fails because of an invalid character.

 This function will revert if the result does not fit in a `uint256`.

`tryParseHexUint(string input, uint256 begin, uint256 end) → bool success, uint256 value`

internal #

Variant of `parseHexUint-string-uint256-uint256` that returns false if the parsing fails because of an invalid character.

 This function will revert if the result does not fit in a `uint256`.

`parseAddress(string input) → address`

internal #

Parse a hexadecimal string (with or without "0x" prefix), and returns the value as an `address`.

Requirements:



parseAddress(string input, uint256 begin, uint256 end) → address internal #

Variant of `parseAddress-string` that parses a substring of `input` located between position `begin` (included) and `end` (excluded).

Requirements:

- The substring must be formatted as `(0x)?[0-9a-fA-F] 40```

tryParseAddress(string input) → bool success, address value internal #

Variant of `parseAddress-string` that returns false if the parsing fails because the input is not a properly formatted address. See `parseAddress-string` requirements.

tryParseAddress(string input, uint256 begin, uint256 end) → bool success, address value internal #

Variant of `parseAddress-string-uint256-uint256` that returns false if the parsing fails because input is not a properly formatted address. See `parseAddress-string-uint256-uint256` requirements.

escapeJSON(string input) → string internal #

Escape special characters in JSON strings. This can be useful to prevent JSON injection in NFT metadata.



This function should only be used in double quoted JSON strings. Single quotes are not escaped.



This function escapes backslashes (including those in `\uXXXX` sequences) and the characters in ranges defined in section 2.5 of RFC-4627 (U+0000 to U+001F, U+0022 and U+005C). All control characters in U+0000 to U+001F are escaped (`\b`, `\t`, `\n`, `\f`, `\r` use short form; others use `\u00XX`). ECMAScript's `JSON.parse` does recover escaped unicode characters that are not in this range, but other tooling may provide different results.

StringsInsufficientHexLength(uint256 value, uint256 length)

error #

The `value` string doesn't fit in the specified `length`.

StringsInvalidChar()

error #

The string being parsed contains characters that are not in scope of the given base.

StringsInvalidAddressFormat()

error #

The string being parsed is not a properly formatted address.

TransientSlot



```
import "@openzeppelin/contracts/utils/TransientSlot.sol";
```



Library for reading and writing value-types to specific transient storage slots.

Transient slots are often used to store temporary values that are removed after the current transaction. This library helps with reading and writing to such slots without the need for inline assembly.

- Example reading and writing values using transient storage:



```
contract Lock {
    using TransientSlot for *;
```



```
    // Define the slot. Alternatively, use the SlotDerivation library to derive the slot.
    bytes32 internal constant _LOCK_SLOT = 0xf4678858b2b588224636b8522b729e7722d32fc491da

    modifier locked() {
        require(!_LOCK_SLOT.asBoolean().tload());

        _LOCK_SLOT.asBoolean().tstore(true);
        _;
        _LOCK_SLOT.asBoolean().tstore(false);
    }
}
```

i Consider using this library along with [SlotDerivation](#).

Functions

- [asAddress\(slot\)](#)
- [asBoolean\(slot\)](#)
- [asBytes32\(slot\)](#)
- [asUint256\(slot\)](#)
- [asInt256\(slot\)](#)
- [tload\(slot\)](#)
- [tstore\(slot, value\)](#)
- [tload\(slot\)](#)
- [tstore\(slot, value\)](#)
- [tload\(slot\)](#)
- [tstore\(slot, value\)](#)
- [tload\(slot\)](#)
- [tstore\(slot, value\)](#)

asAddress(bytes32 slot) → TransientSlot.AddressSlot

internal #

Cast an arbitrary slot to a AddressSlot.

asBoolean(bytes32 slot) → TransientSlot.BooleanSlot

internal #

Cast an arbitrary slot to a BooleanSlot.

asBytes32(bytes32 slot) → TransientSlot.Bytes32Slot

internal #

Cast an arbitrary slot to a Bytes32Slot.

asUint256(bytes32 slot) → TransientSlot.Uint256Slot

internal #

Cast an arbitrary slot to a Uint256Slot.

asInt256(bytes32 slot) → TransientSlot.Int256Slot

internal #

Cast an arbitrary slot to a Int256Slot.

tload(TransientSlot.AddressSlot slot) → address value

internal #

Load the value held at location `slot` in transient storage.



Store `value` at location `slot` in transient storage.

`tload(TransientSlot.BooleanSlot slot) → bool value`

internal #

Load the value held at location `slot` in transient storage.

`tstore(TransientSlot.BooleanSlot slot, bool value)`

internal #

Store `value` at location `slot` in transient storage.

`tload(TransientSlot.Bytes32Slot slot) → bytes32 value`

internal #

Load the value held at location `slot` in transient storage.

`tstore(TransientSlot.Bytes32Slot slot, bytes32 value)`

internal #

Store `value` at location `slot` in transient storage.

`tload(TransientSlot.Uint256Slot slot) → uint256 value`

internal #

Load the value held at location `slot` in transient storage.

`tstore(TransientSlot.Uint256Slot slot, uint256 value)`

internal #

 Store `value` at location `slot` in transient storage.

`tload(TransientSlot.Int256Slot slot) → int256 value` internal #

Load the value held at location `slot` in transient storage.

`tstore(TransientSlot.Int256Slot slot, int256 value)` internal #

Store `value` at location `slot` in transient storage.

InteroperableAddress



```
import "@openzeppelin/contracts/utils/draft-InteroperableAddress.sol";
```



Helper library to format and parse [ERC-7930](#) interoperable addresses.

Functions

- [formatV1\(chainType, chainReference, addr\)](#)
- [formatEvmV1\(chainid, addr\)](#)
- [formatEvmV1\(chainid\)](#)
- [formatEvmV1\(addr\)](#)
- [parseV1\(self\)](#)
- [parseV1Calldata\(self\)](#)
- [tryParseV1\(self\)](#)

OpenZeppelin Docs

- [tryParseV1Calldata\(self\)](#)
- [parseEvmV1\(self\)](#)
- [parseEvmV1Calldata\(self\)](#)
- [tryParseEvmV1\(self\)](#)
- [tryParseEvmV1Calldata\(self\)](#)

Errors

- [InteroperableAddressParsingError\(\)](#)
- [InteroperableAddressEmptyReferenceAndAddress\(\)](#)

formatV1(bytes2 chainType, bytes chainReference, bytes addr) → bytes internal #

Format an ERC-7930 interoperable address (version 1) from its components `chainType`, `chainReference` and `addr`. This is a generic function that supports any chain type, chain reference and address supported by ERC-7930, including interoperable addresses with empty chain reference or empty address.

formatEvmV1(uint256 chainid, address addr) → bytes internal #

Variant of `formatV1-bytes2-bytes-bytes-` specific to EVM chains. Returns the ERC-7930 interoperable address (version 1) for a given chainid and ethereum address.

formatEvmV1(uint256 chainid) → bytes internal #

Variant of `formatV1-bytes2-bytes-bytes-` that specifies an EVM chain without an address.

 **formatEvmV1(address addr) → bytes**

internal #

Variant of `formatV1` that specifies an EVM address without a chain reference.

parseV1(bytes self) → bytes2 chainType, bytes chainReference, bytes addr internal #

Parse a ERC-7930 interoperable address (version 1) into its different components. Reverts if the input is not following a version 1 of ERC-7930.

 Trailing bytes after a valid v1 encoding are ignored. The same decoded address may therefore correspond to multiple distinct input byte strings.

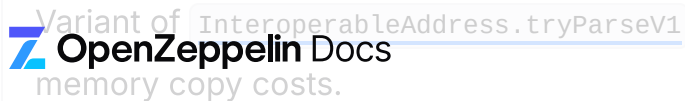
parseV1Calldata(bytes self) → bytes2 chainType, bytes chainReference, bytes addr internal #

Variant of `InteroperableAddress.parseV1` that handles calldata slices to reduce memory copy costs.

tryParseV1(bytes self) → bool success, bytes2 chainType, bytes chainReference, bytes addr internal #

Variant of `InteroperableAddress.parseV1` that does not revert on invalid input. Instead, it returns `false` as the first return value to indicate parsing failure when the input does not follow version 1 of ERC-7930.

tryParseV1Calldata(bytes self) → bool success, bytes2 chainType, bytes chainReference, bytes addr internal #



Variant of `InteroperableAddress.tryParseV1` that handles calldata slices to reduce

memory copy costs.

`parseEvmV1(bytes self) → uint256 chainId, address addr`

internal #

Parse a ERC-7930 interoperable address (version 1) corresponding to an EIP-155 chain. The `chainId` and `addr` return values will be zero if the input doesn't include a `chainReference` or an address, respectively.

i Trailing bytes after a valid v1 encoding are ignored. The same decoded (`chainId`, `addr`) may therefore correspond to multiple distinct input byte strings.

Requirements:

- The input must be a valid ERC-7930 interoperable address (version 1)
- The underlying `chainType` must be "eip-155"

`parseEvmV1Calldata(bytes self) → uint256 chainId, address addr`

internal #

Variant of `InteroperableAddress.parseEvmV1` that handles calldata slices to reduce memory copy costs.

`tryParseEvmV1(bytes self) → bool success, uint256 chainId, address addr` internal #

Variant of `InteroperableAddress.parseEvmV1` that does not revert on invalid input. Instead, it returns `false` as the first return value to indicate parsing failure when the input does not follow version 1 of ERC-7930.

`tryParseEvmV1Calldata(bytes self) → bool success, uint256 chainId, address addr`

internal #



Variant of `InteroperableAddress.tryParseEvmV1` that handles calldata slices to reduce memory copy costs.

InteroperableAddressParsingError(bytes)

error #

InteroperableAddressEmptyReferenceAndAddress()

error #

ERC165



```
import "@openzeppelin/contracts/utils/introspection/ERC165.sol";
```



Implementation of the [IERC165](#) interface.

Contracts that want to implement ERC-165 should inherit from this contract and override [ERC165.supportsInterface](#) to check for the additional interface id that will be supported. For example:

```
function supportsInterface(bytes4 interfaceId) public view virtual override returns (bool) {
    return interfaceId == type(MyInterface).interfaceId || super.supportsInterface(interf
}
```

Functions

- [supportsInterface\(interfaceId\)](#)

supportsInterface(bytes4 interfaceId) → bool

public #



Returns true if this contract implements the interface defined by `interfaceId`. See the corresponding [ERC section](#) to learn more about how these ids are created.

This function call must use less than 30 000 gas.

ERC165Checker



```
import "@openzeppelin/contracts/utils/introspection/ERC165Checker.sol";
```



Library used to query support of an interface declared via [IERC165](#).

Note that these functions return the actual result of the query: they do not `revert` if an interface is not supported. It is up to the caller to decide what to do in these cases.

Functions

- [supportsERC165\(account\)](#)
- [supportsInterface\(account, interfaceId\)](#)
- [getSupportedInterfaces\(account, interfaceIds\)](#)
- [supportsAllInterfaces\(account, interfaceIds\)](#)
- [supportsERC165InterfaceUnchecked\(account, interfaceId\)](#)

supportsERC165(address account) → bool

internal #

Returns true if `account` supports the [IERC165](#) interface.

supportsInterface(address account, bytes4 interfaceId) → bool internal #



Returns true if `account` supports the interface defined by `interfaceId`. Support for `IERC165` itself is queried automatically.

See [IERC165.supportsInterface](#).

getSupportedInterfaces(address account, bytes4[] interfaceIds) → bool[] internal #

Returns a boolean array where each value corresponds to the interfaces passed in and whether they're supported or not. This allows you to batch check interfaces for a contract where your expectation is that some interfaces may not be supported.

See [IERC165.supportsInterface](#).

supportsAllInterfaces(address account, bytes4[] interfaceIds) → bool internal #

Returns true if `account` supports all the interfaces defined in `interfaceIds`. Support for `IERC165` itself is queried automatically.

Batch-querying can lead to gas savings by skipping repeated checks for `IERC165` support.

See [IERC165.supportsInterface](#).

supportsERC165InterfaceUnchecked(address account, bytes4 interfaceId) → bool internal #

Assumes that `account` contains a contract that supports ERC-165, otherwise the behavior of this method is undefined. This precondition can be checked with

[ERC165Checker.supportsERC165](#).

 **OpenZeppelin Docs** Some precompiled contracts will falsely indicate support for a given interface, so caution should be exercised when using this function.

Interface identification is specified in ERC-165.

IERC165



```
import "@openzeppelin/contracts/utils/introspection/IERC165.sol";
```



Interface of the ERC-165 standard, as defined in the [ERC](#).

Implementers can declare support of contract interfaces, which can then be queried by others ([ERC165Checker](#)).

For an implementation, see [ERC165](#).

Functions

- [supportsInterface\(interfaceId\)](#)

supportsInterface(bytes4 interfaceId) → bool

external #

Returns true if this contract implements the interface defined by `interfaceId`. See the corresponding [ERC section](#) to learn more about how these ids are created.

This function call must use less than 30 000 gas.



```
import "@openzeppelin/contracts/utils/math/Math.sol";
```



Standard math utilities missing in the Solidity language.

Functions

- [add512\(a, b\)](#)
- [mul512\(a, b\)](#)
- [tryAdd\(a, b\)](#)
- [trySub\(a, b\)](#)
- [tryMul\(a, b\)](#)
- [tryDiv\(a, b\)](#)
- [tryMod\(a, b\)](#)
- [saturatingAdd\(a, b\)](#)
- [saturatingSub\(a, b\)](#)
- [saturatingMul\(a, b\)](#)
- [ternary\(condition, a, b\)](#)
- [max\(a, b\)](#)
- [min\(a, b\)](#)
- [average\(a, b\)](#)
- [ceilDiv\(a, b\)](#)
- [mulDiv\(x, y, denominator\)](#)
- [mulDiv\(x, y, denominator, rounding\)](#)
- [mulShr\(x, y, n\)](#)
- [mulShr\(x, y, n, rounding\)](#)

OpenZeppelin Docs

- [invMod\(a, n\)](#)
- [invModPrime\(a, p\)](#)
- [modExp\(b, e, m\)](#)
- [tryModExp\(b, e, m\)](#)
- [modExp\(b, e, m\)](#)
- [tryModExp\(b, e, m\)](#)
- [sqrt\(a\)](#)
- [sqrt\(a, rounding\)](#)
- [log2\(x\)](#)
- [log2\(value, rounding\)](#)
- [log10\(value\)](#)
- [log10\(value, rounding\)](#)
- [log256\(x\)](#)
- [log256\(value, rounding\)](#)
- [unsignedRoundsUp\(rounding\)](#)
- [clz\(x\)](#)

add512(uint256 a, uint256 b) → uint256 high, uint256 low

internal #

Return the 512-bit addition of two uint256.

The result is stored in two 256 variables such that $\text{sum} = \text{high} * 2^{256} + \text{low}$.

mul512(uint256 a, uint256 b) → uint256 high, uint256 low

internal #

Return the 512-bit multiplication of two uint256.



The result is stored in two 256 variables such that $\text{product} = \text{high} * 2^{256} + \text{low}$.

`tryAdd(uint256 a, uint256 b) → bool success, uint256 result` internal #

Returns the addition of two unsigned integers, with a success flag (no overflow).

`trySub(uint256 a, uint256 b) → bool success, uint256 result` internal #

Returns the subtraction of two unsigned integers, with a success flag (no overflow).

`tryMul(uint256 a, uint256 b) → bool success, uint256 result` internal #

Returns the multiplication of two unsigned integers, with a success flag (no overflow).

`tryDiv(uint256 a, uint256 b) → bool success, uint256 result` internal #

Returns the division of two unsigned integers, with a success flag (no division by zero).

`tryMod(uint256 a, uint256 b) → bool success, uint256 result` internal #

Returns the remainder of dividing two unsigned integers, with a success flag (no division by zero).

`saturatingAdd(uint256 a, uint256 b) → uint256` internal #

Unsigned saturating addition, bounds to `$2^{256} - 1$` instead of overflowing.

Unsigned saturating subtraction, bounds to zero instead of overflowing.

saturatingMul(uint256 a, uint256 b) → uint256

internal #

Unsigned saturating multiplication, bounds to $2^{256} - 1$ instead of overflowing.

ternary(bool condition, uint256 a, uint256 b) → uint256

internal #

Branchless ternary evaluation for `condition ? a : b`. Gas costs are constant.

 This function may reduce bytecode size and consume less gas when used standalone. However, the compiler may optimize Solidity ternary operations (i.e. `condition ? a : b`) to only compute one branch when needed, making this function more expensive.

max(uint256 a, uint256 b) → uint256

internal #

Returns the largest of two numbers.

min(uint256 a, uint256 b) → uint256

internal #

Returns the smallest of two numbers.

average(uint256 a, uint256 b) → uint256

internal #

Returns the average of two numbers. The result is rounded towards zero.

`ceilDiv(uint256 a, uint256 b) → uint256`

internal #



Returns the ceiling of the division of two numbers.

This differs from standard division with `/` in that it rounds towards infinity instead of rounding towards zero.

`mulDiv(uint256 x, uint256 y, uint256 denominator) → uint256 result`

internal #

Calculates $\text{floor}(x * y / \text{denominator})$ with full precision. Throws if result overflows a `uint256` or `denominator == 0`.

Original credit to Remco Bloemen under MIT license (<https://xn--2-umb.com/21/muldiv>) with further edits by Uniswap Labs also under MIT license.

`mulDiv(uint256 x, uint256 y, uint256 denominator, enum Math.Rounding rounding) → uint256`

internal #

Calculates $x * y / \text{denominator}$ with full precision, following the selected rounding direction.

`mulShr(uint256 x, uint256 y, uint8 n) → uint256 result`

internal #

Calculates $\text{floor}(x * y >> n)$ with full precision. Throws if result overflows a `uint256`.

`mulShr(uint256 x, uint256 y, uint8 n, enum Math.Rounding rounding) → uint256`

internal #

Calculates $x * y >> n$ with full precision, following the selected rounding direction.

Calculate the modular multiplicative inverse of a number in $\mathbb{Z}/n\mathbb{Z}$.

If n is a prime, then $\mathbb{Z}/n\mathbb{Z}$ is a field. In that case all elements are invertible, except 0. If n is not a prime, then $\mathbb{Z}/n\mathbb{Z}$ is not a field, and some elements might not be invertible.

If the input value is not invertible, 0 is returned.

i If you know for sure that n is (big) a prime, it may be cheaper to use Fermat's little theorem and get the inverse using `Math.modExp(a, n - 2, n)`. See [Math.invModPrime](#).

invModPrime(uint256 a, uint256 p) → uint256

internal #

Variant of [Math.invMod](#). More efficient, but only works if `p` is known to be a prime greater than 2.

From [Fermat's little theorem](#), we know that if p is prime, then $a^{p-1} \equiv 1 \pmod p$. As a consequence, we have $a * a^{p-2} \equiv 1 \pmod p$, which means that a^{p-2} is the modular multiplicative inverse of a in $\mathbb{F}_p$.

i this function does NOT check that `p` is a prime greater than 2.

modExp(uint256 b, uint256 e, uint256 m) → uint256

internal #

Returns the modular exponentiation of the specified base, exponent and modulus ($b ** e \% m$)

Requirements:

- modulus can't be zero
- underlying staticcall to precompile must succeed



The result is not valid if the underlying call succeeds. When using this function, make sure the chain you're using it on supports the precompiled contract for modular exponentiation at address 0x05 as specified in [EIP-198](#). Otherwise, the underlying function will succeed given the lack of a revert, but the result may be incorrectly interpreted as 0.

tryModExp(uint256 b, uint256 e, uint256 m) → bool success, uint256 result

internal #

Returns the modular exponentiation of the specified base, exponent and modulus ($b^{**}e \% m$). It includes a success flag indicating if the operation succeeded. Operation will be marked as failed if trying to operate modulo 0 or if the underlying precompile reverted.

 The result is only valid if the success flag is true. When using this function, make sure the chain you're using it on supports the precompiled contract for modular exponentiation at address 0x05 as specified in [EIP-198](#). Otherwise, the underlying function will succeed given the lack of a revert, but the result may be incorrectly interpreted as 0.

modExp(bytes b, bytes e, bytes m) → bytes

internal #

Variant of [Math.modExp](#) that supports inputs of arbitrary length.

tryModExp(bytes b, bytes e, bytes m) → bool success, bytes result

internal #

Variant of [Math.tryModExp](#) that supports inputs of arbitrary length.

sqrt(uint256 a) → uint256

internal #

Returns the square root of a number. If the number is not a perfect square, the value is rounded towards zero.



This method is based on Newton's method for computing square roots; the algorithm is restricted to only using integer operations.

`sqrt(uint256 a, enum Math.Rounding rounding) → uint256`

internal #

Calculates `sqrt(a)`, following the selected rounding direction.

`log2(uint256 x) → uint256 r`

internal #

Return the log in base 2 of a positive value rounded towards zero. Returns 0 if given 0.

`log2(uint256 value, enum Math.Rounding rounding) → uint256`

internal #

Return the log in base 2, following the selected rounding direction, of a positive value. Returns 0 if given 0.

`log10(uint256 value) → uint256`

internal #

Return the log in base 10 of a positive value rounded towards zero. Returns 0 if given 0.

`log10(uint256 value, enum Math.Rounding rounding) → uint256`

internal #

Return the log in base 10, following the selected rounding direction, of a positive value. Returns 0 if given 0.

`log256(uint256 x) → uint256 r`

internal #

Return the log in base 256 of a positive value rounded towards zero. Returns 0 if given 0.

**log256(uint256 value, enum Math.Rounding rounding) → uint256**

internal #

Return the log in base 256, following the selected rounding direction, of a positive value. Returns 0 if given 0.

unsignedRoundsUp(enum Math.Rounding rounding) → bool

internal #

Returns whether a provided rounding mode is considered rounding up for unsigned integers.

clz(uint256 x) → uint256

internal #

Counts the number of leading zero bits in a uint256.

SafeCast



```
import "@openzeppelin/contracts/utils/math/SafeCast.sol";
```



Wrappers over Solidity's uintXX/intXX/bool casting operators with added overflow checks.

Downcasting from uint256/int256 in Solidity does not revert on overflow. This can easily result in undesired exploitation or bugs, since developers usually assume that overflows raise errors. `SafeCast` restores this intuition by reverting the transaction when such an operation overflows.

Using this library instead of the unchecked operations eliminates an entire class of bugs, so it's recommended to use it always.

Functions

- [toUint248\(value\)](#)
- [toUint240\(value\)](#)
- [toUint232\(value\)](#)
- [toUint224\(value\)](#)
- [toUint216\(value\)](#)
- [toUint208\(value\)](#)
- [toUint200\(value\)](#)
- [toUint192\(value\)](#)
- [toUint184\(value\)](#)
- [toUint176\(value\)](#)
- [toUint168\(value\)](#)
- [toUint160\(value\)](#)
- [toUint152\(value\)](#)
- [toUint144\(value\)](#)
- [toUint136\(value\)](#)
- [toUint128\(value\)](#)
- [toUint120\(value\)](#)
- [toUint112\(value\)](#)
- [toUint104\(value\)](#)
- [toUint96\(value\)](#)
- [toUint88\(value\)](#)
- [toUint80\(value\)](#)
- [toUint72\(value\)](#)



- [toUint64\(value\)](#)

- [toUint56\(value\)](#)

- [toUint48\(value\)](#)

- [toUint40\(value\)](#)

- [toUint32\(value\)](#)

- [toUint24\(value\)](#)

- [toUint16\(value\)](#)

- [toUint8\(value\)](#)

- [toUint256\(value\)](#)

- [toInt248\(value\)](#)

- [toInt240\(value\)](#)

- [toInt232\(value\)](#)

- [toInt224\(value\)](#)

- [toInt216\(value\)](#)

- [toInt208\(value\)](#)

- [toInt200\(value\)](#)

- [toInt192\(value\)](#)

- [toInt184\(value\)](#)

- [toInt176\(value\)](#)

- [toInt168\(value\)](#)

- [toInt160\(value\)](#)

- [toInt152\(value\)](#)

- [toInt144\(value\)](#)

- [toInt136\(value\)](#)

- [toInt128\(value\)](#)

- [toInt120\(value\)](#)

- [toInt112\(value\)](#)



- [`toInt104\(value\)`](#)

- [`toInt96\(value\)`](#)

- [`toInt88\(value\)`](#)

- [`toInt80\(value\)`](#)

- [`toInt72\(value\)`](#)

- [`toInt64\(value\)`](#)

- [`toInt56\(value\)`](#)

- [`toInt48\(value\)`](#)

- [`toInt40\(value\)`](#)

- [`toInt32\(value\)`](#)

- [`toInt24\(value\)`](#)

- [`toInt16\(value\)`](#)

- [`toInt8\(value\)`](#)

- [`toInt256\(value\)`](#)

- [`toUint\(b\)`](#)

Errors

- [`SafeCastOverflowedUintDowncast\(bits, value\)`](#)

- [`SafeCastOverflowedIntToUint\(value\)`](#)

- [`SafeCastOverflowedIntDowncast\(bits, value\)`](#)

- [`SafeCastOverflowedUintToInt\(value\)`](#)

`toUint248(uint256 value) → uint248`

internal #



Returns the downcasted uint248 from uint256, reverting on overflow (when the input is greater than largest uint248).

Counterpart to Solidity's `uint248` operator.

Requirements:

- input must fit into 248 bits

`toUint240(uint256 value) → uint240`

internal #

Returns the downcasted uint240 from uint256, reverting on overflow (when the input is greater than largest uint240).

Counterpart to Solidity's `uint240` operator.

Requirements:

- input must fit into 240 bits

`toUint232(uint256 value) → uint232`

internal #

Returns the downcasted uint232 from uint256, reverting on overflow (when the input is greater than largest uint232).

Counterpart to Solidity's `uint232` operator.

Requirements:

- input must fit into 232 bits

`toUint224(uint256 value) → uint224`

internal #



Returns the downcasted uint224 from uint256, reverting on overflow (when the input is greater than largest uint224).

Counterpart to Solidity's `uint224` operator.

Requirements:

- input must fit into 224 bits

`toUint216(uint256 value) → uint216`

internal #

Returns the downcasted uint216 from uint256, reverting on overflow (when the input is greater than largest uint216).

Counterpart to Solidity's `uint216` operator.

Requirements:

- input must fit into 216 bits

`toUint208(uint256 value) → uint208`

internal #

Returns the downcasted uint208 from uint256, reverting on overflow (when the input is greater than largest uint208).

Counterpart to Solidity's `uint208` operator.

Requirements:

- input must fit into 208 bits

`toUint200(uint256 value) → uint200`

internal #



Returns the downcasted uint200 from uint256, reverting on overflow (when the input is greater than largest uint200).

Counterpart to Solidity's `uint200` operator.

Requirements:

- input must fit into 200 bits

`toUint192(uint256 value) → uint192`

internal #

Returns the downcasted uint192 from uint256, reverting on overflow (when the input is greater than largest uint192).

Counterpart to Solidity's `uint192` operator.

Requirements:

- input must fit into 192 bits

`toUint184(uint256 value) → uint184`

internal #

Returns the downcasted uint184 from uint256, reverting on overflow (when the input is greater than largest uint184).

Counterpart to Solidity's `uint184` operator.

Requirements:

- input must fit into 184 bits

`toUint176(uint256 value) → uint176`

internal #



Returns the downcasted uint176 from uint256, reverting on overflow (when the input is greater than largest uint176).

Counterpart to Solidity's `uint176` operator.

Requirements:

- input must fit into 176 bits

`toUint168(uint256 value) → uint168`

internal #

Returns the downcasted uint168 from uint256, reverting on overflow (when the input is greater than largest uint168).

Counterpart to Solidity's `uint168` operator.

Requirements:

- input must fit into 168 bits

`toUint160(uint256 value) → uint160`

internal #

Returns the downcasted uint160 from uint256, reverting on overflow (when the input is greater than largest uint160).

Counterpart to Solidity's `uint160` operator.

Requirements:

- input must fit into 160 bits

`toUint152(uint256 value) → uint152`

internal #



Returns the downcasted uint152 from uint256, reverting on overflow (when the input is greater than largest uint152).

Counterpart to Solidity's `uint152` operator.

Requirements:

- input must fit into 152 bits

`toUint144(uint256 value) → uint144`

internal #

Returns the downcasted uint144 from uint256, reverting on overflow (when the input is greater than largest uint144).

Counterpart to Solidity's `uint144` operator.

Requirements:

- input must fit into 144 bits

`toUint136(uint256 value) → uint136`

internal #

Returns the downcasted uint136 from uint256, reverting on overflow (when the input is greater than largest uint136).

Counterpart to Solidity's `uint136` operator.

Requirements:

- input must fit into 136 bits

`toUint128(uint256 value) → uint128`

internal #



Returns the downcasted uint128 from uint256, reverting on overflow (when the input is greater than largest uint128).

Counterpart to Solidity's `uint128` operator.

Requirements:

- input must fit into 128 bits

`toUint120(uint256 value) → uint120`

internal #

Returns the downcasted uint120 from uint256, reverting on overflow (when the input is greater than largest uint120).

Counterpart to Solidity's `uint120` operator.

Requirements:

- input must fit into 120 bits

`toUint112(uint256 value) → uint112`

internal #

Returns the downcasted uint112 from uint256, reverting on overflow (when the input is greater than largest uint112).

Counterpart to Solidity's `uint112` operator.

Requirements:

- input must fit into 112 bits

`toUint104(uint256 value) → uint104`

internal #



Returns the downcasted uint104 from uint256, reverting on overflow (when the input is greater than largest uint104).

Counterpart to Solidity's `uint104` operator.

Requirements:

- input must fit into 104 bits

`toUint96(uint256 value) → uint96`

internal #

Returns the downcasted uint96 from uint256, reverting on overflow (when the input is greater than largest uint96).

Counterpart to Solidity's `uint96` operator.

Requirements:

- input must fit into 96 bits

`toUint88(uint256 value) → uint88`

internal #

Returns the downcasted uint88 from uint256, reverting on overflow (when the input is greater than largest uint88).

Counterpart to Solidity's `uint88` operator.

Requirements:

- input must fit into 88 bits

`toUint80(uint256 value) → uint80`

internal #



Returns the downcasted uint80 from uint256, reverting on overflow (when the input is greater than largest uint80).

Counterpart to Solidity's `uint80` operator.

Requirements:

- input must fit into 80 bits

`toUint72(uint256 value) → uint72`

internal #

Returns the downcasted uint72 from uint256, reverting on overflow (when the input is greater than largest uint72).

Counterpart to Solidity's `uint72` operator.

Requirements:

- input must fit into 72 bits

`toUint64(uint256 value) → uint64`

internal #

Returns the downcasted uint64 from uint256, reverting on overflow (when the input is greater than largest uint64).

Counterpart to Solidity's `uint64` operator.

Requirements:

- input must fit into 64 bits

`toUint56(uint256 value) → uint56`

internal #



OpenZeppelin Docs

Returns the downcasted uint56 from uint256, reverting on overflow (when the input is greater than largest uint56).

Counterpart to Solidity's `uint56` operator.

Requirements:

- input must fit into 56 bits

`toUint48(uint256 value) → uint48`

internal #

Returns the downcasted uint48 from uint256, reverting on overflow (when the input is greater than largest uint48).

Counterpart to Solidity's `uint48` operator.

Requirements:

- input must fit into 48 bits

`toUint40(uint256 value) → uint40`

internal #

Returns the downcasted uint40 from uint256, reverting on overflow (when the input is greater than largest uint40).

Counterpart to Solidity's `uint40` operator.

Requirements:

- input must fit into 40 bits

`toUint32(uint256 value) → uint32`

internal #



Returns the downcasted uint32 from uint256, reverting on overflow (when the input is greater than largest uint32).

Counterpart to Solidity's `uint32` operator.

Requirements:

- input must fit into 32 bits

`toUint24(uint256 value) → uint24`

internal #

Returns the downcasted uint24 from uint256, reverting on overflow (when the input is greater than largest uint24).

Counterpart to Solidity's `uint24` operator.

Requirements:

- input must fit into 24 bits

`toUint16(uint256 value) → uint16`

internal #

Returns the downcasted uint16 from uint256, reverting on overflow (when the input is greater than largest uint16).

Counterpart to Solidity's `uint16` operator.

Requirements:

- input must fit into 16 bits

`toUint8(uint256 value) → uint8`

internal #



Returns the downcasted uint8 from uint256, reverting on overflow (when the input is greater than largest uint8).

Counterpart to Solidity's `uint8` operator.

Requirements:

- input must fit into 8 bits

toUint256(int256 value) → uint256

internal #

Converts a signed int256 into an unsigned uint256.

Requirements:

- input must be greater than or equal to 0.

toInt248(int256 value) → int248 downcasted

internal #

Returns the downcasted int248 from int256, reverting on overflow (when the input is less than smallest int248 or greater than largest int248).

Counterpart to Solidity's `int248` operator.

Requirements:

- input must fit into 248 bits

toInt240(int256 value) → int240 downcasted

internal #

Returns the downcasted int240 from int256, reverting on overflow (when the input is less than smallest int240 or greater than largest int240).

Counterpart to Solidity's `int240` operator.

- input must fit into 240 bits

toInt232(int256 value) → int232 downcasted

internal #

Returns the downcasted int232 from int256, reverting on overflow (when the input is less than smallest int232 or greater than largest int232).

Counterpart to Solidity's `int232` operator.

Requirements:

- input must fit into 232 bits

toInt224(int256 value) → int224 downcasted

internal #

Returns the downcasted int224 from int256, reverting on overflow (when the input is less than smallest int224 or greater than largest int224).

Counterpart to Solidity's `int224` operator.

Requirements:

- input must fit into 224 bits

toInt216(int256 value) → int216 downcasted

internal #

Returns the downcasted int216 from int256, reverting on overflow (when the input is less than smallest int216 or greater than largest int216).

Counterpart to Solidity's `int216` operator.

Requirements:



toInt208(int256 value) → int208 downcasted

internal #

Returns the downcasted int208 from int256, reverting on overflow (when the input is less than smallest int208 or greater than largest int208).

Counterpart to Solidity's `int208` operator.

Requirements:

- input must fit into 208 bits

toInt200(int256 value) → int200 downcasted

internal #

Returns the downcasted int200 from int256, reverting on overflow (when the input is less than smallest int200 or greater than largest int200).

Counterpart to Solidity's `int200` operator.

Requirements:

- input must fit into 200 bits

toInt192(int256 value) → int192 downcasted

internal #

Returns the downcasted int192 from int256, reverting on overflow (when the input is less than smallest int192 or greater than largest int192).

Counterpart to Solidity's `int192` operator.

Requirements:

**toInt184(int256 value) → int184 downcasted**

internal #

Returns the downcasted int184 from int256, reverting on overflow (when the input is less than smallest int184 or greater than largest int184).

Counterpart to Solidity's `int184` operator.

Requirements:

- input must fit into 184 bits

toInt176(int256 value) → int176 downcasted

internal #

Returns the downcasted int176 from int256, reverting on overflow (when the input is less than smallest int176 or greater than largest int176).

Counterpart to Solidity's `int176` operator.

Requirements:

- input must fit into 176 bits

toInt168(int256 value) → int168 downcasted

internal #

Returns the downcasted int168 from int256, reverting on overflow (when the input is less than smallest int168 or greater than largest int168).

Counterpart to Solidity's `int168` operator.

Requirements:



`toInt160(int256 value) → int160 downcasted`

internal #

Returns the downcasted int160 from int256, reverting on overflow (when the input is less than smallest int160 or greater than largest int160).

Counterpart to Solidity's `int160` operator.

Requirements:

- input must fit into 160 bits

`toInt152(int256 value) → int152 downcasted`

internal #

Returns the downcasted int152 from int256, reverting on overflow (when the input is less than smallest int152 or greater than largest int152).

Counterpart to Solidity's `int152` operator.

Requirements:

- input must fit into 152 bits

`toInt144(int256 value) → int144 downcasted`

internal #

Returns the downcasted int144 from int256, reverting on overflow (when the input is less than smallest int144 or greater than largest int144).

Counterpart to Solidity's `int144` operator.

Requirements:



toInt136(int256 value) → int136 downcasted

internal #

Returns the downcasted int136 from int256, reverting on overflow (when the input is less than smallest int136 or greater than largest int136).

Counterpart to Solidity's `int136` operator.

Requirements:

- input must fit into 136 bits

toInt128(int256 value) → int128 downcasted

internal #

Returns the downcasted int128 from int256, reverting on overflow (when the input is less than smallest int128 or greater than largest int128).

Counterpart to Solidity's `int128` operator.

Requirements:

- input must fit into 128 bits

toInt120(int256 value) → int120 downcasted

internal #

Returns the downcasted int120 from int256, reverting on overflow (when the input is less than smallest int120 or greater than largest int120).

Counterpart to Solidity's `int120` operator.

Requirements:



toInt112(int256 value) → int112 downcasted

internal #

Returns the downcasted int112 from int256, reverting on overflow (when the input is less than smallest int112 or greater than largest int112).

Counterpart to Solidity's `int112` operator.

Requirements:

- input must fit into 112 bits

toInt104(int256 value) → int104 downcasted

internal #

Returns the downcasted int104 from int256, reverting on overflow (when the input is less than smallest int104 or greater than largest int104).

Counterpart to Solidity's `int104` operator.

Requirements:

- input must fit into 104 bits

toInt96(int256 value) → int96 downcasted

internal #

Returns the downcasted int96 from int256, reverting on overflow (when the input is less than smallest int96 or greater than largest int96).

Counterpart to Solidity's `int96` operator.

Requirements:

toInt88(int256 value) → int88 downcasted

internal #

Returns the downcasted int88 from int256, reverting on overflow (when the input is less than smallest int88 or greater than largest int88).

Counterpart to Solidity's `int88` operator.

Requirements:

- input must fit into 88 bits

toInt80(int256 value) → int80 downcasted

internal #

Returns the downcasted int80 from int256, reverting on overflow (when the input is less than smallest int80 or greater than largest int80).

Counterpart to Solidity's `int80` operator.

Requirements:

- input must fit into 80 bits

toInt72(int256 value) → int72 downcasted

internal #

Returns the downcasted int72 from int256, reverting on overflow (when the input is less than smallest int72 or greater than largest int72).

Counterpart to Solidity's `int72` operator.

Requirements:

toInt64(int256 value) → int64 downcasted

internal #

Returns the downcasted int64 from int256, reverting on overflow (when the input is less than smallest int64 or greater than largest int64).

Counterpart to Solidity's `int64` operator.

Requirements:

- input must fit into 64 bits

toInt56(int256 value) → int56 downcasted

internal #

Returns the downcasted int56 from int256, reverting on overflow (when the input is less than smallest int56 or greater than largest int56).

Counterpart to Solidity's `int56` operator.

Requirements:

- input must fit into 56 bits

toInt48(int256 value) → int48 downcasted

internal #

Returns the downcasted int48 from int256, reverting on overflow (when the input is less than smallest int48 or greater than largest int48).

Counterpart to Solidity's `int48` operator.

Requirements:

toInt40(int256 value) → int40 downcasted

internal #

Returns the downcasted int40 from int256, reverting on overflow (when the input is less than smallest int40 or greater than largest int40).

Counterpart to Solidity's `int40` operator.

Requirements:

- input must fit into 40 bits

toInt32(int256 value) → int32 downcasted

internal #

Returns the downcasted int32 from int256, reverting on overflow (when the input is less than smallest int32 or greater than largest int32).

Counterpart to Solidity's `int32` operator.

Requirements:

- input must fit into 32 bits

toInt24(int256 value) → int24 downcasted

internal #

Returns the downcasted int24 from int256, reverting on overflow (when the input is less than smallest int24 or greater than largest int24).

Counterpart to Solidity's `int24` operator.

Requirements:

**toInt16(int256 value) → int16 downcasted**

internal #

Returns the downcasted int16 from int256, reverting on overflow (when the input is less than smallest int16 or greater than largest int16).

Counterpart to Solidity's `int16` operator.

Requirements:

- input must fit into 16 bits

toInt8(int256 value) → int8 downcasted

internal #

Returns the downcasted int8 from int256, reverting on overflow (when the input is less than smallest int8 or greater than largest int8).

Counterpart to Solidity's `int8` operator.

Requirements:

- input must fit into 8 bits

toInt256(uint256 value) → int256

internal #

Converts an unsigned uint256 into a signed int256.

Requirements:

- input must be less than or equal to `maxInt256`.

Cast a boolean (false or true) to a uint256 (0 or 1) with no jump.

SafeCastOverflowedUintDowncast(uint8 bits, uint256 value)

error #

Value doesn't fit in a uint of `bits` size.

SafeCastOverflowedIntToUint(int256 value)

error #

An int value doesn't fit in a uint of `bits` size.

SafeCastOverflowedIntDowncast(uint8 bits, int256 value)

error #

Value doesn't fit in an int of `bits` size.

SafeCastOverflowedUintToInt(uint256 value)

error #

A uint value doesn't fit in an int of `bits` size.

SignedMath



```
import "@openzeppelin/contracts/utils/math/SignedMath.sol";
```



Functions

- [ternary\(condition, a, b\)](#)
- [max\(a, b\)](#)
- [min\(a, b\)](#)
- [average\(a, b\)](#)
- [abs\(n\)](#)

ternary(bool condition, int256 a, int256 b) → int256

internal #

Branchless ternary evaluation for `a ? b : c`. Gas costs are constant.

 This function may reduce bytecode size and consume less gas when used standalone. However, the compiler may optimize Solidity ternary operations (i.e. `a ? b : c`) to only compute one branch when needed, making this function more expensive.

max(int256 a, int256 b) → int256

internal #

Returns the largest of two signed numbers.

min(int256 a, int256 b) → int256

internal #

Returns the smallest of two signed numbers.

average(int256 a, int256 b) → int256

internal #

 Returns the average of two signed numbers without overflow. The result is rounded towards zero.

abs(int256 n) → uint256

internal #

Returns the absolute unsigned value of a signed value.

Accumulators



```
import "@openzeppelin/contracts/utils/structs/Accumulators.sol";
```



Structure concatenating an arbitrary number of bytes buffers with limited memory allocation.

The Accumulators library provides a memory-efficient alternative to repeated concatenation of bytes. Instead of copying data on each concatenation ($O(n^2)$ complexity), it builds a linked list of references to existing data and performs a single memory allocation during flattening ($O(n)$ complexity).

Uses 0x00 as sentinel value for empty state (i.e. null pointers)

How it works

1. Create an empty accumulator with null head/tail pointers
2. Add data using `RLP.push` (append) or `Accumulators.shift` (prepend). It creates linked list nodes
3. Each node stores a reference to existing data (no copying)
4. Call `Accumulators.flatten` to materialize the final concatenated result in a single operation

Performance

- Addition: $O(1)$ per operation (just pointer manipulation)



- Flattening: $O(n)$ single pass with one memory allocation

- Memory: Minimal overhead until flattening (only stores references)

Functions

- [accumulator\(\)](#)
- [push\(self, data\)](#)
- [push\(self, data\)](#)
- [shift\(self, data\)](#)
- [shift\(self, data\)](#)
- [flatten\(self\)](#)

accumulator() → struct Accumulators.Accumulator self

internal #

Create a new (empty) accumulator

push(struct Accumulators.Accumulator self, bytes data) → struct Accumulators.Accumulator

internal #

Add a bytes buffer to (the end of) an Accumulator

push(struct Accumulators.Accumulator self, Memory.Slice data) → struct Accumulators.Accumulator

internal #

Add a memory slice to (the end of) an Accumulator

 **OpenZeppelin** Docs `struct Accumulators.Accumulator self, bytes data) → struct Accumulators.Accumulator` internal #

Add a bytes buffer to (the beginning of) an Accumulator

`shift(struct Accumulators.Accumulator self, Memory.Slice data) → struct Accumulators.Accumulator` internal #

Add a memory slice to (the beginning of) an Accumulator

`flatten(struct Accumulators.Accumulator self) → bytes result` internal #

Flatten all the bytes entries in an Accumulator into a single buffer

BitMaps



```
import "@openzeppelin/contracts/utils/structs/BitMaps.sol";
```



Library for managing uint256 to bool mapping in a compact and efficient way, provided the keys are sequential. Largely inspired by Uniswap's [merkle-distributor](#).

BitMaps pack 256 booleans across each bit of a single 256-bit slot of `uint256` type. Hence booleans corresponding to 256 *sequential* indices would only consume a single slot, unlike the regular `bool` which would consume an entire slot for a single value.

This results in gas savings in two ways:

- Setting a zero value to non-zero only once every 256 times

- Accessing the same warm slot for every 256 *sequential* indices



Functions

- [get\(bitmap, index\)](#)
- [setTo\(bitmap, index, value\)](#)
- [set\(bitmap, index\)](#)
- [unset\(bitmap, index\)](#)

get(struct BitMaps.BitMap bitmap, uint256 index) → bool

internal #

Returns whether the bit at `index` is set.

setTo(struct BitMaps.BitMap bitmap, uint256 index, bool value)

internal #

Sets the bit at `index` to the boolean `value`.

set(struct BitMaps.BitMap bitmap, uint256 index)

internal #

Sets the bit at `index`.

unset(struct BitMaps.BitMap bitmap, uint256 index)

internal #

Unsets the bit at `index`.



```
import "@openzeppelin/contracts/utils/structs/Checkpoints.sol";
```

This library defines the `Trace*` struct, for checkpointing values as they change at different points in time, and later looking up past values by block number. See [votes](#) as an example.

To create a history of checkpoints define a variable type `Checkpoints.Trace*` in your contract, and store a new checkpoint for the current transaction block using the [RLP.push](#) function.

Functions

- [push\(self, key, value\)](#)
- [lowerLookup\(self, key\)](#)
- [upperLookup\(self, key\)](#)
- [upperLookupRecent\(self, key\)](#)
- [latest\(self\)](#)
- [latestCheckpoint\(self\)](#)
- [length\(self\)](#)
- [at\(self, pos\)](#)
- [push\(self, key, value\)](#)
- [lowerLookup\(self, key\)](#)
- [upperLookup\(self, key\)](#)
- [upperLookupRecent\(self, key\)](#)
- [latest\(self\)](#)
- [latestCheckpoint\(self\)](#)
- [length\(self\)](#)
- [at\(self, pos\)](#)
- [push\(self, key, value\)](#)



- [lowerLookup\(self, key\)](#)
- [upperLookup\(self, key\)](#)
- [upperLookupRecent\(self, key\)](#)
- [latest\(self\)](#)
- [latestCheckpoint\(self\)](#)
- [length\(self\)](#)
- [at\(self, pos\)](#)
- [push\(self, key, value\)](#)
- [lowerLookup\(self, key\)](#)
- [upperLookup\(self, key\)](#)
- [upperLookupRecent\(self, key\)](#)
- [latest\(self\)](#)
- [latestCheckpoint\(self\)](#)
- [length\(self\)](#)
- [at\(self, pos\)](#)

Errors

- [CheckpointUnorderedInsertion\(\)](#)

push(struct Checkpoints.Trace256 self, uint256 key, uint256 value) → internal #
uint256 oldValue, uint256 newValue

Pushes a (`key`, `value`) pair into a Trace256 so that it is stored as the checkpoint.

Returns previous value and new value.

lowerLookup(struct Checkpoints.Trace256 self, uint256 key) → uint256 internal #

Returns the value in the first (oldest) checkpoint with key greater or equal than the search key, or zero if there is none.

upperLookup(struct Checkpoints.Trace256 self, uint256 key) → uint256 internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

upperLookupRecent(struct Checkpoints.Trace256 self, uint256 key) → uint256 internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

 This is a variant of [Checkpoints.upperLookup](#) that is optimized to find "recent" checkpoint (checkpoints with high keys).

latest(struct Checkpoints.Trace256 self) → uint256 internal #

Returns the value in the most recent checkpoint, or zero if there are no checkpoints.

latestCheckpoint(struct Checkpoints.Trace256 self) → bool exists, uint256 _key, uint256 _value internal #



Returns whether there is a checkpoint in the structure (i.e. it is not empty), and if so the key and value in the most recent checkpoint.

length(struct Checkpoints.Trace256 self) → uint256

internal #

Returns the number of checkpoints.

at(struct Checkpoints.Trace256 self, uint32 pos) → struct Checkpoints.Checkpoint256

internal #

Returns checkpoint at given position.

push(struct Checkpoints.Trace224 self, uint32 key, uint224 value) → uint224 oldValue, uint224 newValue

internal #

Pushes a (`key`, `value`) pair into a Trace224 so that it is stored as the checkpoint.

Returns previous value and new value.

⚠ Never accept `key` as a user input, since an arbitrary `type(uint32).max` key set will disable the library.

lowerLookup(struct Checkpoints.Trace224 self, uint32 key) → uint224

internal #

Returns the value in the first (oldest) checkpoint with key greater or equal than the search key, or zero if there is none.

upperLookup(struct Checkpoints.Trace224 self, uint32 key) → uint224

internal #



Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

upperLookupRecent(struct Checkpoints.Trace224 self, uint32 key) → uint224 internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

i This is a variant of [Checkpoints.upperLookup](#) that is optimized to find "recent" checkpoint (checkpoints with high keys).

latest(struct Checkpoints.Trace224 self) → uint224 internal #

Returns the value in the most recent checkpoint, or zero if there are no checkpoints.

latestCheckpoint(struct Checkpoints.Trace224 self) → bool exists, uint32 _key, uint224 _value internal #

Returns whether there is a checkpoint in the structure (i.e. it is not empty), and if so the key and value in the most recent checkpoint.

length(struct Checkpoints.Trace224 self) → uint256 internal #

Returns the number of checkpoints.

at(struct Checkpoints.Trace224 self, uint32 pos) → struct Checkpoints.Checkpoint224 internal #

Returns checkpoint at given position.

OpenZeppelin Docs `Trace208 self, uint48 key, uint208 value) → uint208 oldValue, uint208 newValue` internal #

Pushes a (`key` , `value`) pair into a `Trace208` so that it is stored as the checkpoint.

Returns previous value and new value.

⚠ Never accept `key` as a user input, since an arbitrary `type(uint48).max` key set will disable the library.

`lowerLookup(struct Checkpoints.Trace208 self, uint48 key) → uint208` internal #

Returns the value in the first (oldest) checkpoint with key greater or equal than the search key, or zero if there is none.

`upperLookup(struct Checkpoints.Trace208 self, uint48 key) → uint208` internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

`upperLookupRecent(struct Checkpoints.Trace208 self, uint48 key) → uint208` internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

ℹ This is a variant of [Checkpoints.upperLookup](#) that is optimized to find "recent" checkpoint (checkpoints with high keys).

`latest(struct Checkpoints.Trace208 self) → uint208` internal #

 Returns the value in the most recent checkpoint, or zero if there are no checkpoints.

latestCheckpoint(struct Checkpoints.Trace208 self) → bool exists, uint48 _key, uint208 _value internal #

Returns whether there is a checkpoint in the structure (i.e. it is not empty), and if so the key and value in the most recent checkpoint.

length(struct Checkpoints.Trace208 self) → uint256 internal #

Returns the number of checkpoints.

at(struct Checkpoints.Trace208 self, uint32 pos) → struct Checkpoints.Checkpoint208 internal #

Returns checkpoint at given position.

push(struct Checkpoints.Trace160 self, uint96 key, uint160 value) → uint160 oldValue, uint160 newValue internal #

Pushes a (`key`, `value`) pair into a Trace160 so that it is stored as the checkpoint.

Returns previous value and new value.

 Never accept `key` as a user input, since an arbitrary `type(uint96).max` key set will disable the library.

lowerLookup(struct Checkpoints.Trace160 self, uint96 key) → uint160 internal #



OpenZeppelin Docs

Returns the value in the first (oldest) checkpoint with key greater or equal than the search key, or zero if there is none.

upperLookup(struct Checkpoints.Trace160 self, uint96 key) → uint160 internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

upperLookupRecent(struct Checkpoints.Trace160 self, uint96 key) → uint160 internal #

Returns the value in the last (most recent) checkpoint with key lower or equal than the search key, or zero if there is none.

i This is a variant of [Checkpoints.upperLookup](#) that is optimized to find "recent" checkpoint (checkpoints with high keys).

latest(struct Checkpoints.Trace160 self) → uint160 internal #

Returns the value in the most recent checkpoint, or zero if there are no checkpoints.

latestCheckpoint(struct Checkpoints.Trace160 self) → bool exists, uint96 _key, uint160 _value internal #

Returns whether there is a checkpoint in the structure (i.e. it is not empty), and if so the key and value in the most recent checkpoint.

length(struct Checkpoints.Trace160 self) → uint256 internal #



`at(struct Checkpoints.Trace160 self, uint32 pos) → struct Checkpoints.Checkpoint160`

internal #

Returns checkpoint at given position.

`CheckpointUnorderedInsertion()`

error #

A value was attempted to be inserted on a past checkpoint.

CircularBuffer



```
import "@openzeppelin/contracts/utils/structs/CircularBuffer.sol";
```



A fixed-size buffer for keeping `bytes32` items in storage.

This data structure allows for pushing elements to it, and when its length exceeds the specified fixed size, new items take the place of the oldest element in the buffer, keeping at most `N` elements in the structure.

Elements can't be removed but the data structure can be cleared. See [CircularBuffer.clear](#).

Complexity:

- insertion ([RLP.push](#)): $O(1)$
- lookup ([CircularBuffer.last](#)): $O(1)$
- inclusion ([CircularBuffer.includes](#)): $O(N)$ (worst case)

• `reset (CircularBuffer.clear): O(1)`

The struct is called `Bytes32CircularBuffer`. Other types can be cast to and from `bytes32`.

This data structure can only be used in storage, and not in memory.

Example usage:

```
contract Example {
    // Add the library methods
    using CircularBuffer for CircularBuffer.Bytes32CircularBuffer;

    // Declare a buffer storage variable
    CircularBuffer.Bytes32CircularBuffer private myBuffer;

    constructor() {
        myBuffer.setup(16); // Initialize the buffer with a non-zero fixed size (e.g., 16)
    }

    function pushValue(bytes32 value) external {
        myBuffer.push(value); // Safe to push because the buffer was initialized in the constructor
    }
}
```

i Make sure to call `CircularBuffer.setup` on your buffer during construction/initialization

Available since v5.1.

Functions

- [`setup\(self, size\)`](#)
- [`clear\(self\)`](#)
- [`push\(self, value\)`](#)
- [`count\(self\)`](#)
- [`length\(self\)`](#)
- [`last\(self, i\)`](#)
- [`includes\(self, value\)`](#)

- [InvalidBufferSize\(\)](#)

setup(struct CircularBuffer.Bytes32CircularBuffer self, uint256 size) internal #

Initialize a new CircularBuffer of a given size.

If the CircularBuffer was already setup and used, calling that function again will reset it to a blank state.

 The size of the buffer will affect the execution of [CircularBuffer.includes](#) function, as it has a complexity of $O(N)$. Consider a large buffer size may render the function unusable.

clear(struct CircularBuffer.Bytes32CircularBuffer self) internal #

Clear all data in the buffer without resetting memory, keeping the existing size.

push(struct CircularBuffer.Bytes32CircularBuffer self, bytes32 value) internal #

Push a new value to the buffer. If the buffer is already full, the new value replaces the oldest value in the buffer.

count(struct CircularBuffer.Bytes32CircularBuffer self) → uint256 internal #

Number of values currently in the buffer. This value is 0 for an empty buffer, and cannot exceed the size of the buffer.



`Buffer.Bytes32CircularBuffer self) → uint256`

internal #

Length of the buffer. This is the maximum number of elements kept in the buffer.

`last(struct CircularBuffer.Bytes32CircularBuffer self, uint256 i) → bytes32`

internal #

Getter for the i-th value in the buffer, from the end.

Reverts with `Panic.ARRAY_OUT_OF_BOUNDS` if trying to access an element that was not pushed, or that was dropped to make room for newer elements.

`includes(struct CircularBuffer.Bytes32CircularBuffer self, bytes32 value) → bool`

internal #

Check if a given value is in the buffer.

`InvalidBufferSize()`

error #

Error emitted when trying to setup a buffer with a size of 0.

DoubleEndedQueue



```
import "@openzeppelin/contracts/utils/structs/DoubleEndedQueue.sol";
```



A sequence of items with the ability to efficiently push and pop items (i.e. insert and remove) on both ends of the sequence (called front and back). Among other access patterns, it can be

used to implement efficient LIFO and FIFO queues. Storage use is optimized, and all operations are $O(1)$ constant time. This includes `CircularBuffer.clear`, given that the existing queue contents are left in storage.

The struct is called `Bytes32Deque`. Other types can be cast to and from `bytes32`. This data structure can only be used in storage, and not in memory.

```
DoubleEndedQueue.Bytes32Deque queue;
```



Functions

- [`pushBack\(deque, value\)`](#)
- [`tryPushBack\(deque, value\)`](#)
- [`popBack\(deque\)`](#)
- [`tryPopBack\(deque\)`](#)
- [`pushFront\(deque, value\)`](#)
- [`tryPushFront\(deque, value\)`](#)
- [`popFront\(deque\)`](#)
- [`tryPopFront\(deque\)`](#)
- [`front\(deque\)`](#)
- [`tryFront\(deque\)`](#)
- [`back\(deque\)`](#)
- [`tryBack\(deque\)`](#)
- [`at\(deque, index\)`](#)
- [`tryAt\(deque, index\)`](#)
- [`clear\(deque\)`](#)
- [`length\(deque\)`](#)
- [`empty\(deque\)`](#)

OpenZeppelin Docs `DoubleEndedQueue.Bytes32Deque deque, bytes32 value)` internal

Inserts an item at the end of the queue.

Reverts with `Panic.RESOURCE_ERROR` if the queue is full.

`tryPushBack(struct DoubleEndedQueue.Bytes32Deque deque, bytes32 value) → bool success` internal #

Attempts to insert an item at the end of the queue.

Returns `false` if the queue is full. Never reverts.

`popBack(struct DoubleEndedQueue.Bytes32Deque deque) → bytes32` internal #

Removes the item at the end of the queue and returns it.

Reverts with `Panic.EMPTY_ARRAY_POP` if the queue is empty.

`tryPopBack(struct DoubleEndedQueue.Bytes32Deque deque) → bool success, bytes32 value` internal #

Attempts to remove the item at the end of the queue and return it.

Returns `(false, 0x00)` if the queue is empty. Never reverts.

`pushFront(struct DoubleEndedQueue.Bytes32Deque deque, bytes32 value)` internal #

Inserts an item at the beginning of the queue.

 Reverts with `Panic.RESOURCE_ERROR` if the queue is full.

OpenZeppelin Docs

`tryPushFront(struct DoubleEndedQueue.Bytes32Deque deque, bytes32 value)` internal #
→ **`bool success`**

Attempts to insert an item at the beginning of the queue.

Returns `false` if the queue is full. Never reverts.

`popFront(struct DoubleEndedQueue.Bytes32Deque deque)` → **`bytes32`** internal #

Removes the item at the beginning of the queue and returns it.

Reverts with `Panic.EMPTY_ARRAY_POP` if the queue is empty.

`tryPopFront(struct DoubleEndedQueue.Bytes32Deque deque)` → **`bool success,`** internal #
`bytes32 value`

Attempts to remove the item at the beginning of the queue and return it.

Returns `(false, 0x00)` if the queue is empty. Never reverts.

`front(struct DoubleEndedQueue.Bytes32Deque deque)` → **`bytes32`** internal #

Returns the item at the beginning of the queue.

Reverts with `Panic.ARRAY_OUT_OF_BOUNDS` if the queue is empty.

`tryFront(struct DoubleEndedQueue.Bytes32Deque deque)` → **`bool success,`** internal #
`bytes32 value`



Attempts to return the item at the beginning of the queue.

Returns `(false, 0x00)` if the queue is empty. Never reverts.

back(struct DoubleEndedQueue.Bytes32Deque deque) → bytes32

internal #

Returns the item at the end of the queue.

Reverts with [Panic.ARRAY_OUT_OF_BOUNDS](#) if the queue is empty.

tryBack(struct DoubleEndedQueue.Bytes32Deque deque) → bool success, bytes32 value

internal #

Attempts to return the item at the end of the queue.

Returns `(false, 0x00)` if the queue is empty. Never reverts.

at(struct DoubleEndedQueue.Bytes32Deque deque, uint256 index) → bytes32 internal #

Return the item at a position in the queue given by `index`, with the first item at 0 and last item at `length(deque) - 1`.

Reverts with [Panic.ARRAY_OUT_OF_BOUNDS](#) if the index is out of bounds.

tryAt(struct DoubleEndedQueue.Bytes32Deque deque, uint256 index) → bool success, bytes32 value

internal #

Attempts to return the item at a position in the queue given by `index`, with the first item at 0 and the last item at `length(deque) - 1`.

Returns `(false, 0x00)` if the index is out of bounds. Never reverts.

Resets the queue back to being empty.

i The current items are left behind in storage. This does not affect the functioning of the queue, but misses out on potential gas refunds.

length(struct DoubleEndedQueue.Bytes32Deque deque) → uint256

internal #

Returns the number of items in the queue.

empty(struct DoubleEndedQueue.Bytes32Deque deque) → bool

internal #

Returns true if the queue is empty.

EnumerableMap



```
import "@openzeppelin/contracts/utils/structs/EnumerableMap.sol";
```



Library for managing an enumerable variant of Solidity's [mapping](#) type.

Maps have the following properties:

- Entries are added, removed, and checked for existence in constant time ($O(1)$).
- Entries are enumerated in $O(n)$. No guarantees are made on the ordering.
- Map can be cleared (all entries removed) in $O(n)$.

OpenZeppelin Docs



```
// Add the library methods
using EnumerableMap for EnumerableMap.UintToAddressMap;

// Declare a set state variable
EnumerableMap.UintToAddressMap private myMap;
}
```

The following map types are supported:

- `uint256 -> address` (`UintToAddressMap`) since v3.0.0
- `address -> uint256` (`AddressToUintMap`) since v4.6.0
- `bytes32 -> bytes32` (`Bytes32ToBytes32Map`) since v4.6.0
- `uint256 -> uint256` (`UintToUintMap`) since v4.7.0
- `bytes32 -> uint256` (`Bytes32ToUintMap`) since v4.7.0
- `uint256 -> bytes32` (`UintToBytes32Map`) since v5.1.0
- `address -> address` (`AddressToAddressMap`) since v5.1.0
- `address -> bytes32` (`AddressToBytes32Map`) since v5.1.0
- `bytes32 -> address` (`Bytes32ToAddressMap`) since v5.1.0
- `bytes -> bytes` (`BytesToBytesMap`) since v5.4.0
- `bytes4 -> address` (`Bytes4ToAddressMap`) since v5.6.0

⚠ Trying to delete such a structure from storage will likely result in data corruption, rendering the structure unusable. See ethereum/solidity#11843 for more info.

In order to clean an `EnumerableMap`, you can either remove all elements one by one or create a fresh instance using an array of `EnumerableMap`.

Functions

- [`set\(map, key, value\)`](#)
- [`remove\(map, key\)`](#)
- [`clear\(map\)`](#)



- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)



- [`set\(map, key, value\)`](#)
- [`remove\(map, key\)`](#)
- [`clear\(map\)`](#)
- [`contains\(map, key\)`](#)
- [`length\(map\)`](#)
- [`at\(map, index\)`](#)
- [`tryGet\(map, key\)`](#)
- [`get\(map, key\)`](#)
- [`keys\(map\)`](#)
- [`keys\(map, start, end\)`](#)
- [`set\(map, key, value\)`](#)
- [`remove\(map, key\)`](#)
- [`clear\(map\)`](#)
- [`contains\(map, key\)`](#)
- [`length\(map\)`](#)
- [`at\(map, index\)`](#)
- [`tryGet\(map, key\)`](#)
- [`get\(map, key\)`](#)
- [`keys\(map\)`](#)
- [`keys\(map, start, end\)`](#)
- [`set\(map, key, value\)`](#)
- [`remove\(map, key\)`](#)
- [`clear\(map\)`](#)
- [`contains\(map, key\)`](#)
- [`length\(map\)`](#)
- [`at\(map, index\)`](#)
- [`tryGet\(map, key\)`](#)



- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)



- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)
- [set\(map, key, value\)](#)
- [remove\(map, key\)](#)
- [clear\(map\)](#)
- [contains\(map, key\)](#)
- [length\(map\)](#)
- [at\(map, index\)](#)
- [tryGet\(map, key\)](#)
- [get\(map, key\)](#)
- [keys\(map\)](#)
- [keys\(map, start, end\)](#)

- [EnumerableMapNonexistentKey\(key\)](#)
- [EnumerableMapNonexistentBytesKey\(key\)](#)

set(struct EnumerableMap.Bytes32ToBytes32Map map, bytes32 key, bytes32 value) → bool internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

remove(struct EnumerableMap.Bytes32ToBytes32Map map, bytes32 key) → bool internal #

Removes a key-value pair from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

clear(struct EnumerableMap.Bytes32ToBytes32Map map) internal #

Removes all the entries from a map. $O(n)$.

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.Bytes32ToBytes32Map map, bytes32 key) → bool internal #

Returns true if the key is in the map. $O(1)$.

OpenZeppelin Docs **EnumerableMap.Bytes32ToBytes32Map map) → uint256** internal

Returns the number of key-value pairs in the map. O(1).

at(struct EnumerableMap.Bytes32ToBytes32Map map, uint256 index) → bytes32 key, bytes32 value internal #

Returns the key-value pair stored at position `index` in the map. O(1).

Note that there are no guarantees on the ordering of entries inside the array, and it may change when more entries are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.Bytes32ToBytes32Map map, bytes32 key) → bool exists, bytes32 value internal #

Tries to return the value associated with `key`. O(1). Does not revert if `key` is not in the map.

get(struct EnumerableMap.Bytes32ToBytes32Map map, bytes32 key) → bytes32 internal #

Returns the value associated with `key`. O(1).

Requirements:

- `key` must be in the map.

keys(struct EnumerableMap.Bytes32ToBytes32Map map) → bytes32[] internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees.

Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
keys(struct EnumerableMap.Bytes32ToBytes32Map map, uint256 start,           internal #  
uint256 end) → bytes32[]
```

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
set(struct EnumerableMap.UintToUintMap map, uint256 key, uint256 value) internal #  
→ bool
```

Adds a key-value pair to a map, or updates the value for an existing key. O(1).

Returns true if the key was added to the map, that is if it was not already present.

```
remove(struct EnumerableMap.UintToUintMap map, uint256 key) → bool           internal #
```

Removes a value from a map. O(1).

Returns true if the key was removed from the map, that is if it was present.

```
clear(struct EnumerableMap.UintToUintMap map)                               internal #
```

 Removes all the entries from a map. $O(n)$.

 This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncalledable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.UintToUintMap map, uint256 key) → bool internal #

Returns true if the key is in the map. $O(1)$.

length(struct EnumerableMap.UintToUintMap map) → uint256 internal #

Returns the number of elements in the map. $O(1)$.

at(struct EnumerableMap.UintToUintMap map, uint256 index) → uint256 key, uint256 value internal #

Returns the element stored at position `index` in the map. $O(1)$. Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.UintToUintMap map, uint256 key) → bool exists, uint256 value internal #

Tries to return the value associated with `key`. $O(1)$. Does not revert if `key` is not in the map.

`get(struct EnumerableMap.UintToUintMap map, uint256 key) → uint256` internal #

 **OpenZeppelin Docs**

Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

`keys(struct EnumerableMap.UintToUintMap map) → uint256[]` internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

`keys(struct EnumerableMap.UintToUintMap map, uint256 start, uint256 end) → uint256[]` internal #

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

`set(struct EnumerableMap.UintToAddressMap map, uint256 key, address value) → bool` internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

 Returns true if the key was added to the map, that is if it was not already present.

`remove(struct EnumerableMap.UintToAddressMap map, uint256 key) → bool` internal #

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

`clear(struct EnumerableMap.UintToAddressMap map)` internal #

Removes all the entries from a map. $O(n)$.

 This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

`contains(struct EnumerableMap.UintToAddressMap map, uint256 key) → bool` internal #

Returns true if the key is in the map. $O(1)$.

`length(struct EnumerableMap.UintToAddressMap map) → uint256` internal #

Returns the number of elements in the map. $O(1)$.

`at(struct EnumerableMap.UintToAddressMap map, uint256 index) → uint256` internal #
`key, address value`

Returns the element stored at position `index` in the map. $O(1)$. Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements: OpenZeppelin Docs

- `index` must be strictly less than `Memory.length`.

`tryGet(struct EnumerableMap.UintToAddressMap map, uint256 key) → bool` internal #
exists, address value

Tries to return the value associated with `key`. $O(1)$. Does not revert if `key` is not in the map.

`get(struct EnumerableMap.UintToAddressMap map, uint256 key) → address` internal #

Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

`keys(struct EnumerableMap.UintToAddressMap map) → uint256[]` internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

`keys(struct EnumerableMap.UintToAddressMap map, uint256 start, uint256 end) → uint256[]` internal #

Returns an array containing a slice of the keys



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

set(struct EnumerableMap.UintToBytes32Map map, uint256 key, bytes32 value) → bool internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

remove(struct EnumerableMap.UintToBytes32Map map, uint256 key) → bool internal #

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

clear(struct EnumerableMap.UintToBytes32Map map) internal #

Removes all the entries from a map. $O(n)$.

 This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.UintToBytes32Map map, uint256 key) → bool internal #

Returns true if the key is in the map. $O(1)$.

Returns the number of elements in the map. O(1).

at(struct EnumerableMap.UintToBytes32Map map, uint256 index) → uint256
key, bytes32 value

internal #

Returns the element stored at position `index` in the map. O(1). Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.UintToBytes32Map map, uint256 key) → bool
exists, bytes32 value

internal #

Tries to return the value associated with `key`. O(1). Does not revert if `key` is not in the map.

get(struct EnumerableMap.UintToBytes32Map map, uint256 key) → bytes32

internal #

Returns the value associated with `key`. O(1).

Requirements:

- `key` must be in the map.

keys(struct EnumerableMap.UintToBytes32Map map) → uint256[]

internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees.

Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
keys(struct EnumerableMap.UintToBytes32Map map, uint256 start, uint256 end) → uint256[] internal #
```

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
set(struct EnumerableMap.AddressToUintMap map, address key, uint256 value) → bool internal #
```

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

```
remove(struct EnumerableMap.AddressToUintMap map, address key) → bool internal #
```

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

```
clear(struct EnumerableMap.AddressToUintMap map) internal #
```

Removes all the entries from a map. $O(n)$.

 This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.AddressToUintMap map, address key) → bool internal #

Returns true if the key is in the map. $O(1)$.

length(struct EnumerableMap.AddressToUintMap map) → uint256 internal #

Returns the number of elements in the map. $O(1)$.

at(struct EnumerableMap.AddressToUintMap map, uint256 index) → address internal #
key, uint256 value

Returns the element stored at position `index` in the map. $O(1)$. Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.AddressToUintMap map, address key) → bool internal #
exists, uint256 value

Tries to return the value associated with `key`. $O(1)$. Does not revert if `key` is not in the map.

`get(struct EnumerableMap.AddressToUintMap map, address key) → uint256` internal #

 **OpenZeppelin Docs**

Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

`keys(struct EnumerableMap.AddressToUintMap map) → address[]` internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

`keys(struct EnumerableMap.AddressToUintMap map, uint256 start, uint256 end) → address[]` internal #

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

`set(struct EnumerableMap.AddressToAddressMap map, address key, address value) → bool` internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

 Returns true if the key was added to the map, that is if it was not already present.

`remove(struct EnumerableMap.AddressToAddressMap map, address key) → bool` internal #

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

`clear(struct EnumerableMap.AddressToAddressMap map)` internal #

Removes all the entries from a map. $O(n)$.

 This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

`contains(struct EnumerableMap.AddressToAddressMap map, address key) → bool` internal #

Returns true if the key is in the map. $O(1)$.

`length(struct EnumerableMap.AddressToAddressMap map) → uint256` internal #

Returns the number of elements in the map. $O(1)$.

`at(struct EnumerableMap.AddressToAddressMap map, uint256 index) → address key, address value` internal #

Returns the element stored at position `index` in the map. $O(1)$. Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements: OpenZeppelin Docs

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.AddressToAddressMap map, address key) → bool internal #
exists, address value

Tries to return the value associated with `key`. $O(1)$. Does not revert if `key` is not in the map.

get(struct EnumerableMap.AddressToAddressMap map, address key) → address internal #

Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

keys(struct EnumerableMap.AddressToAddressMap map) → address[] internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

keys(struct EnumerableMap.AddressToAddressMap map, uint256 start, uint256 end) → address[] internal #

Returns an array containing a slice of the keys



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
set(struct EnumerableMap.AddressToBytes32Map map, address key, bytes32 value) → bool internal #
```

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

```
remove(struct EnumerableMap.AddressToBytes32Map map, address key) → bool internal #
```

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

```
clear(struct EnumerableMap.AddressToBytes32Map map) internal #
```

Removes all the entries from a map. $O(n)$.

⚠ This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

```
contains(struct EnumerableMap.AddressToBytes32Map map, address key) → bool internal #
```

Returns true if the key is in the map. $O(1)$.

`length(struct EnumerableMap.AddressToBytes32Map map) → uint256` internal #



Returns the number of elements in the map. $O(1)$.

`at(struct EnumerableMap.AddressToBytes32Map map, uint256 index) → address key, bytes32 value` internal #

Returns the element stored at position `index` in the map. $O(1)$. Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

`tryGet(struct EnumerableMap.AddressToBytes32Map map, address key) → bool exists, bytes32 value` internal #

Tries to return the value associated with `key`. $O(1)$. Does not revert if `key` is not in the map.

`get(struct EnumerableMap.AddressToBytes32Map map, address key) → bytes32` internal #

Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

`keys(struct EnumerableMap.AddressToBytes32Map map) → address[]` internal #

Returns an array containing all the keys



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
keys(struct EnumerableMap.AddressToBytes32Map map, uint256 start,  
uint256 end) → address[]
```

internal #

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
set(struct EnumerableMap.Bytes32ToUintMap map, bytes32 key, uint256  
value) → bool
```

internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

```
remove(struct EnumerableMap.Bytes32ToUintMap map, bytes32 key) → bool
```

internal #

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

```
clear(struct EnumerableMap.Bytes32ToUintMap map)
```

internal #

Removes all the entries from a map. $O(n)$.



This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.Bytes32ToUintMap map, bytes32 key) → bool internal #

Returns true if the key is in the map. O(1).

length(struct EnumerableMap.Bytes32ToUintMap map) → uint256 internal #

Returns the number of elements in the map. O(1).

at(struct EnumerableMap.Bytes32ToUintMap map, uint256 index) → bytes32 internal #
key, uint256 value

Returns the element stored at position `index` in the map. O(1). Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.Bytes32ToUintMap map, bytes32 key) → bool internal #
exists, uint256 value

Tries to return the value associated with `key`. O(1). Does not revert if `key` is not in the map.

get(struct EnumerableMap.Bytes32ToUintMap map, bytes32 key) → uint256 internal #

 Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

keys(struct EnumerableMap.Bytes32ToUintMap map) → bytes32[]

internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

keys(struct EnumerableMap.Bytes32ToUintMap map, uint256 start, uint256 end) → bytes32[]

internal #

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

set(struct EnumerableMap.Bytes32ToAddressMap map, bytes32 key, address value) → bool

internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

`remove(struct EnumerableMap.Bytes32ToAddressMap map, bytes32 key) → bool` internal #



Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

`clear(struct EnumerableMap.Bytes32ToAddressMap map)` internal #

Removes all the entries from a map. $O(n)$.



This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

`contains(struct EnumerableMap.Bytes32ToAddressMap map, bytes32 key) → bool` internal #

Returns true if the key is in the map. $O(1)$.

`length(struct EnumerableMap.Bytes32ToAddressMap map) → uint256` internal #

Returns the number of elements in the map. $O(1)$.

`at(struct EnumerableMap.Bytes32ToAddressMap map, uint256 index) → bytes32 key, address value` internal #

Returns the element stored at position `index` in the map. $O(1)$. Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

```
tryGet(struct EnumerableMap.Bytes32ToAddressMap map, bytes32 key) → bool internal #
```

exists, address value

Tries to return the value associated with `key`. O(1). Does not revert if `key` is not in the map.

```
get(struct EnumerableMap.Bytes32ToAddressMap map, bytes32 key) → address internal #
```

Returns the value associated with `key`. O(1).

Requirements:

- `key` must be in the map.

```
keys(struct EnumerableMap.Bytes32ToAddressMap map) → bytes32[] internal #
```

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

```
keys(struct EnumerableMap.Bytes32ToAddressMap map, uint256 start, uint256 end) → bytes32[] internal #
```

Returns an array containing a slice of the keys



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

set(struct EnumerableMap.Bytes4ToAddressMap map, bytes4 key, address value) → bool internal #

Adds a key-value pair to a map, or updates the value for an existing key. $O(1)$.

Returns true if the key was added to the map, that is if it was not already present.

remove(struct EnumerableMap.Bytes4ToAddressMap map, bytes4 key) → bool internal #

Removes a value from a map. $O(1)$.

Returns true if the key was removed from the map, that is if it was present.

clear(struct EnumerableMap.Bytes4ToAddressMap map) internal #

Removes all the entries from a map. $O(n)$.

 This function has an unbounded cost that scales with map size. Developers should keep in mind that using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.Bytes4ToAddressMap map, bytes4 key) → bool internal #

Returns true if the key is in the map. $O(1)$.

Returns the number of elements in the map. O(1).

at(struct EnumerableMap.Bytes4ToAddressMap map, uint256 index) → bytes4 internal #
key, address value

Returns the element stored at position `index` in the map. O(1). Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.Bytes4ToAddressMap map, bytes4 key) → bool internal #
exists, address value

Tries to return the value associated with `key`. O(1). Does not revert if `key` is not in the map.

get(struct EnumerableMap.Bytes4ToAddressMap map, bytes4 key) → address internal #

Returns the value associated with `key`. O(1).

Requirements:

- `key` must be in the map.

keys(struct EnumerableMap.Bytes4ToAddressMap map) → bytes4[] internal #

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees.

Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

keys(struct EnumerableMap.Bytes4ToAddressMap map, uint256 start, uint256 end) → bytes4[] internal #

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

set(struct EnumerableMap.BytesToBytesMap map, bytes key, bytes value) → bool internal #

Adds a key-value pair to a map, or updates the value for an existing key. O(1).

Returns true if the key was added to the map, that is if it was not already present.

remove(struct EnumerableMap.BytesToBytesMap map, bytes key) → bool internal #

Removes a key-value pair from a map. O(1).

Returns true if the key was removed from the map, that is if it was present.

clear(struct EnumerableMap.BytesToBytesMap map) internal #

Removes all the entries from a map. $O(n)$.

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncallable if the map grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableMap.BytesToBytesMap map, bytes key) → bool internal #

Returns true if the key is in the map. $O(1)$.

length(struct EnumerableMap.BytesToBytesMap map) → uint256 internal #

Returns the number of key-value pairs in the map. $O(1)$.

at(struct EnumerableMap.BytesToBytesMap map, uint256 index) → bytes key, bytes value internal #

Returns the key-value pair stored at position `index` in the map. $O(1)$.

Note that there are no guarantees on the ordering of entries inside the array, and it may change when more entries are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

tryGet(struct EnumerableMap.BytesToBytesMap map, bytes key) → bool exists, bytes value internal #

Tries to return the value associated with `key`. $O(1)$. Does not revert if `key` is not in the map.

EnumerableMap.BytesToBytesMap map, bytes key) → bytes value internal

Returns the value associated with `key`. $O(1)$.

Requirements:

- `key` must be in the map.

keys(struct EnumerableMap.BytesToBytesMap map) → bytes[] internal

Returns an array containing all the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

keys(struct EnumerableMap.BytesToBytesMap map, uint256 start, uint256 end) → bytes[] internal

Returns an array containing a slice of the keys

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the map grows to a point where copying to memory consumes too much gas to fit in a block.

EnumerableMapNonexistentKey(bytes32 key) error



Query for a nonexistent map key.

`EnumerableMapNonexistentBytesKey(bytes_key)`

error #

Query for a nonexistent map key.

EnumerableSet



```
import "@openzeppelin/contracts/utils/structs/EnumerableSet.sol";
```



Library for managing sets of primitive types.

Sets have the following properties:

- Elements are added, removed, and checked for existence in constant time ($O(1)$).
- Elements are enumerated in $O(n)$. No guarantees are made on the ordering.
- Set can be cleared (all elements removed) in $O(n)$.

```
contract Example {
    // Add the library methods
    using EnumerableSet for EnumerableSet.AddressSet;

    // Declare a set state variable
    EnumerableSet.AddressSet private mySet;
}
```



The following types are supported:

- `bytes32` (`Bytes32Set`) since v3.3.0
- `address` (`AddressSet`) since v3.3.0

- `uint256` (`UIntSet`) since v3.3.0
- `string` (`StringSet`) since v5.4.0
- `bytes` (`BytesSet`) since v5.4.0
- `bytes4` (`Bytes4Set`) since v5.6.0

⚠ Trying to delete such a structure from storage will likely result in data corruption, rendering the structure unusable. See [ethereum/solidity#11843](https://github.com/ethereum/solidity/issues/11843) for more info.

In order to clean an `EnumerableSet`, you can either remove all elements one by one or create a fresh instance using an array of `EnumerableSet`.

Functions

- [`add\(set, value\)`](#)
- [`remove\(set, value\)`](#)
- [`clear\(set\)`](#)
- [`contains\(set, value\)`](#)
- [`length\(set\)`](#)
- [`at\(set, index\)`](#)
- [`values\(set\)`](#)
- [`values\(set, start, end\)`](#)
- [`add\(set, value\)`](#)
- [`remove\(set, value\)`](#)
- [`clear\(set\)`](#)
- [`contains\(set, value\)`](#)
- [`length\(set\)`](#)
- [`at\(set, index\)`](#)
- [`values\(set\)`](#)
- [`values\(set, start, end\)`](#)
- [`add\(set, value\)`](#)



- [remove\(set, value\)](#)
- [clear\(set\)](#)
- [contains\(set, value\)](#)
- [length\(set\)](#)
- [at\(set, index\)](#)
- [values\(set\)](#)
- [values\(set, start, end\)](#)
- [add\(set, value\)](#)
- [remove\(set, value\)](#)
- [clear\(set\)](#)
- [contains\(set, value\)](#)
- [length\(set\)](#)
- [at\(set, index\)](#)
- [values\(set\)](#)
- [values\(set, start, end\)](#)
- [add\(set, value\)](#)
- [remove\(set, value\)](#)
- [clear\(set\)](#)
- [contains\(set, value\)](#)
- [length\(set\)](#)
- [at\(set, index\)](#)
- [values\(set\)](#)
- [values\(set, start, end\)](#)
- [add\(set, value\)](#)
- [remove\(set, value\)](#)
- [clear\(set\)](#)
- [contains\(set, value\)](#)



- [length\(set\)](#)
- [at\(set, index\)](#)
- [values\(set\)](#)
- [values\(set, start, end\)](#)

add(struct EnumerableSet.Bytes32Set set, bytes32 value) → bool internal #

Add a value to a set. $O(1)$.

Returns true if the value was added to the set, that is if it was not already present.

remove(struct EnumerableSet.Bytes32Set set, bytes32 value) → bool internal #

Removes a value from a set. $O(1)$.

Returns true if the value was removed from the set, that is if it was present.

clear(struct EnumerableSet.Bytes32Set set) internal #

Removes all the values from a set. $O(n)$.

⚠ Developers should keep in mind that this function has an unbounded cost and using it may render the function uncalleable if the set grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableSet.Bytes32Set set, bytes32 value) → bool internal #

Returns true if the value is in the set. $O(1)$.

`length(struct EnumerableSet.Bytes32Set set) → uint256` internal #



Returns the number of values in the set. $O(1)$.

`at(struct EnumerableSet.Bytes32Set set, uint256 index) → bytes32` internal #

Returns the value stored at position `index` in the set. $O(1)$.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

`values(struct EnumerableSet.Bytes32Set set) → bytes32[]` internal #

Return the entire set in an array



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

`values(struct EnumerableSet.Bytes32Set set, uint256 start, uint256 end) → bytes32[]` internal #

Return a slice of the set in an array



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

add(struct EnumerableSet.Bytes4Set set, bytes4 value) → bool internal #

Add a value to a set. O(1).

Returns true if the value was added to the set, that is if it was not already present.

remove(struct EnumerableSet.Bytes4Set set, bytes4 value) → bool internal #

Removes a value from a set. O(1).

Returns true if the value was removed from the set, that is if it was present.

clear(struct EnumerableSet.Bytes4Set set) internal #

Removes all the values from a set. O(n).

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncallable if the set grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableSet.Bytes4Set set, bytes4 value) → bool internal #

Returns true if the value is in the set. O(1).

length(struct EnumerableSet.Bytes4Set set) → uint256 internal #



Returns the number of values in the set. $O(1)$.

`at(struct EnumerableSet.Bytes4Set set, uint256 index) → bytes4` internal #

Returns the value stored at position `index` in the set. $O(1)$.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

`values(struct EnumerableSet.Bytes4Set set) → bytes4[]` internal #

Return the entire set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

`values(struct EnumerableSet.Bytes4Set set, uint256 start, uint256 end) → bytes4[]` internal #

Return a slice of the set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

 **OpenZeppelin Docs** `add(struct EnumerableSet.AddressSet set, address value) → bool` internal #

Add a value to a set. $O(1)$.

Returns true if the value was added to the set, that is if it was not already present.

`remove(struct EnumerableSet.AddressSet set, address value) → bool` internal #

Removes a value from a set. $O(1)$.

Returns true if the value was removed from the set, that is if it was present.

`clear(struct EnumerableSet.AddressSet set)` internal #

Removes all the values from a set. $O(n)$.

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncallable if the set grows to the point where clearing it consumes too much gas to fit in a block.

`contains(struct EnumerableSet.AddressSet set, address value) → bool` internal #

Returns true if the value is in the set. $O(1)$.

`length(struct EnumerableSet.AddressSet set) → uint256` internal #

Returns the number of values in the set. $O(1)$.

`at(struct EnumerableSet.AddressSet set, uint256 index) → address` internal #

 Returns the value stored at position `index` in the set. $O(1)$.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

values(struct EnumerableSet.AddressSet set) → address[]

internal #

Return the entire set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

values(struct EnumerableSet.AddressSet set, uint256 start, uint256 end) → address[]

internal #

Return a slice of the set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

add(struct EnumerableSet.UintSet set, uint256 value) → bool

internal #

Add a value to a set. $O(1)$.

 Returns true if the value was added to the set, that is if it was not already present.

OpenZeppelin Docs

```
remove(struct EnumerableSet.UintSet set, uint256 value) → bool internal #
```

Removes a value from a set. $O(1)$.

Returns true if the value was removed from the set, that is if it was present.

```
clear(struct EnumerableSet.UintSet set) internal #
```

Removes all the values from a set. $O(n)$.

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncalleable if the set grows to the point where clearing it consumes too much gas to fit in a block.

```
contains(struct EnumerableSet.UintSet set, uint256 value) → bool internal #
```

Returns true if the value is in the set. $O(1)$.

```
length(struct EnumerableSet.UintSet set) → uint256 internal #
```

Returns the number of values in the set. $O(1)$.

```
at(struct EnumerableSet.UintSet set, uint256 index) → uint256 internal #
```

Returns the value stored at position `index` in the set. $O(1)$.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

- `index` must be strictly less than `Memory.length`.

values(struct EnumerableSet.UintSet set) → uint256[]

internal #

Return the entire set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

values(struct EnumerableSet.UintSet set, uint256 start, uint256 end) → uint256[]

internal #

Return a slice of the set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

add(struct EnumerableSet.StringSet set, string value) → bool

internal #

Add a value to a set. O(1).

Returns true if the value was added to the set, that is if it was not already present.

remove(struct EnumerableSet.StringSet set, string value) → bool

internal #



Removes a value from a set. $O(1)$.

Returns true if the value was removed from the set, that is if it was present.

clear(struct EnumerableSet.StringSet set)

internal #

Removes all the values from a set. $O(n)$.

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncallable if the set grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableSet.StringSet set, string value) → bool

internal #

Returns true if the value is in the set. $O(1)$.

length(struct EnumerableSet.StringSet set) → uint256

internal #

Returns the number of values on the set. $O(1)$.

at(struct EnumerableSet.StringSet set, uint256 index) → string

internal #

Returns the value stored at position `index` in the set. $O(1)$.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

values(struct EnumerableSet.StringSet set) → string[]

internal #



Return the entire set in an array

⚠ This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

values(struct EnumerableSet.StringSet set, uint256 start, uint256 end) → string[]

internal #

Return a slice of the set in an array

⚠ This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

add(struct EnumerableSet.BytesSet set, bytes value) → bool

internal #

Add a value to a set. O(1).

Returns true if the value was added to the set, that is if it was not already present.

remove(struct EnumerableSet.BytesSet set, bytes value) → bool

internal #

Removes a value from a set. O(1).

Returns true if the value was removed from the set, that is if it was present.

OpenZeppelin Docs

internal #

Removes all the values from a set. $O(n)$.

 Developers should keep in mind that this function has an unbounded cost and using it may render the function uncalleable if the set grows to the point where clearing it consumes too much gas to fit in a block.

contains(struct EnumerableSet.BytesSet set, bytes value) → bool

internal #

Returns true if the value is in the set. $O(1)$.

length(struct EnumerableSet.BytesSet set) → uint256

internal #

Returns the number of values on the set. $O(1)$.

at(struct EnumerableSet.BytesSet set, uint256 index) → bytes

internal #

Returns the value stored at position `index` in the set. $O(1)$.

Note that there are no guarantees on the ordering of values inside the array, and it may change when more values are added or removed.

Requirements:

- `index` must be strictly less than `Memory.length`.

values(struct EnumerableSet.BytesSet set) → bytes[]

internal #

Return the entire set in an array



This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

```
values(struct EnumerableSet.BytesSet set, uint256 start, uint256 end) → internal #
bytes[]
```

Return a slice of the set in an array

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

Heap



```
import "@openzeppelin/contracts/utils/structs/Heap.sol";
```



Library for managing binary heap that can be used as priority queue.

Heaps are represented as a tree of values where the first element (index 0) is the root, and where the node at index i is the child of the node at index $(i-1)/2$ and the parent of nodes at index $2i+1$ and $2i+2$. Each node stores an element of the heap.

The structure is ordered so that, per the comparator, each node has lower priority than its parent; as a consequence, the highest-priority value is at the root. This value can be looked up in constant time ($O(1)$) at `heap.tree[0]`. By default, the comparator is `Comparators.lt`,

which treats smaller values as higher priority (min-heap). Using `Comparators.gt` yields a max-heap.

The structure is designed to perform the following operations with the corresponding complexities:

- peek (get the highest priority value): $O(1)$
- insert (insert a value): $O(\log(n))$
- pop (remove the highest priority value): $O(\log(n))$
- replace (replace the highest priority value with a new value): $O(\log(n))$
- length (get the number of elements): $O(1)$
- clear (remove all elements): $O(1)$



This library allows for the use of custom comparator functions. Given that manipulating memory can lead to unexpected behavior. Consider verifying that the comparator does not manipulate the Heap's state directly and that it follows the Solidity memory safety rules.

Available since v5.1.

Functions

- [`peek\(self\)`](#)
- [`pop\(self\)`](#)
- [`pop\(self, comp\)`](#)
- [`insert\(self, value\)`](#)
- [`insert\(self, value, comp\)`](#)
- [`replace\(self, newValue\)`](#)
- [`replace\(self, newValue, comp\)`](#)
- [`length\(self\)`](#)
- [`clear\(self\)`](#)

Lookup the root element of the heap.

pop(struct Heap.Uint256Heap self) → uint256

internal #

Remove (and return) the root element for the heap using the default comparator.

- ⓘ All inserting and removal from a heap should always be done using the same comparator. Mixing comparator during the lifecycle of a heap will result in undefined behavior.

pop(struct Heap.Uint256Heap self, function (uint256,uint256) view returns (bool) comp) → uint256

internal #

Remove (and return) the root element for the heap using the provided comparator.

- ⓘ All inserting and removal from a heap should always be done using the same comparator. Mixing comparator during the lifecycle of a heap will result in undefined behavior.

insert(struct Heap.Uint256Heap self, uint256 value)

internal #

Insert a new element in the heap using the default comparator.

- ⓘ All inserting and removal from a heap should always be done using the same comparator. Mixing comparator during the lifecycle of a heap will result in undefined behavior.

insert(struct Heap.Uint256Heap self, uint256 value, function (uint256,uint256) view returns (bool) comp)

internal #

Insert a new element in the heap using the provided comparator.

All inserting and removal from a heap should always be done using the same comparator. Mixing comparator during the lifecycle of a heap will result in undefined behavior.

replace(struct Heap.Uint256Heap self, uint256 newValue) → uint256

internal #

Return the root element for the heap, and replace it with a new value, using the default comparator. This is equivalent to using `Heap.pop` and `Heap.insert`, but requires only one rebalancing operation.

i All inserting and removal from a heap should always be done using the same comparator. Mixing comparator during the lifecycle of a heap will result in undefined behavior.

replace(struct Heap.Uint256Heap self, uint256 newValue, function (uint256,uint256) view returns (bool) comp) → uint256

internal #

Return the root element for the heap, and replace it with a new value, using the provided comparator. This is equivalent to using `Heap.pop` and `Heap.insert`, but requires only one rebalancing operation.

i All inserting and removal from a heap should always be done using the same comparator. Mixing comparator during the lifecycle of a heap will result in undefined behavior.

length(struct Heap.Uint256Heap self) → uint256

internal #

Returns the number of elements in the heap.

clear(struct Heap.Uint256Heap self)

internal #

Removes all elements in the heap.



```
import "@openzeppelin/contracts/utils/structs/MerkleTree.sol";
```



Library for managing Merkle Tree data structures.

Each tree is a complete binary tree with the ability to sequentially insert leaves, changing them from a zero to a non-zero value and updating its root. This structure allows inserting commitments (or other entries) that are not stored, but can be proven to be part of the tree at a later time if the root is kept. See MerkleProof.

A tree is defined by the following parameters:

- Depth: The number of levels in the tree, it also defines the maximum number of leaves as $2^{**}depth$.
- Zero value: The value that represents an empty leaf. Used to avoid regular zero values to be part of the tree.
- Hashing function: A cryptographic hash function used to produce internal nodes. Defaults to Hashes.commutativeKeccak256.

i Building trees using non-commutative hashing functions (i.e. $H(a, b) \neq H(b, a)$) is supported. However, proving the inclusion of a leaf in such trees is not possible with the MerkleProof library since it only supports *commutative* hashing functions.

Available since v5.1.

Functions

- setup(self, treeDepth, zero)
- setup(self, treeDepth, zero, fnHash)
- push(self, leaf)

OpenZeppelin Docs

- [push\(self, leaf, fnHash\)](#)
- [update\(self, index, oldValue, newValue, proof\)](#)
- [update\(self, index, oldValue, newValue, proof, fnHash\)](#)
- [depth\(self\)](#)

Errors

- [MerkleTreeUpdateInvalidIndex\(index, length\)](#)
- [MerkleTreeUpdateInvalidProof\(\)](#)

```
setup(struct MerkleTree.Bytes32PushTree self, uint8 treeDepth, bytes32 zero) → bytes32 initialRoot internal #
```

Initialize a [MerkleTree.Bytes32PushTree](#) using [Hashes.commutativeKeccak256](#) to hash internal nodes. The capacity of the tree (i.e. number of leaves) is set to `2**treeDepth`.

Calling this function on MerkleTree that was already setup and used will reset it to a blank state.

Once a tree is setup, any push to it must use the same hashing function. This means that values should be pushed to it using the default [push](#) function.

 The zero value should be carefully chosen since it will be stored in the tree representing empty leaves. It should be a value that is not expected to be part of the tree.

```
setup(struct MerkleTree.Bytes32PushTree self, uint8 treeDepth, bytes32 zero, function (bytes32,bytes32) view returns (bytes32) fnHash) → bytes32 initialRoot internal #
```

Same as [setup](#), but allows to specify a custom hashing function.



Once a tree is setup, any push to it must use the same hashing function. This means that values should be pushed to it using the custom push function, which should be the same one as used during the setup.



Providing a custom hashing function is a security-sensitive operation since it may compromise the soundness of the tree.



Consider verifying that the hashing function does not manipulate the memory state directly and that it follows the Solidity memory safety rules. Otherwise, it may lead to unexpected behavior.

```
push(struct MerkleTree.Bytes32PushTree self, bytes32 leaf) → uint256  
index, bytes32 newRoot
```

internal #

Insert a new leaf in the tree, and compute the new root. Returns the position of the inserted leaf in the tree, and the resulting root.

Hashing the leaf before calling this function is recommended as a protection against second pre-image attacks.

This variant uses `Hashes.commutativeKeccak256` to hash internal nodes. It should only be used on merkle trees that were setup using the same (default) hashing function (i.e. by calling [the default setup](#) function).

```
push(struct MerkleTree.Bytes32PushTree self, bytes32 leaf, function  
(bytes32,bytes32) view returns (bytes32) fnHash) → uint256 index,  
bytes32 newRoot
```

internal #

Insert a new leaf in the tree, and compute the new root. Returns the position of the inserted leaf in the tree, and the resulting root.

Hashing the leaf before calling this function is recommended as a protection against second pre-image attacks.



This variant uses a custom hashing function to hash internal nodes. It should only be called with the same function as the one used during the initial setup of the merkle tree.

```
update(struct MerkleTree.Bytes32PushTree self, uint256 index, bytes32  
oldValue, bytes32 newValue, bytes32[] proof) → bytes32 oldRoot, bytes32 internal #  
newRoot
```

Change the value of the leaf at position `index` from `oldValue` to `newValue`. Returns the recomputed "old" root (before the update) and "new" root (after the update). The caller must verify that the reconstructed old root is the last known one.

The `proof` must be an up-to-date inclusion proof for the leaf being updated. This means that this function is vulnerable to front-running. Any `RLP.push` or `MerkleTree.update` operation (that changes the root of the tree) would render all "in flight" updates invalid.

This variant uses `Hashes.commutativeKeccak256` to hash internal nodes. It should only be used on merkle trees that were setup using the same (default) hashing function (i.e. by calling [the default setup](#) function).

```
update(struct MerkleTree.Bytes32PushTree self, uint256 index, bytes32  
oldValue, bytes32 newValue, bytes32[] proof, function (bytes32,bytes32) internal #  
view returns (bytes32) fnHash) → bytes32 oldRoot, bytes32 newRoot
```

Change the value of the leaf at position `index` from `oldValue` to `newValue`. Returns the recomputed "old" root (before the update) and "new" root (after the update). The caller must verify that the reconstructed old root is the last known one.

The `proof` must be an up-to-date inclusion proof for the leaf being update. This means that this function is vulnerable to front-running. Any `RLP.push` or `MerkleTree.update` operation (that changes the root of the tree) would render all "in flight" updates invalid.

This variant uses a custom hashing function to hash internal nodes. It should only be called with the same function as the one used during the initial setup of the merkle tree.

Tree's depth (set at initialization)

MerkleTreeUpdateInvalidIndex(uint256 index, uint256 length)

error #

Error emitted when trying to update a leaf that was not previously pushed.

MerkleTreeUpdateInvalidProof()

error #

Error emitted when the proof used during an update is invalid (could not reproduce the side).

Time



```
import "@openzeppelin/contracts/utils/types/Time.sol";
```



This library provides helpers for manipulating time-related objects.

It uses the following types:

- `uint48` for timepoints
- `uint32` for durations

While the library doesn't provide specific types for timepoints and duration, it does provide:

- a `Delay` type to represent duration that can be programmed to change value automatically at a given point

- additional helper functions

Functions

- [`timestamp\(\)`](#)
- [`blockNumber\(\)`](#)
- [`toDelay\(duration\)`](#)
- [`getFull\(self\)`](#)
- [`get\(self\)`](#)
- [`withUpdate\(self, newValue, minSetback\)`](#)
- [`unpack\(self\)`](#)
- [`pack\(valueBefore, valueAfter, effect\)`](#)

`timestamp()` → **`uint48`**

internal #

Get the block timestamp as a Timepoint.

`blockNumber()` → **`uint48`**

internal #

Get the block number as a Timepoint.

`toDelay(uint32 duration)` → **`Time.Delay`**

internal #

Wrap a duration into a Delay to add the one-step "update in the future" feature

getFull(Time.Delay self) → uint32 valueBefore, uint32 valueAfter, uint48 internal #

Get the current value plus the pending value and effect timepoint if there is a scheduled change. If the effect timepoint is 0, then the pending value should not be considered.

get(Time.Delay self) → uint32 internal #

Get the current value.

withUpdate(Time.Delay self, uint32 newValue, uint32 minSetback) → Time.Delay updatedDelay, uint48 effect internal #

Update a Delay object so that it takes a new duration after a timepoint that is automatically computed to enforce the old delay at the moment of the update. Returns the updated Delay object and the timestamp when the new delay becomes effective.

unpack(Time.Delay self) → uint32 valueBefore, uint32 valueAfter, uint48 effect internal #

Split a delay into its components: valueBefore, valueAfter and effect (transition timepoint).

pack(uint32 valueBefore, uint32 valueAfter, uint48 effect) → Time.Delay internal #

pack the components into a Delay object.

< ERC6909

Previous Page

Cryptography >

Next Page